R9 — agent walkthrough

Every feature shipped this year, driven in a browser at 390×844

streetfight · 28–29 August 2026 · tested against 6cf5608

Screenshot edition — the same report as docs/r9_agent_walkthrough_2026-08-29.md, with the evidence in it.

3	30	44	26	13
blocker	major	minor	cosmetic	question

117 findings across all 28 R9 checklist lines. No code was changed and nothing was fixed.

What this is, and what it is not

This is the agent pass R9 asks for: thirteen agents, one per group of checklist lines, each driving a real Chromium at 390×844 with DPR 3 and touch, against the running app. Every screenshot in this document was taken by an agent during that pass, at that viewport.

It is not the gate. R9 is explicit that the gate is Charles doing the same pass by hand on his own phone, because an agent cannot judge whether a flow makes sense to someone who has never seen it, or whether a button is reachable one-handed with a box of armbands in the other. This went first so that time is spent on judgement rather than on finding 500s.

Three limitations, stated up front

CharlesBot was stubbed. There is no OPENROUTER_API_KEY in the test container. A local stand-in served schema-conforming replies, with `backend.vision_client.OPENROUTER_URL` rebound in the launching process only — no repository file was touched. So everything downstream of the model is genuinely exercised: queue annotation, the ranked candidate list, the zoom indicator, auto-actions, escalation, the confidence gate, appeals and the replay workbench. Whether a real model reads a real photograph correctly is not, and remains what #4 and R1/R2 are for.

The camera is Chromium's fake device. The green field in most screenshots is a test pattern, not a bug — no shot photo in this pass contains a person. One agent worked around this for the loot flow by rendering a real item QR to a Y4M and feeding it to the browser as the camera, so the real scanner path was exercised.

Headless. The phone's own lock button, real GPS drift, real cameras and real thumbs are out of reach. Where that mattered the agent said so rather than guessing.

Freshness

The pass ran against `6cf5608`. Five PRs (#165–#169) landed on master while it was running. #168 reworks the kit-check page agent A8 walked — its three findings were re-checked against master and all still stand. #167 renames the admin version footer A12 measured, which makes A12's line references stale though its finding is unaffected. Anything here should still be confirmed against the current head before it is acted on.

Coverage

All 28 checklist lines were driven. Verdicts are the testing agent's own.

Checklist line	Verdict	Agent
Join a team via QR/link and pick an outfit at /pick (#10): ranked outfit list, canonical-first ordering, colour swatches, pagination.	WORKS WITH CAVEATS	A1
The "recommended" vs "not ideal" outfit badges and the "show non-recommended outfits" reveal link.	WORKS	A1
The name-then-confirm flow — entering a name, seeing the committed outfit, and the outfit becoming locked in.	WORKS WITH CAVEATS	A1
An admin clearing a player's outfit so they can pick again.	WORKS WITH CAVEATS	A2
The identity workbench / AdminIdentity.js: viewing a player's effective appearance and any overrides, and recording an override for a misdressed player so they stay distinguishable from their teammates.	WORKS WITH CAVEATS — the substance is right, the mobile layout is not	A2
Renaming a team from the admin dashboard.	WORKS	A2
Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).	WORKS WITH CAVEATS	A3
Taking a shot photo of another player, including the on-screen crosshair overlay.	WORKS WITH CAVEATS	A4
The shot status bubble after taking a shot — every visual state it can be in, and that it stays on screen rather than disappearing.	WORKS WITH CAVEATS	A4
The screen staying awake during play (R3 — Screen Wake Lock), and that the phone's own lock button still turns the screen off on request.	WORKS WITH CAVEATS (the "lock button" half is NOT TESTABLE HERE)	A5
The map view, including drop/circle locations on the live Westminster venue map (#12).	WORKS WITH CAVEATS	A5
The ticker feed for game announcements.	WORKS	A5
Appealing a resolved shot as either shooter or target (R8), including seeing the appeal budget run out.	WORKS WITH CAVEATS	A6
The contested-shots list an appeal reopens (R8), separate from the main queue.	WORKS	A6
`User-facing copy says "CharlesBot", never "AI", anywhere a verdict or explanation is shown (#1, #2 — "CharlesBot thinks: hit on *name*").`	WORKS WITH CAVEATS	A7
The reference-photo kit check at /admin/reference (R7): capturing a player's photo at the door and reading the resulting verdict, including the "unreadable photo identifies nobody" case.	WORKS WITH CAVEATS	A8
The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.	WORKS WITH CAVEATS	A9
Recording a "hit a bystander" outcome on a shot.	WORKS	A9
Toggling ai_shot_review_enabled, ai_auto_actions_enabled, ai_escalation_enabled and ai_resolve_everything_enabled, and confirming each one actually changes queue behaviour (not just the toggle state)	WORKS (all four gates observably change behaviour, not just state)	A10
Not re-reviewing an already-reviewed shot when the AI toggle is switched on and off again	WORKS	A10
Running an escalated review by hand on a shot (#11 — "Run escalated review")	WORKS WITH CAVEATS	A10
The per-shot map (R5) showing GPS accuracy and the shooter's compass heading.	WORKS	A11
Admin shot history and notes on a player.	WORKS WITH CAVEATS — the **notes** half works well; the **history "on a player"** half…	A11
Downloading all shot images as a zip.	WORKS WITH CAVEATS	A11
The admin nav (finger-sized button row) on a real phone screen, not just a desktop browser.	WORKS WITH CAVEATS	A12
The running app version shown on admin pages, and that it matches what is actually deployed.	WORKS WITH CAVEATS	A12
The replay workbench at /admin/replay (R1) — trialling a prompt/zoom/schema change against real shots without it touching stored data.	WORKS	A12
Cuts across both: confirm the game can be reset (resetdb) and replayed from a clean state without any of the above breaking, since that is exactly what happens between the dry run and the night itself.	WORKS WITH CAVEATS	A13

Blockers (3)

Three. One makes the game incoherent for a player and is invisible when it happens; the other two sit around the reset and decide when the print run can be scheduled.

Every join QR printed before a `resetdb` is dead, and says so with a raw UUID

BLOCKER *found by agent A13*

Cuts across both: confirm the game can be reset (resetdb) and replayed from a clean state without any of the above breaking, since that is exactly what happens between the dry run and the night itself.

What happens

a team join QR encodes `(game_id, team_id, slot=None)`, HMAC-signed (`backend/join_codes.py:104`). `reset_db` drops the tables and re-mints the ten sample teams with **fresh `uuid4()`s** (`backend/reset_db.py:30`), and any hand-made game is destroyed outright. The signature still verifies — `SECRET_KEY` is unchanged — so the code parses, and then the team lookup 404s. Scanning a pre-reset code lands the player on `/pick` and shows a dismissible popup reading, in full: `Team 7979f5ca-0a95-4bdb-a589-f8b2b2dd56c0 not found` behind which is the ordinary onboarding page telling them to "Scan your team's join QR code with your camera app…" — i.e. to scan the same dead code again. `a13_08_post_old_join_qr.png`. API: `GET /api/join_options` → 400/404 `{"detail":"Team … not found"}`.

How to reproduce

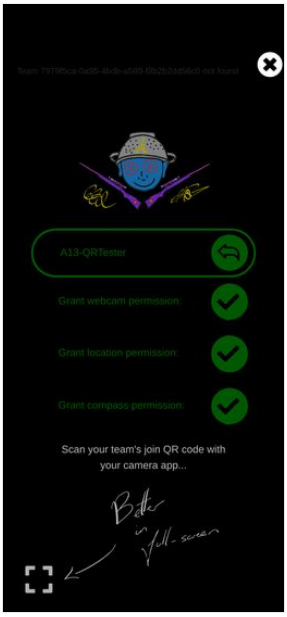
generate join QRs for a game, `npm run resetdb`, scan one.

Where

`backend/reset_db.py:30` (`SAMPLE_TEAMS = [(uuid4(), …)]`), `backend/main.py` `_decoded_join_code` / `join_options`, popup text from `identity_admin.join_options`'s 404 detail.

Why it matters for the 19th

this is a scheduling fact, not just a bug. Roadmap #8 is a print run. If the join QRs are printed before the last `resetdb`, every one of them is waste paper and every player is stuck at a screen that tells them to rescan the code that just failed. **The print run has to happen after the final reset**, or the final reset has to be `admin_reset_game` (see below) rather than `resetdb`. Separately, a player-facing error that names a UUID and offers no next step is the wrong message for a dark street.



a13_08_post_old_join_qr.png

After a `resetdb` the sample game cannot generate join QR codes at all

BLOCKER *found by agent A13*

Cuts across both: confirm the game can be reset (resetdb) and replayed from a clean state without any of the above breaking, since that is exactly what happens between the dry run and the night itself.

What happens

`MAKE_DEBUG_ENTRIES` creates **ten** teams. The team channel (the hat, `TEAM_CHANNEL`) has **seven** colours. `admin_join_qr_codes` therefore fails outright: `400 {"detail":"10 teams need a colour each, but 'hat' only has 7"}` In the admin UI this is the "Generate" button under the game on the admin home; pressing it produces a red **API errors** banner across the top of the page and no QR codes at all — `a13_23_sample_game_join_qr_400.png`. There is no partial result and no hint that deleting teams would fix it.

How to reproduce

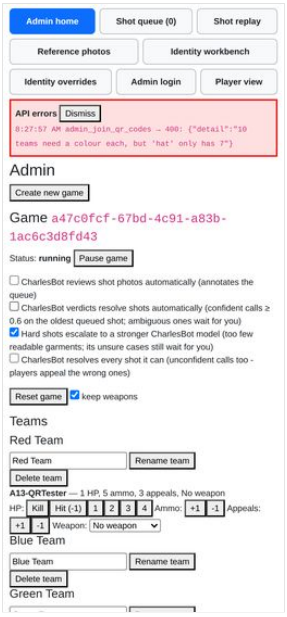
`npm run resetdb`, open `/admin`, press **Generate**.

Where

`backend/reset_db.py:18` (`TEAM_COLOURS`, ten entries) versus `backend/identity/allocation.py:166` (`assign_team_colours` raises when `len(existing) > len(all_labels)`); surfaced by `react-ui/src/JoinQRCodes.js:23`.

Why it matters for the 19th

the *default* state after every reset is one where the single, active, ready-to-play sample game cannot produce the artefact the night depends on. Charles will not hit it if he makes his own game with ≤7 teams, but he will hit it the moment he tries to use the game that `resetdb` hands him — which is the obvious thing to do at 6pm on the 19th. Either the sample game should be created with at most seven teams, or the error should tell the admin to delete some.



a13_23_sample_game_join_qr_400.png

Every cookieless client shares one player identity

BLOCKER *found by agent A0*

Cuts across everything: the player's browser session

What happens

`backend/user_id.py` keys its `no_cookie_clients` dict on `request.client.host` — the TCP peer IP. Ten independent browser contexts, each with its own empty cookie jar, all hitting the app at once were assigned **the same** player UUID: `ctx0..ctx9 -> b0234858-a04a-4bd3-8747-8cb012e9e4ba (distinct: 1 of 10)` They are not ten players; they are one player with ten screens. Whoever's `set_name` lands last wins, they share ammo, HP, appeals and shot history.

How to reproduce

ten fresh Playwright contexts, `goto /api/hello`, then `GET /api/my_id` concurrently. Script and output in `scratch/driver` (see the coordinator transcript).

Where

`backend/user_id.py:13` (`temporary_id = request_or_ws.client.host`) and `assign_new_ID` at the bottom of the same file. The comment there explains the intent — hold one UUID for concurrent cookieless requests *from one client* — and the mechanism is right; the **key** is wrong.

Why it matters for the 19th

every player's first request is cookieless. The window is only as long as it takes a client to come back with its cookie, but at kick-off that is exactly when everybody loads the app at once. Two players who overlap in that window become the same player, and there is no error — the game just quietly misbehaves for them all night. Worth noting the dry run on 30 August is the first realistic chance to hit this, and with a handful of people starting at slightly different moments it may well *not* reproduce, which is what makes it worth fixing rather than watching for.

Major findings (30)

Things likely to cost the admin or a player real trouble on the night.

The pagination controls are unstyled default browser buttons, 41x24 and 63x24 px

MAJOR *found by agent A1*

Join a team via QR/link and pick an outfit at /pick (#10): ranked outfit list, canonical-first ordering, colour swatches, pagination.

What happens

`Previous` (63x24) and `Next` (41x24) are bare `<button>`s with no class, so they render as light-grey OS chrome at the two bottom corners of an otherwise black, custom-styled page. They are roughly half the 44px iOS touch minimum in height and about a quarter of it in width for `Next`, and they sit hard against the screen edges.

How to reproduce

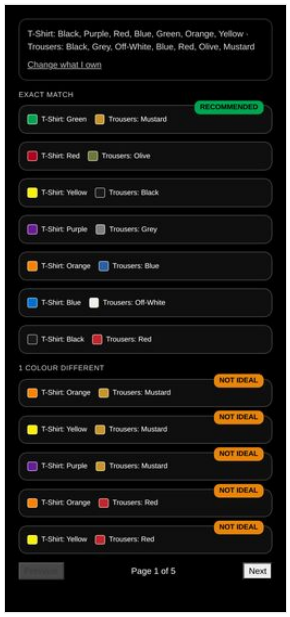
`/pick?j=<team code>` → tick everything → "Show me outfits" → "Show more outfits" → scroll to the bottom. See `A1_10_showall_pagination.png`.

Where

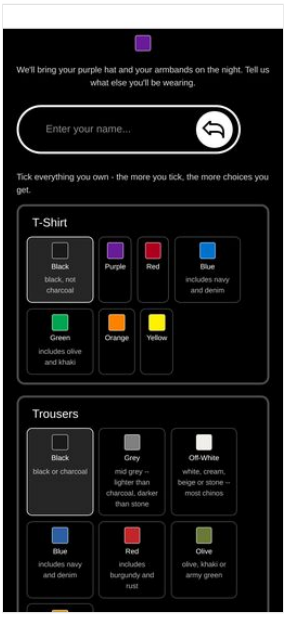
`react-ui/src/PickOutfit.js:580-600` (the `.pagination` block — the two buttons get no `className`); `react-ui/src/PickOutfit.module.css:260`.

Why it matters for the 19th

this is the only way to reach 37 of the 49 outfits, it will be operated one-handed in a pub doorway, and it directly contradicts the house style in `CLAUDE.md` ("big, bulky, unmissable buttons"). The same applies to the 85x24 "Yes, I'm sure" button in the no-outfits empty state (`A1_25_empty_state.png`).



A1_10_showall_pagination.png



A1_25_empty_state.png

A name typed but not submitted leaves "Lock in my choice" dead

MAJOR *found by agent A1*

The name-then-confirm flow — entering a name, seeing the committed outfit, and the outfit becoming locked in.

What happens

`NameEntry` only calls `set_name` on Enter or on the circular arrow button. `PickOutfitForm`'s `name` state (and therefore `hasName`) only updates from that callback. So a player who types "A1-Alice" into the box, ticks "I will wear this on the night", and taps the big white button gets nothing: the button is disabled, the name is sitting visibly in the box, and the only explanation is one line of body text above the button reading "Enter your name above first - an outfit has to belong to somebody."

How to reproduce

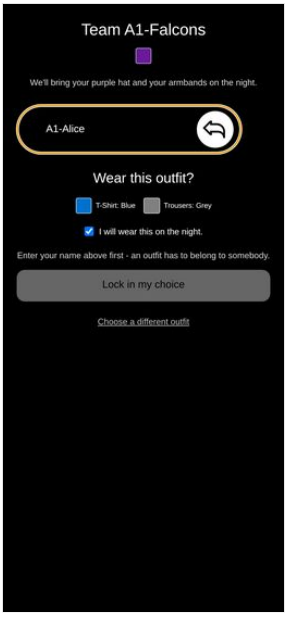
`/pick?j=…` → pick any outfit → type a name but do **not** press Enter → tick the box → the button stays greyed. `A1_17_typed_name_not_submitted.png` shows exactly this: name visible, box outlined amber, message says to enter a name, button grey.

Where

`react-ui/src/PickOutfit.js:449-455` (`nameEntry`) + `:302-313` (the `hasName` gate); `react-ui/src/OnboardingView.js:47-87` (`NameEntry` — commit only on Enter / arrow-button).

Why it matters for the 19th

phone keyboards show "return"/"go", but plenty of people tap Done or just tap away. This is the single most likely place for the door queue to stall, and the on-screen text actively contradicts what the player sees. (On the onboarding screen this is less harmful because there is nothing else to do; here it silently blocks the commit.)



A1_17_typed_name_not_submitted.png

The name requirement is client-side only — `pick_outfit` will place a nameless player

MAJOR *found by agent A1*

The name-then-confirm flow — entering a name, seeing the committed outfit, and the outfit becoming locked in.

What happens

`POST /pick_outfit` with a valid team code, a valid appearance and `confirmed: true` from a session with **no name** returns 200, creates the `User` row, puts them in the team and burns identity slot 18. `user_info` afterwards: `{"name": null, "team_name": "A1-Falcons", "identity_slot": 18}`. The admin's player list then shows *unnamed (no team)* / unnamed rows (`A1_31_admin_qr_cards.png`, Players section).

How to reproduce

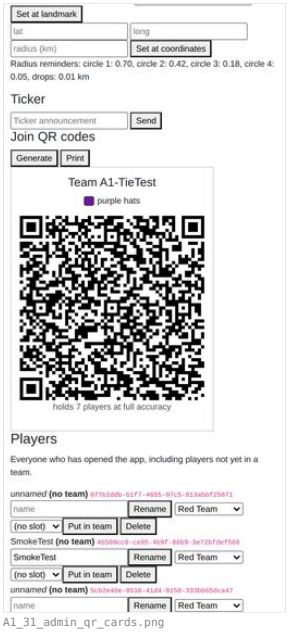
from a fresh cookie jar, `fetch("/api/pick_outfit", {method:"POST", body: JSON.stringify({data: "<team code>", wardrobe:{}, appearance:{hat:"purple", tshirt:"green", trousers:"mustard", armbands:"blue"}, confirmed:true})})`.

Where

`backend/identity_admin.py:726-760` (`pick_outfit` checks `confirmed` but never the name); the gate lives only in `react-ui/src/PickOutfit.js:310`.

Why it matters for the 19th

`join_options` goes to real trouble not to create a `User` row for link-preview bots, and then `pick_outfit` will create one for anybody. A nameless slot-holder consumes one of the 7 slots a hat colour affords, is invisible in the roster, and cannot be matched to a person by the door staff or by `ReferencePhotos`. Two lines of defence are cheap here; today there are none. (Note: my own test session is one such nameless slot-18 holder in game `8836c645-…`, team A1-Falcons — that is my doing, not a pre-existing state.)



A1_31_admin_qr_cards.png

Scanning the wrong team's QR after locking in shows you the wrong team and the wrong hat colour

MAJOR *found by agent A1*

The name-then-confirm flow — entering a name, seeing the committed outfit, and the outfit becoming locked in.

What happens

`join_options` returns `you` for any caller who is in the code's **game**, regardless of which team the code belongs to. `PickOutfit` then renders `ResultScreen` whenever `you.slot` is set. So A1-Alice — a Falcon in a purple hat — scanning the A1-Badgers card sees a page headed "Team A1-Badgers", a **red** hat swatch, "We'll bring your **red** hat and your armbands on the night", then "You're set / T-Shirt blue / Trousers grey / This is final. Screenshot it."

How to reproduce

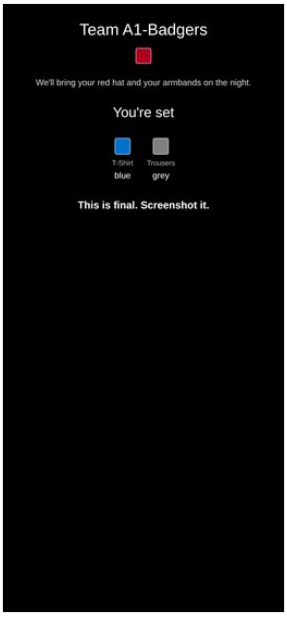
lock in an outfit with team A's code, then open `/pick?j=<team B's code>` in the same session. `A1_21_other_team_after_locked.png`.

Where

`backend/identity_admin.py:593` + `:603` (caller looked up across `game_users`, not the team's roster) and `react-ui/src/PickOutfit.js:641-646` (`alreadyPicked` never compares `joinData.team_id` with the player's team).

Why it matters for the 19th

on the night there will be one printed card per team on a table and people will scan the nearest one. The page tells them, confidently and finally, that they are in the wrong team wearing the wrong hat — and invites them to screenshot it. It should say "you're already set up in A1-Falcons" instead.



A1_21_other_team_after_locked.png

There is no per-player shot history — only a 40-deep Next/Previous pager over every shot in every game

MAJOR *found by agent A11*

Admin shot history and notes on a player.

What happens

the only history is `ShotQueue`'s "Show adjudicated shots" checkbox, which appends the checked shots to the same single-shot pager. There is no list, no filter, no search, and no link from a player anywhere to their shots. To answer "what has happened to A11-Target tonight?" the admin taps

How to reproduce

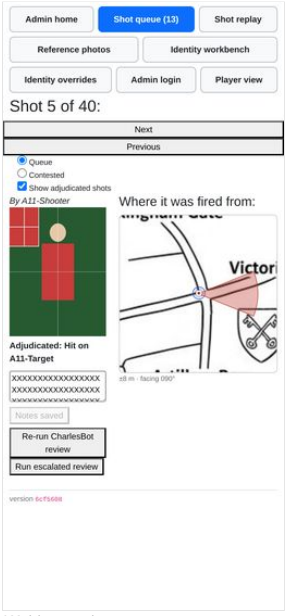
`/admin/shots` → tick "Show adjudicated shots" → try to find one named player's shots. `A11-history-view.png`.

Where

`react-ui/src/ShotQueue.js:628` (`ShotQueuePanel`, index-based navigation), `backend/main.py` — no `admin_get_user_shots`; `backend/admin_interface.py:967` `get_shots_ids(include_checked)`.

Why it matters for the 19th

the moment a player says "that hit was wrong" or "I've been shot three times and only lost one HP", the admin has to find that player's shots under time pressure on a phone. At one shot per tap with a ~1 s image load between taps, a 150-shot night makes this unusable. This is also the exact query an appeal (R8) makes you want to run.



A11-history-view.png

The adjudicated-shot history never says *when* a shot was fired

MAJOR *found by agent A11*

Admin shot history and notes on a player.

What happens

the history view shows `By A11-Shooter`, the photo, `Adjudicated: Hit on A11-Target`, the map caption and the notes box — and no timestamp anywhere. `time_created` is already in the payload (`ShotModel.time_created`, `backend/model.py:535`); the queue simply never renders it. The player-facing history *does* show a time (`formatShotTime`, `ShotHistory.js`), so the admin has strictly less information than the players do.

How to reproduce

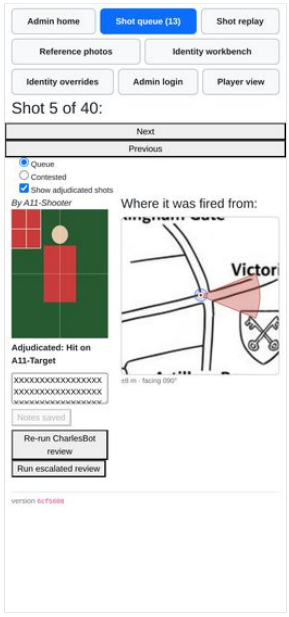
tick "Show adjudicated shots", look at any checked shot. `A11-history-view.png`.

Where

`react-ui/src/ShotQueue.js:806` (`<em>By {shot.user.name}</em>` — the only header line) and the `Adjudicated:` line just below it.

Why it matters for the 19th

an adjudication argument is almost always about ordering — was he already out when this was taken? Two shots by the same shooter with the same photo and the same verdict are literally indistinguishable in this view. The fix is rendering a field that is already being fetched.



A11-history-view.png

Four of the seven admin pages scroll horizontally at 390px

MAJOR *found by agent A12*

The admin nav (finger-sized button row) on a real phone screen, not just a desktop browser.

What happens

`document.documentElement.scrollWidth` at a 390px viewport: `/admin/reference` **778px**, `/admin/identity-overrides` **778px**, `/admin/identity` 610px, `/admin` 408px, `/admin/shots` 401px (`/admin/replay` and `/admin/login` are clean at 390px). The two 778px pages are the same cause: the `GameSelector <select>` renders its longest option (`a47c0fcf (Red Team, Blue Team, Green Team, Yellow Team, Purple Team, Orange Team, Pink Team, Cyan Team, Brown Team, Black Team)`) with no width cap, so a native select 769px wide doubles the page. Swipe right and you get a blank white page with the tail of one dropdown and no nav at all — see `A12-reference-scrolled-right.png` (scrollX 388). The sample game has ten teams, which is exactly what a real game night looks like. The lesser offenders: `/admin/identity` is the scheme table (582px), `/admin` the expanded MapView mini-map (408px), `/admin/shots` the Queue / Contested / "Show adjudicated shots" labels (390px wide starting at x=11).

How to reproduce

log in as admin at 390x844, go to `/admin/reference` with a game that has several teams, swipe left-to-right.

Where

`react-ui/src/ReferencePhotos.js:376-396` and `react-ui/src/AdminIdentity.js:657-677` — the same `GameSelector` component copy-pasted into both files, neither with a `max-width`.

Why it matters for the 19th

the door kit-check page (`/admin/reference`) is used standing up with a queue of players in front of you; a page that slides sideways under your thumb and hides the nav is the wrong thing to be fighting. The fix is one CSS line, and it is the same line in two places.

A12-reference-scrolled-right.png

A phone left open across a reset never notices, and never self-heals

MAJOR *found by agent A13*

Cuts across both: confirm the game can be reset (resetdb) and replayed from a clean state without any of the above breaking, since that is exactly what happens between the dry run and the night itself.

What happens

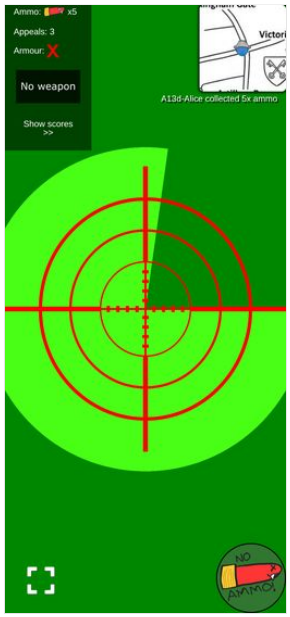
with a player's app open and mid-game, I wiped the database. Sixty seconds later the page was still showing "Ammo x5", the old ticker line, the crosshair and the map — indistinguishable from a live game (`a13_21_live_page_60s_after_reset.png`, and `a13_04_post_player_no_reload.png` from the first reset, which still showed a "My shots >> 1" badge for shots that no longer exist). No error, no spinner, no prompt. The SSE stream stays healthy (`readyState = 1`, zero `onerror`) because keepalives keep arriving; but the user stream only emits when something *triggers* that user's event (`UserInterface.generate_user_updates`, `asyncio_triggers` keyed on `user_id`), and a wiped database triggers nothing. The app only re-fetches `user_info` on such an event (`UpdateListener.js` → `registerListener("user", …)` in `UserMode.js:147`), so it never looks again. Recovery is fine once you reload: the session cookie's UUID is reused, `get_user_id` (`backend/user_id.py`) hands it back, `UserInterface._make_user` recreates a blank row with the same id, and the player gets clean onboarding — "Enter your name…", `a13_05_post_player_reload.png`. No crash, no 500, no infinite spinner, no ghost identity in the data. The stale *screen* is the whole problem. Admin contexts are unaffected: the admin flag lives in the signed session cookie, so after a reset the admin is still authed, the queue reads "Shot 0 of 0" and the admin home shows the single clean sample game (`a13_06`, `a13_07`).

How to reproduce

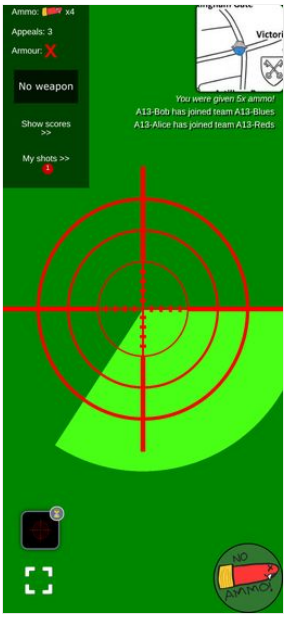
join a player, leave `/` open, run `resetdb`, watch for a minute.

Why it matters for the 19th

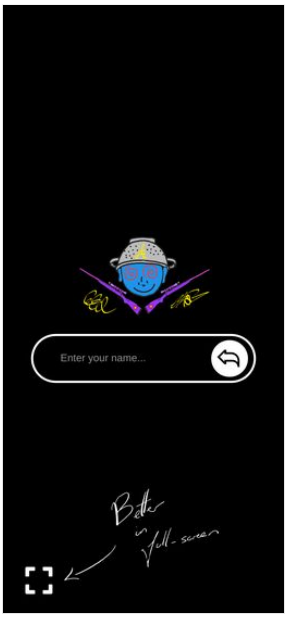
this is exactly the phone that was at the dry run. It does not fail loudly; it lies quietly. A player who never closes the tab between 30 August and the 19th will walk out believing they have five rounds and a live game. Whatever the fix (a version/epoch on the SSE stream, or a periodic `user_info` poll), the *operational* mitigation is worth writing on the run sheet: **everyone force-reloads after the final reset.**



a13_21_live_page_60s_after_reset.png



a13_04_post_player_no_reload.png



a13_05_post_player_reload.png

The first shot of the night renders as "Shot 0 of 1" and a blank panel

MAJOR *found by agent A13*

Cuts across both: confirm the game can be reset (resetdb) and replayed from a clean state without any of the above breaking, since that is exactly what happens between the dry run and the night itself.

What happens

on an empty queue the index is clamped to `shot_ids.length - 1`, which is **-1**, and it is never restored when the list refills: `js // react-ui/src/ShotQueue.js:652 const shownIdx = Math.min(currentShotIdx, shot_ids.length - 1); if (shownIdx !== currentShotIdx) setCurrentShotIdx(shownIdx);` With one shot in the list, `Math.min(-1, 0) === -1`, so no correction happens, the header renders `currentShotIdx + 1` = **0** (line 753), and `shotsInQueue[-1]` is `undefined`, so nothing is drawn. The tab counter still says "Shot queue (1)". `a13_29_first_shot_of_the_night.png` — "Shot queue (1)" above "Shot 0 of 1:" and an empty page. Pressing **Next** reveals the shot normally. Same bug on the Contested toggle: switching to Contested from an empty Queue shows "Shot 0 of 1" with nothing on it, so a lodged appeal looks like it never arrived (`a13_26`, `a13_27` — after **Next**, the full contested panel appears with the reason, the standing verdict and the appeal time).

How to reproduce

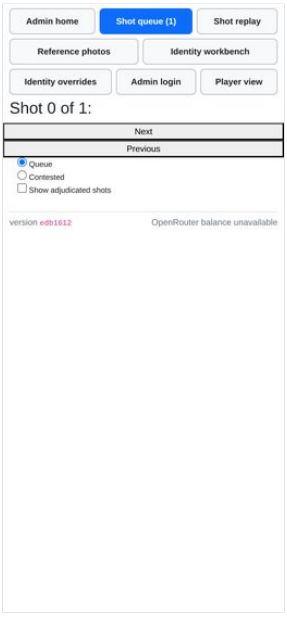
`resetdb`, open `/admin/shots` — "Shot 0 of 0" — leave it open, have a player fire the first shot. It never appears until you press Next.

Where

`react-ui/src/ShotQueue.js:652` and `:753`. Reproduced on the current revision `edb1612`.

Why it matters for the 19th

the admin sits on the shot queue from the moment the game starts, which on a fresh database means sitting on an empty one. The very first shot of the night arrives into a screen that shows nothing, with a counter that says there is one. An admin who does not think to press Next concludes the queue is broken.



a13_29_first_shot_of_the_night.png

Pressing Save after "Clear outfit" silently un-clears the player

MAJOR *found by agent A2*

An admin clearing a player's outfit so they can pick again.

What happens

`Clear outfit` nulls the slot on the server and the roster row updates correctly, but the editor panel underneath does **not** reset: its slot `<select>` still shows the slot the player just lost (and drops the "(current)" marker, so it now disagrees with the row above it without saying so). Pressing `Save` — the only other button in the panel, sitting right below it — posts that stale slot back through `admin_identity_set` and the player is re-assigned the outfit that was just thrown away.

How to reproduce

`/admin/identity-overrides` → pick the game → tap `A2-Dan` (slot 2) → `Clear outfit` → accept the confirm. Roster row now reads `none`. Without touching anything else, press `Save`. Row reads `2` again. Observed exactly that: `Dan slot before clear: 2 / after clear: none / after a bare Save: 2`.

Where

`react-ui/src/AdminIdentity.js:408` — `PlayerEditor`'s initialising `useEffect` is keyed on `[player.user_id]` only, so a reload of the report for the *same* player never re-seeds `selectedSlot` / `channelOverrides`. `clearOutfit` (`:443`) calls `onSaved(updated)`, which keeps the same player selected. The same staleness hits the SuggestPanel (`:213`, keyed on `[player.user_id, channelNames.join(",")]`): after saving an override, its "fixed" dropdowns still show the *pre-save* colours (see `A2-identity-05b-override-saved-top.png`: tshirt override saved to red, fixed select still says yellow; and `A2-identity-07-collision-forced.png`: effective outfit purple/olive, fixed selects still red/red/red), so a `Suggest` run right after a save solves against the wrong outfit.

Why it matters for the 19th

this is exactly the door-desk sequence — clear the guest who turned up in the wrong shirt, then poke at the same panel — and the undo is invisible: the row says `none`, the panel says `2`, and the panel wins. The admin walks away believing the outfit is free while the player's join code will now bounce ("slot 2 is already used by A2-Dan").

A2-identity-05b-override-saved-top.png

A2-identity-07-collision-forced.png

A knocked-out player cannot collect a medpack from a link

MAJOR *found by agent A3*

Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).

What happens

`CollectItemFromQueryParam` is rendered inside `BulletCount`, and `UserMode` only renders `BulletCount` while the player is alive. So a knocked-out (or dead) player who opens an item URL gets nothing at all: no request is made, the `?d=…` stays in the address bar, hp stays 0. The knocked-out screen meanwhile says "Get a medkit quick!". The in-game camera scanner (`QRParser`, mounted inside `WebcamView`, which *is* rendered when dead) is unaffected — that path works.

How to reproduce

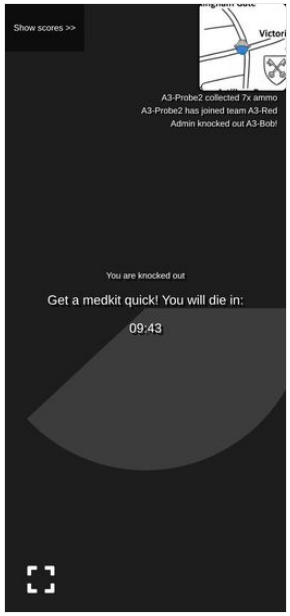
knock a player out (`admin_hit_user`), then navigate their browser to a medpack `encoded_url`. Screenshot `a3_25_bob_medpack_url_while_knocked_out.png`: still knocked out, URL still carries `?d=`. The same payload posted to `collect_item` directly returns 200 and revives them (`scratch/a3/run3.json`, steps `medpack_url_while_knocked_out` vs `medpack_api_while_knocked_out`).

Where

`react-ui/src/UserMode.js:74` (`isAlive ? <BulletCount…`) and `react-ui/src/BulletCount.js:106`.

Why it matters for the 19th

the roadmap's dry-run plan is "QR codes go out on WhatsApp, players follow links". A medpack delivered as a link is exactly the item a player needs while knocked out, and it is the one case where the link path is dead. Anyone who scans with the *phone's* camera app rather than in-game lands on the same URL and hits the same wall.



a3_25_bob_medpack_url_while_knocked_out.png

Every failure on the link path is completely silent

MAJOR *found by agent A3*

Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).

What happens

`CollectItemFromQueryParam` logs the response to the console and unconditionally `navigate("/")`. Player pages leave `apiErrorHandler` unset. So "already collected", "you are dead", "medpack on a live player", "you already have this weapon", "signature mismatch" and a 500 all look identical to the player: the app blinks back to the game screen with nothing changed and nothing said. The camera path at least flashes red and plays `error.mp3` on a 403 (`QRParser.js`), so the two routes to the same item give opposite feedback.

How to reproduce

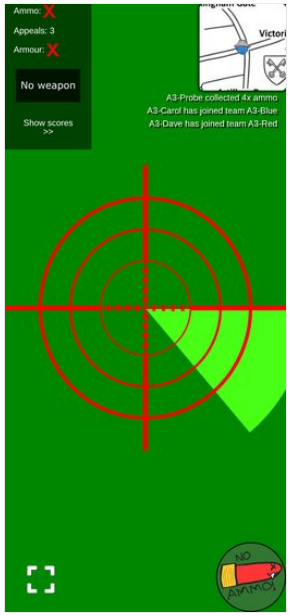
`a3_20_bob_second_scan_of_used_item.png` — Bob opened an already-collected ammo URL; ammo still shows ✗, no message, no flash, no ticker line.

Where

`react-ui/src/CollectItemsFromQueryParams.js:29-34`; `react-ui/src/utils.js:26-28`.

Why it matters for the 19th

"I scanned it and nothing happened" is the support call, and neither the player nor the admin can tell whether the code was already taken, the player was dead, or the QR was damaged.



a3_20_bob_second_scan_of_used_item.png

HP is not restored when an appeal is upheld, and nothing reminds the admin

MAJOR *found by agent A6*

Appealing a resolved shot as either shooter or target (R8), including seeing the appeal budget run out.

What happens

exactly what R8 decided ("HP is never auto-unwound... the admin repairs HP and ammo by hand"), but there is no affordance for the second half. Verified: Alice on Pewster (1 damage) hit Bob, Bob 10 → **9 HP**; Bob appealed; admin re-ruled Missed; appeal upheld and the appeal refunded — Bob still on **9 HP** (`a6_33_bob_hp_after_upheld.png`). The contested view has no HP control and no prompt; the admin has to remember, leave the queue, find the roster and fix it. If the shot was a knockout the player is still out.

How to reproduce

as above with any weapon that does damage.

Where

`backend/admin_interface.py:1192` `_settle_appeal` docstring; `react-ui/src/ShotQueue.js:407` `AppealDetails` (no HP affordance).

Why it matters for the 19th

an upheld appeal that leaves the player exactly as wounded as before is not a correction from the player's point of view, and this is the busiest, most distracted moment the admin will have all night.

a6_33_bob_hp_after_upheld.png

The shot photograph is 134×71 CSS pixels on a phone

MAJOR *found by agent A9*

The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.

What happens

the thing the admin is there to judge — a 2048×1080 photo of a person — is rendered at 134×71 px, while the map next to it gets roughly 520 px of the same 390 px screen (it overflows). Measured with `boundingBox()`: `SHOT IMG css 134×71, natural 2048×1080`. You cannot tell a hat colour from it, let alone a person; in `a9_queue_exactmatch_top.png` the fake camera feed is a green postage stamp under the shooter's name.

How to reproduce

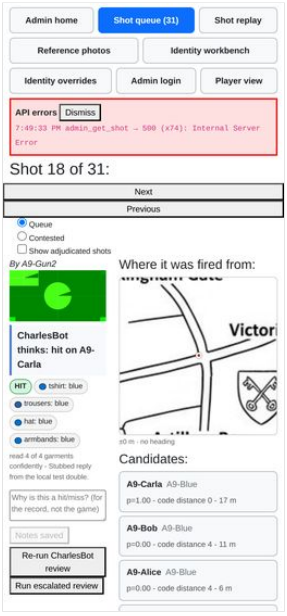
`/admin/shots` at 390×844, any shot. Measure `document.querySelector("img[alt='The next shot in the queue']").getBoundingClientRect()`.

Where

`react-ui/src/ShotQueue.js:804-860` wraps the photo in `<Row><Col>` … `<Col>` with no responsive breakpoint, so the two columns stay side by side at 390 px; the photo column gets ~134 px and `react-ui/src/ShotQueue.module.css:1-3` is `.shotImg { width: 100% }`.

Why it matters for the 19th

the whole queue exists so a human can look at the picture. On the phone the admin will actually be holding, they can't. Everything else on the page — the ranking, the flags, the zoom tag — is decoration around a picture too small to use.



a9_queue_exactmatch_top.png

The ranked list is read-only — acting on it means finding the same name again in a second list

MAJOR *found by agent A9*

The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.

What happens

the Candidates panel names the person and scores them, and then has no button. To record the hit the admin scrolls past the ranking into a *separate* roster grouped by team in join order, finds the same name among (in my game) twelve rows, and presses a 31×24 "Hit". `a9_queue_exactmatch_candidates.png` shows both lists in one screenshot: A9-Carla at the top of the ranking, and A9-Carla eleven rows down in the roster.

How to reproduce

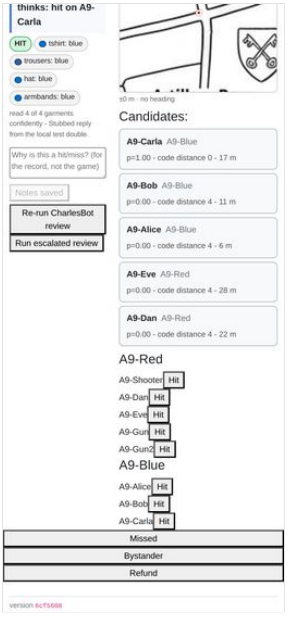
any reviewed shot at `/admin/shots`.

Where

`react-ui/src/ShotQueue.js:518-560` (`RankedCandidates`, renders `<li>` with no control) and `:833-856` (the roster that has the buttons).

Why it matters for the 19th

#3's stated point is that the admin sees who CharlesBot thinks it is. Making them re-find that name in an unranked list at 3 a.m. with a queue backed up is where the wrong player gets shot. A "Hit" button on the candidate row is the obvious fix and the roster stays as the escape hatch.



a9_queue_exactmatch_candidates.png

An adjudication gives no confirmation — the panel silently turns into a different shot

MAJOR *found by agent A9*

The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.

What happens

pressing Hit/Missed/Bystander re-fetches the queue; the index stays put, so the same position now holds the *next* shot. Scroll position is preserved, so the admin is left staring mid-page at a different shot's roster with no toast, no "recorded as HIT on A9-Target2", and no undo. `a9_adj_before_hit.png` and `a9_adj_after_hit.png` differ only in which names are in the ranking. If the button did nothing (or hit the wrong row) it looks identical.

How to reproduce

`scratch/a9/adjudicate2.js`; header goes `Shot 36 of 38` → `Shot 36 of 37`.

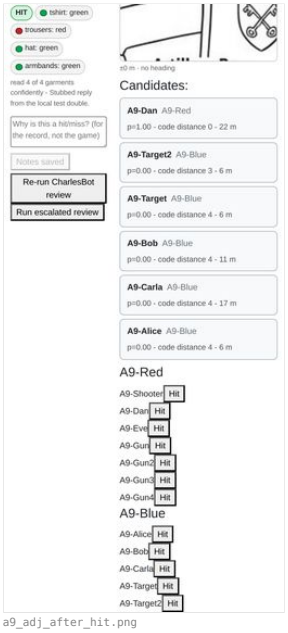
Where

`react-ui/src/ShotQueue.js:686-712` — `hitUser` / `markShotMissed` / `markShotBystander` all `evictShotFromCache(...).then(update)` with no user feedback.

Why it matters for the 19th

resolutions are terminal (`CLAUDE.md`), and R8's appeals exist precisely because they get contested. An admin who cannot see what they just did cannot notice a fat-fingered ruling, and given the 31×24 buttons two rows apart, they will make some.

a9_adj_before_hit.png



a9_adj_after_hit.png

An in-flight request silently logs the admin back out

MAJOR *found by agent A0*

Cuts across everything: the player's browser session

What happens

the session is a signed cookie (`SessionMiddleware`, `backend/main.py:122`). A request that was already in flight when `admin_authenticate` succeeded carries the *pre-login* session and its response writes that older session straight back over the cookie, discarding `admin_authed`. Logging in on the running React app — which polls `user_info`, `my_id` and the ticker continuously — therefore fails intermittently: `admin_authenticate` returns `true`, the `Set-Cookie` carries `{"admin_authed": "true"}`, and the very next `admin_is_authed` returns `false`.

How to reproduce

open the app at `/`, and *immediately* (before boot requests settle) `POST /api/admin_authenticate?password=password`, then `GET /api/admin_is_authed`. Observed `false` with the cookie visibly set to the authed value. Doing the same on a bare same-origin page (`/api/hello`), with no app polling, is reliably `true` — that is the workaround the

harness uses (`harness.js newAdmin`).

Where

`backend/admin_auth.py mark_admin_authed`, plus every endpoint that writes to `request.session` (`backend/user_id.py assign_new_ID` does too — same root cause as the finding above: last writer wins on a cookie-backed session).

Why it matters for the 19th

the admin logs in on a phone, appears to succeed, and then every admin page 403s until they try again. Recoverable, but maddening mid-game with a queue backing up — and the same last-writer-wins mechanism is what makes the identity bug above sharp.

A greyscale (mode "L") shot image 500s the admin's shot view and the zip download

MAJOR *found by agent A0*

Follow-up: a greyscale photo poisons the admin queue and the zip

What happens

`backend/image_processing.py draw_cross_on_image` (:248) and `annotate_image_with_stats` (:60) both hardcode an RGB fill, `line_color = (255, 255, 255)`, and hand it to PIL's `draw.line` / `draw.text`. On a single-channel image PIL raises `TypeError: color must be int or single-element tuple`. `load_image` (:36) never normalises the mode, so whatever the shot was stored as is what gets drawn on. Reproduced directly against the repo's own functions, no server involved: `RGB draw_cross_on_image OK RGB annotate_image_with_stats OK L draw_cross_on_image TypeError: color must be int or single-element tuple L annotate_image_with_stats TypeError: color must be int or single-element tuple CMYK draw_cross_on_image OK`

The admin shot queue is global while every CharlesBot toggle is per-game

MAJOR *found by agent A10*

PRIORITY FINDING: the "one game blocks everyone" worry is FALSE — but the admin's queue view is still global

What happens

`GET /api/admin_get_shots_info` takes no game id (`backend/main.py:527`, `admin_interface.py:968-975`) and returns every unchecked shot in the database, oldest first. The four toggles it is read next to (`ai_shot_review_enabled`, `ai_auto_actions_enabled`, `ai_escalation_enabled`, `ai_resolve_everything_enabled`) are all columns on `Game`. The result is a queue page whose contents obey no toggle the admin can see on that page: shots from a game with review off sit interleaved with annotated shots from a game with it on, and there is no game column to tell them apart.

How to reproduce

open `/admin` → shot queue with more than one game in the DB. 36 shots were listed; 30-odd belonged to games I did not create.

Where

`backend/main.py:527`, `backend/admin_interface.py:968`

Why it matters for the 19th

if only one game exists on the night this is invisible. If a test game or an aborted game is left in the DB, its shots permanently occupy the top of the admin's list and every real shot is below them — the admin scrolls past dead shots all night. Worth deleting stale games before kickoff.

One unrenderable shot image 500s `admin_get_shot` and floods the queue page with uncaught errors

MAJOR *found by agent A10*

Other things found on the way

What happens

`GET /api/admin_get_shot?shot_id=06e89772-…` returns **500**. Backend traceback: `backend/main.py:564 admin_get_shot` → `admin_interface.py:942 markup_shot_model` → `image_processing.py:254 draw_cross_on_image` → PIL `TypeError: color must be int or single-element tuple`. The aim-cross is drawn in an RGB colour onto an image that is not RGB (a greyscale/palette PNG), and nothing catches it. The queue page then makes it worse: `update()` in `ShotQueue.js:663-670` does `Promise.all(shot_ids.map(getShotFromCache))` over **every** id from the global `admin_get_shots_info`, and `fetchAndCache` throws on a non-ok response. Each bad shot becomes an unhandled rejection — `Error fetching shot: Internal Server Error` ×3 in `page.consoleErrors` — and in the CRA dev server the error overlay covers the page and swallows every click, so the shot queue is unusable until the overlay is dismissed. In a production build there is no overlay, so it degrades to console noise plus one shot that never loads, but the underlying 500 is real either way.

How to reproduce

have any shot in the DB whose photo is not an RGB image (mine were RGB; shot `06e89772-…` is another agent's), then open `/admin/shots`.

Where

`backend/image_processing.py:254`, `backend/admin_interface.py:942`, `react-ui/src/ShotQueue.js:663-670`.

Why it matters for the 19th

real phone cameras send RGB JPEGs, so the PIL crash is unlikely to fire on the night — but the *frontend* half is not conditional on that: any single shot whose fetch fails (500, timeout, a dropped connection on a slow venue wifi) takes the whole preload down with an unhandled rejection, on the admin's only adjudication screen, and the global queue means the bad shot need not even belong to the game being run. Worth a `.catch` either way.

Filenames cannot identify a shot — the timestamp in them is the moment of the zip, not of the shot

MAJOR *found by agent A11*

Downloading all shot images as a zip.

What happens

every entry is `{shooter name}_{time.time()}.png`, where `time.time()` is read while *rendering the zip*. In my archive all 40 files carry timestamps inside a 2-second window (`A11-Shooter_1787945802.6375282.png` … `_1787945802.8645744.png`) — the moment I pressed the button. There is no shot id, no target, no game and no fired-at time. Eight shots by one shooter are eight near-identical filenames whose only ordering is DB insertion order, and the mapping back to a shot in the queue is unrecoverable.

How to reproduce

`POST /api/admin_dump_images`, list the archive.

Where

`backend/image_processing.py:31` (`filename = f"{name}_{time.time()}.png"`), called from `backend/postprocess_shot_images.py:_render_shot_images`.

Why it matters for the 19th

this dump is the post-game artefact — the pile of photos you go through afterwards to settle arguments and to feed the replay harness. Filenames that encode neither *when* nor *which shot* mean the only way back to a shot record is eyeballing the burned-in caption. `shot_model.id` and `shot_model.time_created` are both in hand at that call site.

One player with "/" in their name makes the whole zip download 500 for everyone

MAJOR *found by agent A11*

Downloading all shot images as a zip.

What happens

`set_name` stores the name unsanitised (`backend/user_interface.py:355` — `.strip()` only), and the filename is interpolated straight into a path. A name containing `/` produces a path with a non-existent parent and PIL raises, taking the whole endpoint down — not just that one image. Confirmed live: renamed my own shooter to `A11-Shooter/x`, `POST /api/admin_dump_images` → **500** in 1.2 s; renamed back → 200. Backend log: `FileNotFoundError: [Errno 2] No such file or directory: '/tmp/shot_images_g_8ppzic/A11-Shooter/x_1787945868.2034075.png'` (`scratch/backend.log:382731`). The name was restored within seconds. A leading `../` is the quieter variant: the parent *does* exist, so the render succeeds, the file lands outside the temp cache dir, `cache_dir.glob("*.png")` never sees it, and it is silently missing from the archive while a stray PNG is left on the server.

How to reproduce

`admin_set_user_name?...&name=Bob/x` (or a player typing it themselves at onboarding), then press the download button.

Where

`backend/image_processing.py:31-33`; `backend/postprocess_shot_images.py:20-33`; no validation in `backend/user_interface.py:355`.

Why it matters for the 19th

players type their own names. One person called `Rob/Bob` or `and/or` and the post-game image dump is dead, with an opaque 500 in the admin error box and no hint which player caused it. Also a real (if low-value) path traversal driven by unvalidated player input.

`collected_as_team` is offered for every item type but only implemented for ammo

MAJOR *found by agent A3*

Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).

What happens

the admin's New-items form shows the `collected_as_team` checkbox for all four types. Ticking it for armour, medpack or weapon produces a perfectly valid signed QR that can never be collected: `_ACTIONS` has no entry for those keys, so `do_item_actions` raises `NotImplementedError` — which, being a `RuntimeError` subclass, is caught by `collect_item` and served to the player as `403 Item collection for (<ItemType.ARMOUR: 'armour'>, True) has not been implemented`. A raw Python enum repr in a player-facing string.

How to reproduce

`scratch/a3/run3.json`, steps `armour_collected_as_team` and `medpack_collected_as_team`.

Where

`backend/item_actions.py:134-148`; `backend/user_interface.py:740`; the checkbox at `react-ui/src/NewItems.js:132`.

Why it matters for the 19th

drops are minted from this form and printed. A team medpack or team armour is an obvious thing to want; nothing warns you it is not a thing until a player is standing in front of the poster.

The crosshair is not in the stored photo, and is nearly invisible in the copy the model sees

MAJOR *found by agent A4*

Taking a shot photo of another player, including the on-screen crosshair overlay.

What happens

two separate things, which together matter for #4. 1. The stored `Shot.image_base64` has **no** crosshair in it. `MyWebcam.capture()` draws only the video frame to the canvas, so what is saved is the bare photo; every place a cross appears is an overlay applied later (a DOM `<img>` in the player's history, `draw_cross_on_image` for the admin, `draw_aim_marker` on the way to the vision model). I sampled the stored JPEG of my first shot: 2048x1080, centre pixel `(26,191,0)`, centre row and column indistinguishable from the background. That is a defensible design — but it means "is the cross burned into the photo" is *no*, and anything reading the raw stored image gets an unmarked frame. 2. In the copy that *is* marked and sent to the model, the marker barely survives. `ai_shot_review._review_image_data` does `prepare_for_vision(draw_aim_marker(image))` — i.e. it draws a **1-pixel** red line on the full-size photo and *then* downsamples with LANCZOS. On my 2048px shot (a 2x reduction) the line came out of `admin_get_shot_vision_images` averaging **R=53 on a G=129 background**, against R=8/G=148 for the unmarked rows: a thin dark-olive tint, not a red line. See `scratch/a4/view/vision_full_crop.png` — that is a 4x NEAREST blow-up and it is still faint. A real phone photo is 4032px, i.e. a 4x reduction, so the marker will be roughly twice as washed out again.

How to reproduce

fire a shot; `GET /api/user_shot_image` and inspect the centre row/column (no marker). Turn on recognition, fire another, then `GET /api/admin_get_shot_vision_images?shot_id=…` and sample the centre row of `full` against a neighbouring row.

Where

`react-ui/src/MyWebcam.js:56-71` (capture draws only the video); `backend/image_processing.py:116-133` (`_draw_aim_marker_on`, `width=1`); `backend/ai_shot_review.py:126` and `backend/shot_escalation.py:569` (`prepare_for_vision(draw_aim_marker(...))` — mark first, shrink after).

Why it matters for the 19th

roadmap #4 is about making the model read the crosshair *centre*. The docstring at `image_processing.py:120` says the full-frame line was chosen so "the marker survives downsizing and JPEG compression". Measured, it does not survive well: the model is being asked to reason about a line whose contrast has been reduced ~85% before it ever sees it. Drawing the marker *after* `prepare_for_vision`, or scaling the line width with the image, would cost nothing and is worth doing before any more prompt work is spent on #4.

The shot cooldown is enforced only in the browser, and a reload clears it

MAJOR *found by agent A4*

Taking a shot photo of another player, including the on-screen crosshair overlay.

What happens

`submit_shot` has no cooldown check of any kind. The only thing that enforces `shot_timeout` is `FireButton`'s local `onCooldown` state and its `setTimeout`. Two consequences I reproduced: * Firing, then reloading the page 0.8 s later, re-enabled the fire button **1.6 s** after the first shot (weapon timeout 6 s), and the second shot was accepted — the two stored shots are 4 s apart. * Three `POST /api/submit_shot` calls issued back to back from the page returned `200, 200, 200` and created three shots, inside one 6 s window.

How to reproduce

`scratch/a4/t20_reload_cd.js` (reload case) and part C of `scratch/a4/t19_cooldown.js` (API case).

Where

`react-ui/src/FireButton.js:44-63` (the only timer); `backend/user_interface.py:473-522` (`submit_shot` checks team, HP and ammo, and nothing else).

Why it matters for the 19th

weapons are differentiated almost entirely by `shot_timeout` (`(1,1)` Eat-a-bullet vs `(3,6)` OMG), so an unenforced cooldown makes the weapon table advisory. A player does not need to be malicious — a PWA that reloads after a crash, or an app-switch that remounts the view, hands them a free extra shot. Ammo is still spent per shot,

which limits the damage.

An upheld appeal by the *target* deletes the shot from their history

MAJOR *found by agent A4*

The shot status bubble after taking a shot — every visual state it can be in, and that it stays on screen rather than disappearing.

What happens

a player who is ruled hit, appeals, and **wins** loses the whole entry. `admin_interface._mark_shot_checked` sets `target_user_id = None` for any non-hit re-ruling, and `user_interface.get_shots_received` selects on `target_user_id` — so the moment the appeal succeeds the shot leaves the appellant's received list. Observed end to end: A4-Alice's bubble went 💥 "Hit you! - shot by A4-Bob" (`a4_50`) → ⚖️ amber "Under appeal" (`a4_54`) → and after the admin re-ruled it a miss, straight back to the **previous, unrelated** shot, the amber CharlesBot one (`a4_55`). `GET /api/user_shots_received` returned `[]`. The `appealUpheld` state therefore exists but can never be shown to the person who won on the received side; the shooter, who did *not* appeal, sees it fine (`user_shots` for A4-Bob: `result: "miss", appeal_state: "upheld"`). The appeal refund itself is correct (3→2 spending, 2 after the refund).

How to reproduce

`scratch/a4/t14_received.js` then `scratch/a4/t15_vanish.js`.

Where

`backend/admin_interface.py:1184-1188` (clears `target_user_id`); `backend/user_interface.py:589-593` (`get_shots_received` filter); `react-ui/src/ShotHistory.js:421` (bubble = `shotList[0]`).

Why it matters for the 19th

this is exactly the "stays on screen rather than disappearing" claim the checklist line makes, and it is the one case where it does not. The player who was hit, spent one of three appeals, and was proved right, watches their own evidence vanish; the only confirmation is one ticker line that scrolls away. It is also the case where they most want the photo back (to show someone) and where they can no longer reach it — `user_shot_image` 404s for them too, because `appeal_party` no longer matches.

`admin_clear_circle` announces the circle as if it had just been set

MAJOR *found by agent A5*

The map view, including drop/circle locations on the live Westminster venue map (#12).

What happens

clearing a circle removes it from the map (correct) *and* posts the "appeared" ticker line to every player. Observed verbatim: clearing DROP put "A supply drop has appeared! It's marked in blue" on the ticker at the same instant the drop circle vanished from the map; clearing EXCLUSION posted "The circle has changed - if you're outside, get moving!". Clearing NEXT posts "The next circle has been announced! Check the map...".

How to reproduce

`POST /api/admin_clear_circle?game_id=<g>&name=DROP` with a player's map open; the drop disappears and the ticker announces a drop.

Where

`backend/main.py:990-1000` (`admin_clear_circle` calls `set_circles` with lat/long/radius `None`) → `backend/admin_interface.py:306-350`, which always `tk.send_ticker_message(message_type, ...)` regardless of whether it set or cleared.

Why it matters for the 19th

a whole team runs to a supply drop that the admin just removed, or is told the circle changed when it was switched off. The admin cannot clear a circle quietly. (Note also that `main.py` defines two functions both literally named `admin_set_circle` — line 951 and line 991 — the second shadowing the first. Harmless, since the decorator registers by path, but it makes the clear endpoint hard to grep for.)

Every adjudication control in the queue is 24 px tall (radios 13 px)

MAJOR *found by agent A9*

The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.

What happens

measured heights — `Next` 390×24, `Previous` 390×24, per-player `Hit` 31×24, `Missed` 390×24, `Bystander` 390×24, `Refund` 390×24, `Run escalated review` 131×24, queue-mode radios and the "Show adjudicated shots" checkbox 13×13. The only control over 24 px is "Re-run CharlesBot review" at 42 px, and only because its label wraps to two lines. This **confirms** the 24 px / 13 px figures another agent reported. The `Hit` buttons also sit flush against the player's name with no gap ("A9-CarlaHit"), and `A9-Target` / `A9-Target2` are adjacent rows.

How to reproduce

`scratch/a9/sizes.json`, produced by `scratch/a9/view_exact.js`.

Where

`react-ui/src/ShotQueue.js:757-800` (nav + radios), `:838-856` (roster Hit buttons), `:861-884` (Missed/Bystander/Refund) — all bare `<button>` with no class.

Why it matters for the 19th

`CLAUDE.md`'s own house style says 3.5em / 3em / 3.2em because this is driven one-handed on a phone. 24 px is roughly half the 44 px touch minimum, and the two 31×24 buttons most likely to be mis-hit are the ones that damage a player. `ReferencePhotos.module.css` already has the pattern to copy.

CharlesBot's overall confidence is never displayed in the queue

MAJOR *found by agent A9*

The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.

What happens

the stored review carries `confidence` (0.92, 0.8, 0.75 in my runs) and nothing on the page shows it. The outcome tag is a bare "HIT"; the per-channel tags show their confidence only when `warnBelow` is set, which the queue deliberately does not do; the headline sentence prints a name with no number when the identification is clean. The *only* numbers an admin gets are the candidates' posteriors — which are the identification term, not "how sure was the model that this is a person at all". An escalated verdict does show a percentage (`escalationVerdictTag`), so the queue is inconsistent with itself.

How to reproduce

`scratch/a9/text_exactmatch.txt` (review confidence 0.92, nowhere on screen) against `scratch/a9/review_exactmatch.json`.

Where

`react-ui/src/ShotQueue.js:265-275` (`outcomeTag`, no confidence) vs `:231-260` (`escalationVerdictTag`, which does show it); `ChannelTags` at `:278-305` with `warnBelow = null`.

Why it matters for the 19th

"CharlesBot's verdict *and confidence*" is half of this roadmap line, and the half that tells a rushed admin whether to look at the photo at all. A 0.55-confidence hit and a 0.95-confidence hit are the same green "HIT" pill today.

`admin_get_shot` 500s on a non-RGB image, which breaks the queue's pre-load

MAJOR *found by agent A9*

The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.

What happens

`draw_cross_on_image` draws with an RGB tuple, so a greyscale (mode `"L"`) upload raises `TypeError: color must be int or single-element tuple` and the endpoint 500s. The queue pre-loads *every* shot in the background, so one bad shot produces an unhandled promise rejection per attempt: I watched the in-app banner climb to `admin_get_shot → 500 (x194)`, and in the dev server the CRA error overlay iframe covered the page and swallowed every click until I removed it by hand. In production there is no overlay, so the visible damage is the red banner, one shot that can never be adjudicated, and `Promise.all` aborting the rest of the pre-load.

How to reproduce

submit a greyscale JPEG via `submit_shot`, then open `/admin/shots`. Traceback at `scratch/backend.log` line ~1615811.

Where

`backend/image_processing.py:254` (`draw_cross_on_image`), reached from `backend/admin_interface.py:942` and `backend/main.py:564`; the swallowed rejection is `react-ui/src/ShotQueue.js:660-668` (`Promise.all(shot_ids.map(getShotFromCache))`).

Why it matters for the 19th

a phone that hands over a greyscale or CMYK JPEG bricks one shot and spams the admin's error banner. A real camera capture is normally RGB, so I believe this is a robustness gap rather than something certain to bite — but the *unhandled rejection* on any failing pre-load is worth hardening whatever the cause.

Minor findings (44)

The no-outfits empty state hides below the whole wardrobe form

MINOR *found by agent A1*

Join a team via QR/link and pick an outfit at /pick (#10): ranked outfit list, canonical-first ordering, colour swatches, pagination.

What happens

when `total === 0`, the wardrobe fieldsets stay on screen and the "No outfits found. Are you sure you don't have any more clothes?" / "Yes, I'm sure" pair is appended *below* the "Show me outfits" button, ~1400px down the page. A player who taps "Show me outfits" sees nothing change above the fold.

How to reproduce

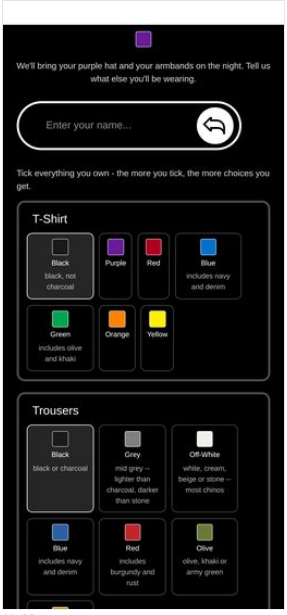
in a team where teammates are already placed, tick only T-shirt Black + Trousers Black, tap "Show me outfits". `A1_25_empty_state.png` is the viewport straight after that tap.

Where

`react-ui/src/PickOutfit.js:545-552` (`showAreYouSure` block is rendered after the wardrobe form).

Why it matters for the 19th

the relaxed-threshold path is exactly the one a badly-dressed guest hits, and it looks like the button did nothing. The relaxed pass itself works — `A1_26` showed the "These are the best options we could find" note and one option, as designed.



A1_25_empty_state.png

The error popup is unreadable on this page — it sits translucent over the page text

MINOR *found by agent A1*

The name-then-confirm flow — entering a name, seeing the committed outfit, and the outfit becoming locked in.

What happens

`Popup`'s panel is `rgba(0,0,0,0.7)` over a `rgba(0,0,0,0.3)` scrim. On `/pick`, whose page is already pure black with white text, 30% of the header prose shows straight through the message, so "That outfit isn't available any more…" is overprinted on "We'll bring your purple hat and your armbands…". The close button also straddles the right screen edge.

How to reproduce

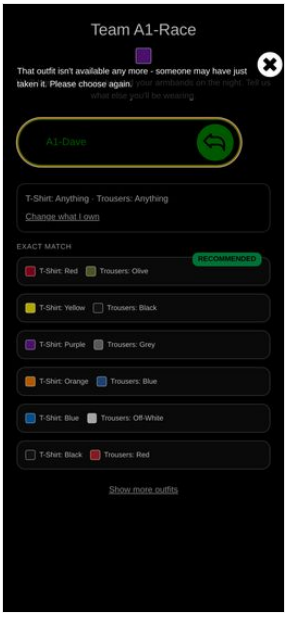
two teammates select the same outfit; the second one confirms. `A1_32_race_conflict.png`.

Where

`react-ui/src/Popup.module.css:10-17` (`.outerContainer` background 0.7 alpha, no opaque panel).

Why it matters for the 19th

this is the *only* error channel on the page, and the race it reports (a whole team picking at once at the door) is the expected case rather than an edge case. The recovery behaviour behind it is excellent — the list refetches with the taken outfit gone — the message just needs an opaque background.



A1_32_race_conflict.png

On a 390px phone the queue's error banner overflows off-screen, and the Hit buttons sit on top of the player names

MINOR *found by agent A10*

Running an escalated review by hand on a shot (#11 — "Run escalated review")

What happens

two things in `A10-queue-shotF-escalate-alert.png`: (1) the red banner reads *"CharlesBot review failed: vision call failed after 3 attempts: OpenRouter returned 50" *and is then cut off by the right edge — it does not wrap, so the admin cannot read* why* it failed, which is the only actionable part; (2) in the team roster below, each "Hit" button overlaps the name to its left — "A10-Shooter1" and "A10-Shooter2" are visibly clipped by their own buttons. With similar player names that is a genuine misidentification risk for the one control that takes a life.

How to reproduce

`/admin/shots` at 390x844 on any shot with an errored review and a team of more than a couple of players.

Where

`react-ui/src/ShotQueue.js` + `ShotQueue.module.css` — the page is a Bootstrap `Row`/`Col` two-column layout that does not collapse at phone width.

Why it matters for the 19th

the shot queue is the admin's main screen and the brief says the game is driven one-handed on a phone. Wrong-name taps are the expensive mistake here.

A10-queue-shotF-escalate-alert.png

A shot fired outside the venue crop shows a blank white box with no explanation

MINOR *found by agent A11*

The per-shot map (R5) showing GPS accuracy and the shooter's compass heading.

What happens

a fix outside the map image (I used Kingston, ~25 km away) renders the dot, the cone and the accuracy circle on plain white — the map background is simply scrolled off. Nothing says "off the map"; the venue drawing is itself mostly white with black lines, so it reads as an empty patch of Westminster rather than as no coverage.

How to reproduce

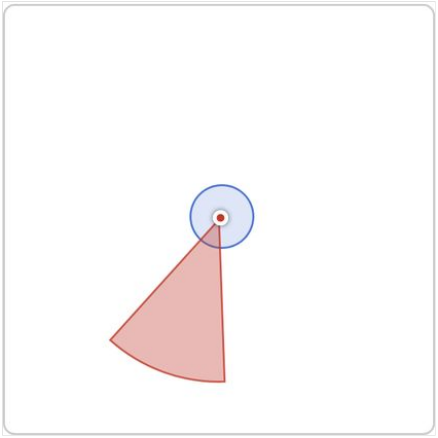
`set_location?latitude=51.4130&longitude=-0.3075&accuracy=15`, fire, open in the queue. `A11-map-faraway.png`.

Where

`react-ui/src/ShotMap.js:62` — no bounds check against `geometry.bottomLeft` / the map extent.

Why it matters for the 19th

a stale or wildly wrong fix (indoors, cold start, a player still on the train) will produce exactly this, and the admin will read "he was standing in a blank bit" instead of "this fix is junk". The caption's ± figure is the only clue and it can be small on a bad-but-confident fix.



A11-map-faraway.png

An unsaved note is discarded silently when you page to another shot

MINOR *found by agent A11*

Admin shot history and notes on a player.

What happens

type into the notes box, tap Next, tap Previous — the typed text is gone, replaced by the last saved value. No warning, no prompt, and the button that would have saved it scrolls with the rest of the page.

How to reproduce

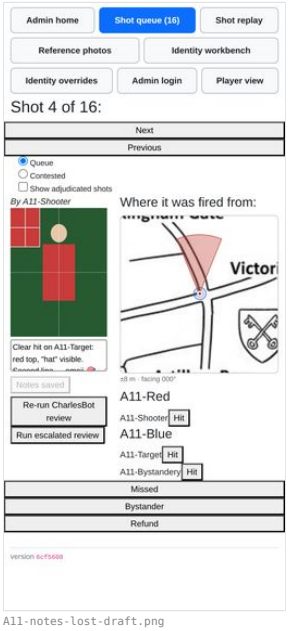
type "UNSAVED DRAFT" without saving → Next → Previous. Observed textarea reverts to the previously saved note. `A11-notes-lost-draft.png`.

Where

`react-ui/src/ShotQueue.js:363` — the `useEffect` on `shot_id` resets `notes` and `dirty` unconditionally.

Why it matters for the 19th

the notes exist to preserve the admin's reasoning for the replay harness. Adjudicating and then paging on is the normal motion, so the reasoning most worth keeping — the one written while thinking about a hard shot — is the one most likely to be thrown away.



A11-notes-lost-draft.png

The notes textarea is ~155 px wide in the queue's left column at 390 px

MINOR *found by agent A11*

Admin shot history and notes on a player.

What happens

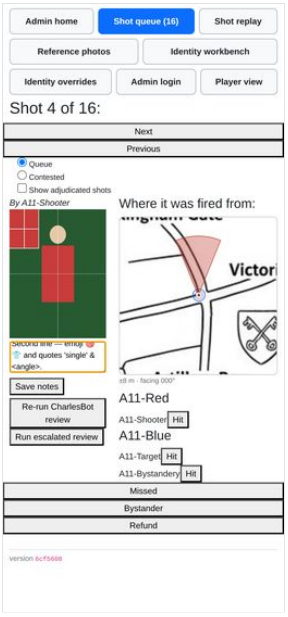
on a phone the two `<Col>`s stay side by side, so the notes box is about 155x100 CSS px — roughly three short lines. Typing a sentence scrolls the first line out of view (`A11-notes-typed.png`: only the second half of my note is visible). A 4000-character note is a wall of two-word lines (`A11-notes-long.png`, `A11-history-view.png`). Below ~576 px the columns stack and the box gets full width (`A11-queue-w320.png`), so the *narrowest* phones get the better notes box.

Where

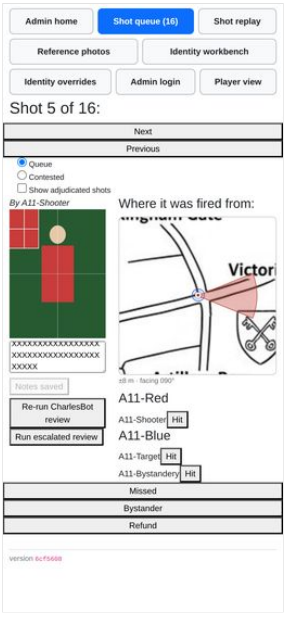
`react-ui/src/ShotQueue.js:800-830` (bare `<Col>`, no breakpoint) and `ShotQueue.module.css:217` (`.notesBox textarea`).

Why it matters for the 19th

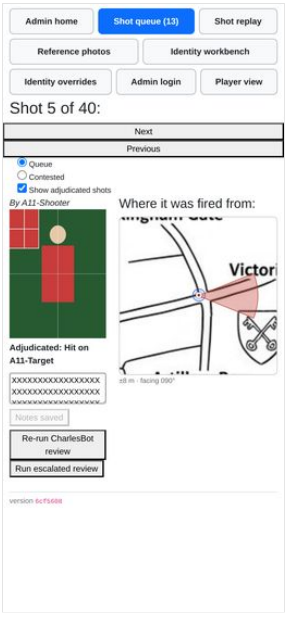
the admin is meant to type reasoning here one-handed, and cannot read back what they typed. *Incidental, probably another agent's line:* the queue page scrolls horizontally by 11 px at 390, 360 and 320 px. The offender is the `.showCheckedToggle` labels (`react-ui/src/ShotQueue.module.css:202`), each `width: 100%` inside a padded Bootstrap `Row`.



A11-notes-typed.png



A11-notes-long.png



A11-history-view.png

A11-queue-w320.png

The download button gives no feedback at all while the zip builds

MINOR *found by agent A11*

Downloading all shot images as a zip.

What happens

the button does not disable, change label, or show a spinner; the page is visually identical during the 6 s render (`A11-zip-during.png`, taken immediately after the click, is indistinguishable from `A11-zip-button.png`). Nothing indicates work is happening or how much is left.

Where

`react-ui/src/AdminMode.js:530-539` — a bare `<button onClick=...>`; `AdminCommon.js:23 adminDownload` has no pending state.

Why it matters for the 19th

the whole zip is rendered and buffered in server memory (`io.BytesIO` in `zip_shot_images`) and then again as a browser blob, with no streaming and no cancel. 43 tiny synthetic shots took 6 s here on localhost; a real night's several-hundred phone-camera photos will take long enough over mobile data that an admin with no feedback will tap again — and each tap starts an independent full re-render of every shot in the database. (The re-tap amplification is inference from the code, not something I hammered the shared server to prove.)

A11-zip-during.png

A11-zip-button.png

A freshly reset player sees a "NO AMMO!" fire button while holding ammo

MINOR *found by agent A13*

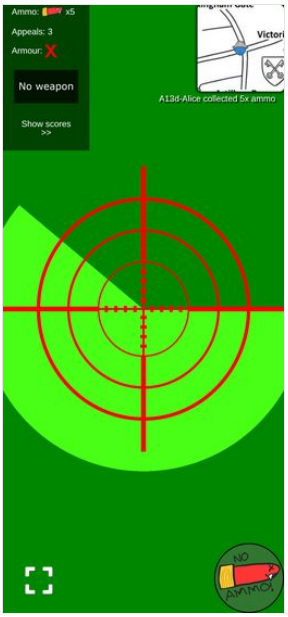
Cuts across both: confirm the game can be reset (resetdb) and replayed from a clean state without any of the above breaking, since that is exactly what happens between the dry run and the night itself.

What happens

`userCanShoot = isInTeam && hasBullets && hasWeapon` (`react-ui/src/FireButton.js:17`), and after any reset a player's default is `shot_damage = 0` → "No weapon". So the HUD reads "Ammo x5" in the top-left and shows the crossed-out **NO AMMO!** button in the bottom-right at the same time (`a13_20_after_admin_reset_game.png`). It is not reset-specific, but the clean state is where every player meets it, because nobody has a weapon yet.

Why it matters for the 19th

at kick-off, before weapons are handed out, every player is looking at a button that tells them the wrong reason they cannot shoot, and the first question the admin gets is "I have ammo, why can't I fire?"



a13_20_after_admin_reset_game.png

The whole page scrolls sideways on a phone because of the game selector

MINOR *found by agent A2*

The identity workbench / AdminIdentity.js: viewing a player's effective appearance and any overrides, and recording an override for a misdressed player so they stay distinguishable from their teammates.

What happens

`documentElement.scrollWidth = 778` against `clientWidth = 390`. The offender is the `Game: <select>` — its intrinsic width is set by the longest `<option>`, which is `"a47c0fcf (Red Team, Blue Team, Green Team, Yellow Team, Purple Team, Orange Team, Pink Team, Cyan Team, Brown Team, Black Team)"`. Everything else on the page is ≤ 393px. The consequence in a screenshot is that the fourth swatch of every roster row sits past the right edge until you swipe (`A2-identity-09-outside-palette.png` vs `A2-identity-10-table-scrolled.png`).

How to reproduce

open `/admin/identity-overrides` at 390x844 with more than one game in the DB; swipe left.

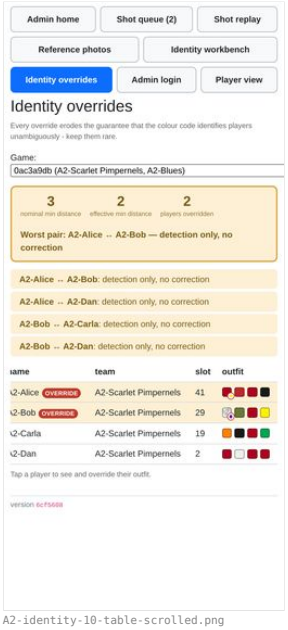
Where

`react-ui/src/AdminIdentity.js:657` (`GameSelector`) — the option label is `id.slice(0,8) + " (" + every team name + ")"`, with no `max-width` / `text-overflow` on the select.

Why it matters for the 19th

cosmetic on the night if only one game exists (the selector hides itself when `games.length <= 1`), but the dev/test DB always has several, and the swatch column — the single thing this page exists to show — is the column that falls off the edge.

A2-identity-09-outside-palette.png



A2-identity-10-table-scrolled.png

The four swatches are unlabelled, and the only labelling is a hover tooltip

MINOR *found by agent A2*

The identity workbench / AdminIdentity.js: viewing a player's effective appearance and any overrides, and recording an override for a misdressed player so they stay distinguishable from their teammates.

What happens

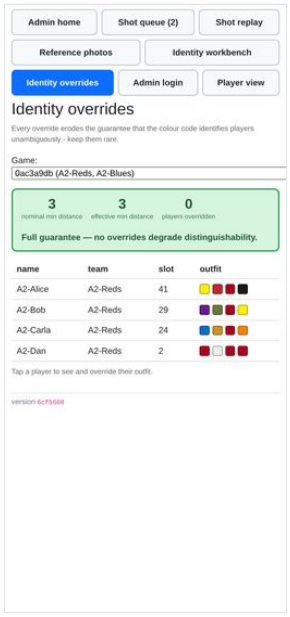
the roster's `outfit` column is four coloured squares in channel order with no header, no legend and no per-swatch caption. Which square is the hat and which the armbands is carried only by `title=` on the span (`AdminIdentity.js:92`, the `title=` attribute), i.e. a hover tooltip — which does not exist on a phone. The editor below does name the channels, but only for the one selected player.

How to reproduce

look at `A2-identity-02-mygame.png` and try to say which square is the trousers without opening a player.

Why it matters for the 19th

the roster view is the "scan the room" view. If an admin has to tap into each player to learn which square is which, the roster is decoration.



A2-identity-02-mygame.png

A player's own HUD never shows their team name, so the rename is only visible in the scoreboard

MINOR *found by agent A2*

Renaming a team from the admin dashboard.

What happens

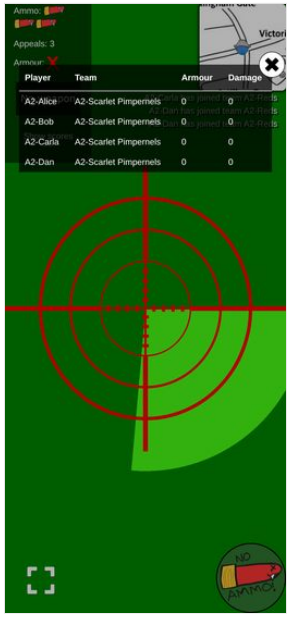
`team_name` is rendered in exactly one player-facing place after onboarding: the scoreboard popup (`Scoreboard.js:47`). `UserMode.js` shows ammo / appeals / armour and nothing about the team; `OnboardingView.js:181` shows `You are in team "X"` but only while the player is still on the onboarding rungs. So a mid-game rename reaches players only if they open the scores.

How to reproduce

`A2-rename-06-carla-live-update.png` — the HUD behind the scoreboard has no team anywhere.

Why it matters for the 19th

low stakes if teams are renamed before kickoff. It only bites if a rename is used mid-game to communicate something.



A2-rename-06-carla-live-update.png

A weapon whose damage/timeout pair is not in the lookup shows as nothing at all

MINOR *found by agent A3*

Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).

What happens

I minted `{"shot_damage": 7, "shot_timeout": 2}`. It is collected happily (the player really does end up with damage 7 / timeout 2), the ticker announces "A3-Alice2 collected a **<unnamed>**" — the literal placeholder, angle brackets and all — and the HUD weapon slot goes **blank**, because `getGunImgFromUser` returns `null` for any pair it doesn't recognise and the slot renders an `<img>` with no `src`. Visually indistinguishable from having no weapon at all.

How to reproduce

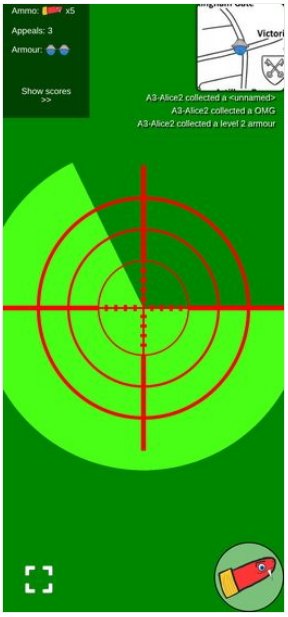
`a3_14_bogus_after.png` (compare `a3_13_omg_after.png`, same player one item earlier).

Where

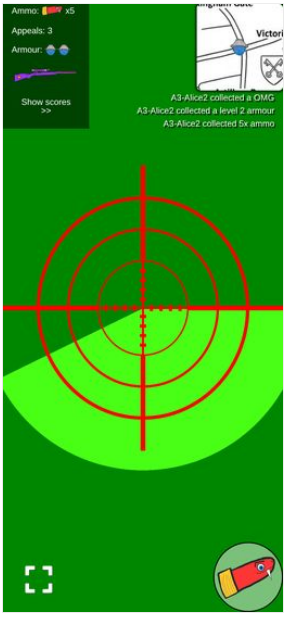
`backend/item_actions.py:124`; `react-ui/src/utils.js:89-99`.

Why it matters for the 19th

the mint form takes two free-number fields with no validation against `WEAPON_NAME_LOOKUP`. One typo when generating the drops and a player carries an invisible gun all night.



a3_14_bogus_after.png



a3_13_omg_after.png

The "new item" pickup overlay does not actually appear

MINOR *found by agent A3*

Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).

What happens

`TemporaryOverlay` is meant to throw the item's icon full-screen for about a second. Sampled every animation frame from the first frame of the document (`scratch/a3/probe3.js`), its computed `display` stayed `none` and its box stayed `0×0` for the whole event while `opacity` animated 0 → 1 → 0 over ~3 s. So the state machine runs, the beep and the vibrate fire, but nothing is painted. A separate slower probe caught one frame where it *was* `display:block` at opacity 0.66 — rendered tiny in the top-left corner over the HUD, not centred (`a3_probe_overlay_1.png`). A 22-frame screenshot burst across a pickup (`a3_burst_00..21.png`) shows no overlay at any point.

How to reproduce

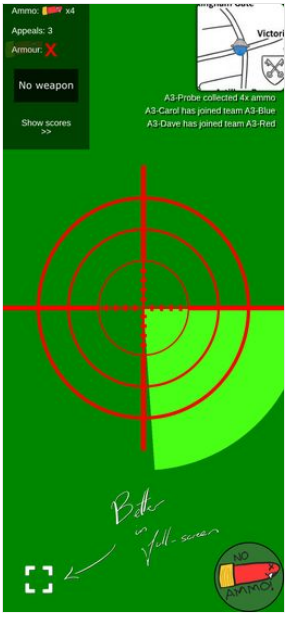
`node scratch/a3/probe3.js`; also `probe4.js` for the burst.

Where

`react-ui/src/TemporaryOverlay.js:41-72` — `display` lives inside the framer variants (`visible.display: "block"` / `hidden.transitionEnd.display: "none"`) alongside `width`/`height` keyframe arrays.

Why it matters for the 19th

it is the only *visual* confirmation of a pickup on the player's own screen; the HUD counter and the ticker still update, so this is polish rather than a blocker. Caveat: measured in headless Chromium. The opacity animation ran normally, so framer-motion itself is working and I do not think this is a headless artefact — but it is a ten-second check on a real phone.



a3_probe_overlay_1.png

The fire button says "NO AMMO!" to a player who has ammo but no weapon

MINOR *found by agent A3*

Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).

What happens

`userCanShoot = isInTeam && hasBullets && hasWeapon`, and the single "can't shoot" artwork is `firebutton_no_ammo.svg`. Players start with `DEFAULT_SHOT_DAMAGE = 0`, so the very first thing that happens after a successful ammo pickup is a HUD reading "Ammo x8" next to a button reading "NO AMMO!".

How to reproduce

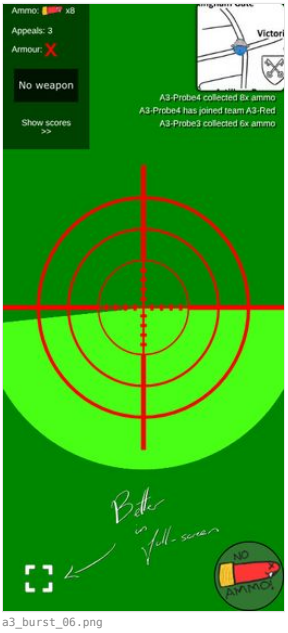
`a3_burst_06.png` (ammo x8, "No weapon", button says NO AMMO!) vs `a3_13_omg_after.png` (ammo + gun, plain button).

Where

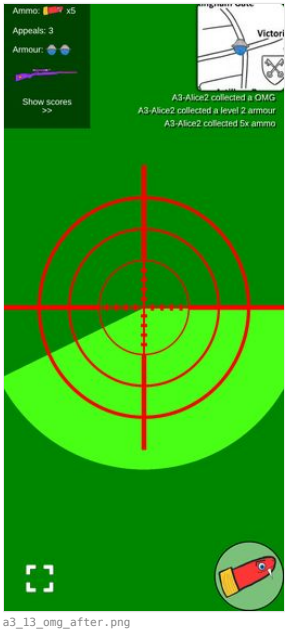
`react-ui/src/FireButton.js:14-17`.

Why it matters for the 19th

it sends people hunting for ammo they already have instead of for a gun, at the exact moment they are learning the game.



a3_burst_06.png



a3_13_omg_after.png

A shot the server refuses is swallowed silently

MINOR *found by agent A4*

Taking a shot photo of another player, including the on-screen crosshair overlay.

What happens

`captureAndUpload` posts, `.then(r => r.json())`, logs, and calls `refreshShots()`. There is no `response.ok` check and no error path. I stubbed `/api/submit_shot` to answer `403 {"detail":"User is dead"}` and pressed fire: the bang played, the button went into its cooldown art for the full 6 s, and the screen said nothing at all — no shot recorded, no ammo spent, no message (`a4_80_refused_shot.png`; the only trace is a console 403).

How to reproduce

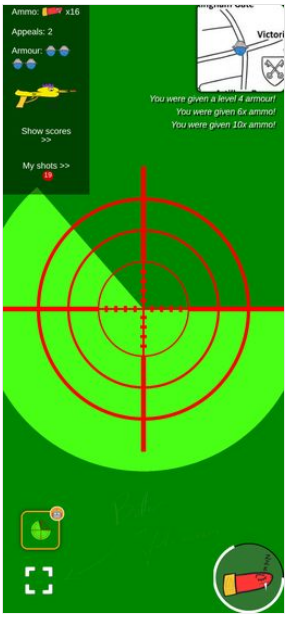
`scratch/a4/t22_refused.js`.

Where

`react-ui/src/MyWebcam.js:96-105`.

Why it matters for the 19th

the real cases are a stale UI (the player was knocked out and the `user` SSE has not landed), a lost ammo race, or a flaky phone network on the night. In all of them the player believes they fired and their status bubble keeps showing the *previous* shot, which reads as the app having eaten the shot. One line of toast would settle it.



a4_80_refused_shot.png

The wake lock is held on the onboarding screen, before the game starts

MINOR *found by agent A5*

The screen staying awake during play (R3 — Screen Wake Lock), and that the phone's own lock button still turns the screen off on request.

What happens

`useWakeLock()` is called at the top of `GetView`, above the early return that shows `OnboardingView`. A player who has opened the app but has not set a name — or whose game is not active yet, or who has not granted camera/location — is already holding a screen lock. Verified: the "enter your name" screen (`a5_wake_onboarding.png`) logged `request(granted)` with no name set and no team.

How to reproduce

open `/` with a fresh cookie jar and a context granted `screen-wake-lock`; the log records a granted request while the name prompt is on screen.

Where

`react-ui/src/UserMode.js:30` (hook) vs the `OnboardingView` return at `react-ui/src/UserMode.js:68`.

Why it matters for the 19th

players will have the app open in the pub well before the game goes active. Their screens will not sleep for that whole time, on the single biggest battery draw in the game, and R3 shipped deliberately without a toggle to turn it off.

a5_wake_onboarding.png

The lock is still held after the player dies, and re-acquired on every return to foreground

MINOR *found by agent A5*

The screen staying awake during play (R3 — Screen Wake Lock), and that the phone's own lock button still turns the screen off on request.

What happens

killing the player (`admin_set_hp?num=0`) leaves the lock held — the hook is unconditional, and a subsequent `visibilitychange` while dead re-requests it (verified in the log). A dead player's phone stays awake until they close the tab. `a5_wake_dead_view.png` shows the YOU DIED view, map and ticker still live.

Where

`react-ui/src/useWakeLock.js:7-35`, mounted at `react-ui/src/UserMode.js:30`.

Why it matters for the 19th

the first players out are the ones with the longest wait and the most to lose from a flat battery; they are also the ones who no longer need the screen. Arguably intended ("mounted unconditionally" in the roadmap), so flagging as a decision to confirm rather than a defect.

a5_wake_dead_view.png

The drop circle is invisible for ~70% of its animation and never shows its real radius

MINOR *found by agent A5*

The map view, including drop/circle locations on the live Westminster venue map (#12).

What happens

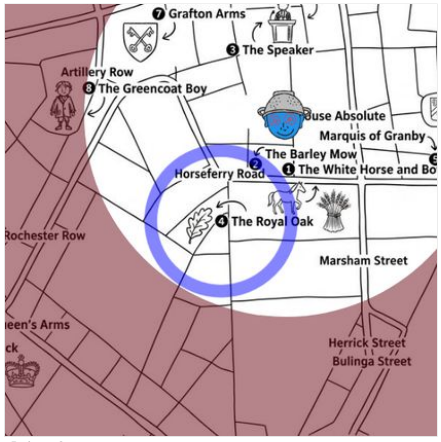
`.dropCircle` runs a 2 s `zoom` keyframe: scale 0.1 → 5.0 while the border fades to fully transparent by 30%, then stays transparent and 5x-scaled for the remaining 1.4 s. Burst screenshots 180 ms apart at the drop location (`a5_drop_0.png` shows a big blue ring, `a5_drop_3.png` shows nothing at all at the same place) confirm the ring is on screen for well under a second in every two, and that the ring you *do* see is up to five times the true radius (a 50 m drop drew a ring covering several streets).

Where

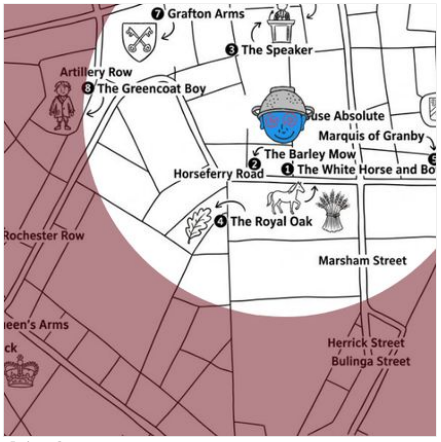
`react-ui/src/MapView.module.css:97-120`.

Why it matters for the 19th

a player glancing at the corner map for a second may see nothing where a drop is, and if they do see it they cannot tell how big the drop area is. It reads as a radar ping rather than a zone; if that is the intent it is fine, but the exclusion and next circles both show their true size and this one does not.



a5_drop_0.png



a5_drop_3.png

An upheld target appeal erases the shot from the winner's own history

MINOR *found by agent A6*

Appealing a resolved shot as either shooter or target (R8), including seeing the appeal budget run out.

What happens

`_record_adjudication` clears `Shot.target_user_id` whenever the new ruling is not a hit, and `get_shots_received` filters on that column. So the moment a target's appeal succeeds, the shot leaves their "My shots" list and `user_shot_image` 404s for them — verified: after A1, A5 and A8 were overturned, Bob's `user_shots_received` held only A3 (the shot an appeal moved *onto* him), and `GET user_shot_image?shot_id=<A8>` returned **404**. The private ticker line "Your appeal was upheld" still carries that shot id and is tappable (`TickerView.js:63` → `openShotHistory(shotId)`), but the shot is no longer in the store, so `selectedShot` is null and the popup opens on the **whole list** instead of the shot. Screenshot `a6_34_bob_tap_upheld_ticker.png`.

How to reproduce

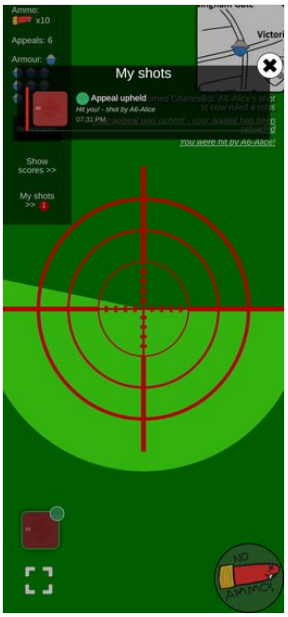
admin rules a shot a hit on Bob → Bob appeals "It missed me" → admin rules Missed → on Bob's phone tap "Your appeal was upheld" in the ticker.

Where

`backend/admin_interface.py:1186` (clears `target_user_id`), `backend/user_interface.py:578 get_shots_received`, `react-ui/src/ShotHistory.js:485` (`selectedShot` lookup).

Why it matters for the 19th

the one moment a player most wants to look at the photo again — "I won, show me" — is the one moment the app cannot show it to them, and the tap that R8 built specifically for this lands nowhere.



a6_34_bob_tap_upheld_ticker.png

The backend accepts an appeal reason belonging to the other party

MINOR *found by agent A6*

Appealing a resolved shot as either shooter or target (R8), including seeing the appeal budget run out.

What happens

`appeal_shot` validates only `reason in APPEAL_REASONS`, not `reason` against the appellant's role. A shooter can lodge `missed`, `not_a_player` or `already_out` on their own shot — verified twice, `200 {"appealed":true}`, and it spent a real appeal from the budget. The admin then sees the raw enum, because `ShotQueue.js`'s `APPEAL_REASON_LABELS.shooter` has no entry for those keys and falls through to the raw value: `A6-Dave (shooter): "already_out"` — screenshot `a6_37_admin_cross_party_reason.png`.

How to reproduce

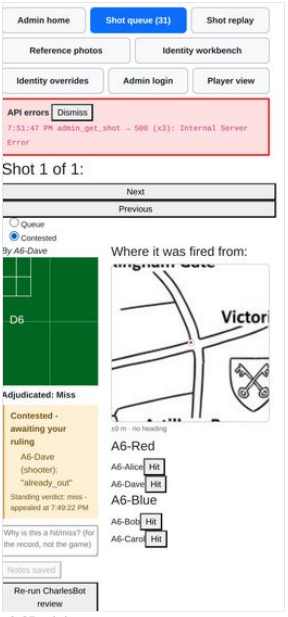
`POST /api/appeal_shot?shot_id=<your own shot>&reason=already_out`.

Where

`backend/user_interface.py:633` (reason check), `backend/user_interface.py:163 appeal_refusal` (role check exists, but only for "target may only appeal a hit"), `react-ui/src/ShotQueue.js:27` and `:447`.

Why it matters for the 19th

not reachable from the app's own UI, so it is only a hazard if somebody pokes the API — but the failure mode is an unreadable grievance in front of the admin mid-adjudication, and the fix is one membership test next to the one already there.



a6_37_admin_cross_party_reason.png

Nothing anywhere tells the admin an appeal is waiting

MINOR *found by agent A6*

The contested-shots list an appeal reopens (R8), separate from the main queue.

What happens

the nav badge reads "Shot queue (31)" — that is the **unchecked** queue only, and contested shots are checked, so the count never moves when an appeal is lodged. The "Contested" radio carries no count either. An admin working the Queue tab has no signal at all; they have to speculatively switch tabs. Confirmed on screen: with 4

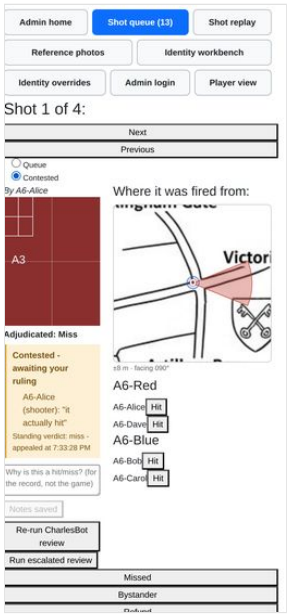
contested shots the badge counted only unchecked ones (`a6_13_admin_queue_contested.png`, `a6_37_admin_cross_party_reason.png`).

Where

`react-ui/src/AdminMode.js` (nav count), `react-ui/src/ShotQueue.js:785` (the Contested radio).

Why it matters for the 19th

R8's whole premise is that the admin stops draining every shot and handles only the contested ones. If appeals are invisible until you go looking, an appeal can sit unanswered all night while the admin believes the queue is clear — and a player who has spent a scarce appeal is waiting on it.



a6_13_admin_queue_contested.png

a6_37_admin_cross_party_reason.png

The public ticker blames CharlesBot for rulings CharlesBot never made

MINOR *found by agent A7*

`User-facing copy says "CharlesBot", never "AI", anywhere a verdict or explanation is shown (#1, #2 — "CharlesBot thinks: hit on *name*").`

What happens

every upheld appeal broadcasts, to the whole game, `The referee overturned CharlesBot: <shooter>'s shot is now ruled a hit`. The string is unconditional — it does not check whether the ruling being overturned was CharlesBot's or a human admin's. I proved the worst case live: in game `af2520c8` (`ai_shot_review_enabled: false`, so CharlesBot never even *looked* at the shot), an admin marked the shot a miss by hand, the player appealed, the admin overturned it, and the town was told CharlesBot had been overturned. `a7_player_ticker_appeal.png` shows the same message on the player HUD for my first cycle.

How to reproduce

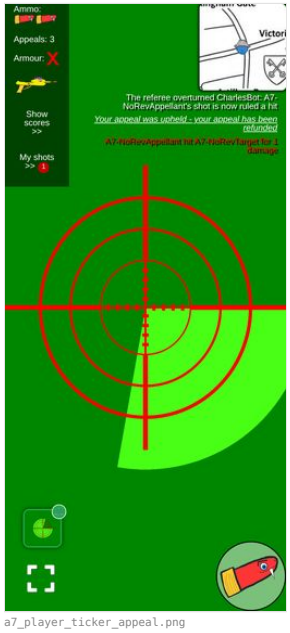
`scratch/a7_appeal2.js` — fire, `admin_mark_shot_missed`, `appeal_shot?reason=actually_hit`, `admin_shot_hit_user`, then read `ticker_messages`.

Where

`backend/ticker_message_dispatcher.py:127-130` (the template) and `backend/admin_interface.py:1249-1260`, which emits it on `if upheld:` with no regard for who made the appealed ruling. It is also the only backend user-visible CharlesBot string, and the only one with no boundary comment.

Why it matters for the 19th

on the night, auto-actions default **off**, so most rulings will be Charles's own. Every upheld appeal will publicly pin his mistake on the bot. It is funny once and confusing thereafter — and it teaches players that CharlesBot is deciding shots when it isn't.



a7_player_ticker_appeal.png

A reading that fits nobody still headlines "Probably <the player you photographed>"

MINOR *found by agent A8*

The reference-photo kit check at /admin/reference (R7): capturing a player's photo at the door and reading the resulting verdict, including the "unreadable photo identifies nobody" case.

What happens

a confident four-channel reading of colours no player is wearing (black / grey / orange / orange) makes the posterior degenerate — the likelihood is zero for every candidate, so `decode()` falls back to the flat prior and hands back p=0.20 for all five. `IdentificationVerdict` sees `matches_expected === true` (the tie happens to put the photographed player first) and prints, in bold, **"Probably A8-Ada (p=0.20), but not clearly enough to call it."** The tone is amber and the flag line does say "the colours fit no outfit cleanly", but the headline — the one thing read at the door — names the right person on evidence that fits nobody. p=0.20 is exactly 1/5, the same "prior handed straight back" tell that R7's amendment already fixed for `readable_channels == 0`; the analogous "the reading explains nobody" case has no equivalent guard.

How to reproduce

photograph any player with the stub set to `{channels: {tshirt: ["black",0.92], trousers: ["grey",0.92], hat: ["orange",0.92], armbands: ["orange",0.92]}}`. See `a8_14_verdict_matches_nobody.png`.

Where

`react-ui/src/ReferencePhotos.js:104-149` (`IdentificationVerdict`); the backend flag is `inconsistent` in `backend/reference_photos.py:_identification`.

Why it matters for the 19th

somebody turning up in the wrong colours is the exact failure the kit check exists to catch, and it is the case where the headline is least helpful. An admin skimming amber banners could read "Probably A8-Ada" as "fine, just a bit unsure" and wave them through.

a8_14_verdict_matches_nobody.png

When the player has not picked an outfit, the page hides the one candidate that matters

MINOR *found by agent A8*

The reference-photo kit check at /admin/reference (R7): capturing a player's photo at the door and reading the resulting verdict, including the "unreadable photo identifies nobody" case.

What happens

photographing `A8-Fay` (on a team, no slot) with a reading that is *exactly* A8-Ada's outfit gives amber "A8-Fay has not picked an outfit, so this photo cannot be checked against them." Underneath it lists `A8-Bo (p=0.00)`, `A8-Cy (p=0.00)`, `A8-Dee (p=0.00)` — because the runners-up are always `ranked.slice(1, 4)`, and in this branch the verdict text never mentioned `ranked[0]`. The top match, **A8-Ada at p=1.00**, is the only line worth showing and is the only one dropped.

How to reproduce

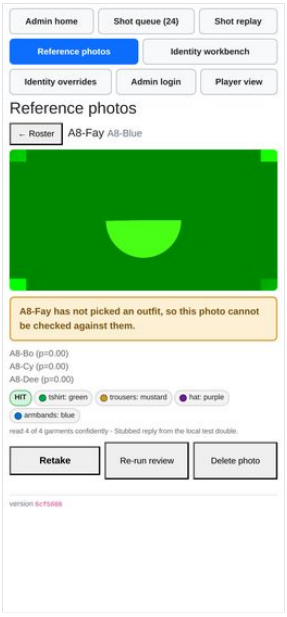
`a8_27_no_outfit_picked.png`; the stored payload has `ranked[0] = A8-Ada, p=0.99999`.

Where

`react-ui/src/ReferencePhotos.js:122` (default text) and `:156` (`ranked.slice(1, 4)`).

Why it matters for the 19th

"this person hasn't picked, and by the way they photograph as A8-Ada" is a collision to fix at the door. As it stands the admin sees three zeroes.



a8_27_no_outfit_picked.png

The review's loading state claims the vision model is not configured

MINOR *found by agent A8*

The reference-photo kit check at /admin/reference (R7): capturing a player's photo at the door and reading the resulting verdict, including the "unreadable photo identifies nobody" case.

What happens

`ReviewView` renders `state == null` as **"Not reviewed - no vision model is configured."** `state` is also null while `admin_get_reference_review` is in flight, so on a slow link opening any player shows that sentence under their stored photo before the real verdict arrives. It is a factual claim about the deployment, not a spinner — and the photo right above it correctly says "Loading the stored photo...".

How to reproduce

deterministic with a 6 s delay routed onto `admin_get_reference_review` — `a8_31_loading_says_not_configured.png`. It also happened spontaneously once on the dev server under load (`scratch/a8/s15.js` output).

Where

`react-ui/src/ReferencePhotos.js:168-173`.

Why it matters for the 19th

the door is the moment an admin decides whether CharlesBot is working. A page that says "no vision model is configured" on a laggy phone will send someone off to debug an env var mid-queue.

a8_31_loading_says_not_configured.png

The game picker forces the page to scroll sideways on a phone

MINOR *found by agent A8*

The reference-photo kit check at /admin/reference (R7): capturing a player's photo at the door and reading the resulting verdict, including the "unreadable photo identifies nobody" case.

What happens

`GameSelector`'s `<select>` labels each game as `"<uuid8> (team, team, team, …)"`. The sample game lists ten team names, so the select sizes itself to **769 px** on a 390 px viewport and `document.documentElement.scrollWidth` is 778 vs a client width of 390 — the whole exemplar page scrolls horizontally, and the selected value is clipped at the right edge. Hiding the select alone restores `scrollWidth = 390`.

How to reproduce

open `/admin/reference` with more than one game in the DB; measured in `scratch/a8/s7.js`. Visible in `a8_02_roster_mygame.png`.

Where

`react-ui/src/ReferencePhotos.js:376-397` — the select has no CSS class and `ReferencePhotos.module.css` gives it no `max-width`.

Why it matters for the 19th

on the night there will probably be one game and `GameSelector` returns null, so this may never bite. But test games survive in the DB, and a sideways-scrolling page is exactly the wrong thing on the one page that is meant to be driven one-handed.

a8_02_roster_mygame.png

The primary tap targets on the page are all under 44px tall

MINOR *found by agent A1*

Join a team via QR/link and pick an outfit at /pick (#10): ranked outfit list, canonical-first ordering, colour swatches, pagination.

What happens

measured heights — outfit row 352x**40**, "Show more outfits" 352x**33**, "Change what I own" 328x**18**, "Choose a different outfit" 352x**33**, the confirm checkbox `<input>` 13x13 (its `<label>` is 352x**18**, so the whole row is tappable but only 18px of it). Only "Lock in my choice" (352x44) and "Show me outfits" reach 44px.

How to reproduce

measure any option row on `/pick`.

Where

`react-ui/src/PickOutfit.module.css:203` (`.optionRow`, `padding: 3mm`, no `min-height`), `:118` (`.linkButton`), `:95` (`.confirmRow`).

Why it matters for the 19th

the exemplar admin page uses `min-height: 3.2em` for a roster row for exactly this reason. A 40px row that is 90% of the screen width is *findable*, so this is not a blocker, but the 18px "Change what I own" link is the recovery path when a player mis-ticks their wardrobe and it is genuinely hard to hit.

"Run escalated review" reports its one foreseeable failure as a raw browser alert()

MINOR *found by agent A10*

Running an escalated review by hand on a shot (#11 — "Run escalated review")

What happens

on a shot with no completed review the button pops a native `alert()`. Everything else on this page reports state in the page itself (the red "CharlesBot review failed" banner, the amber "Needs your call" pill). A modal alert is also the one thing that cannot be dismissed one-handed without aiming.

How to reproduce

open a shot whose `ai_review_state` is `error` or null in `/admin/shots`, press "Run escalated review".

Where

`react-ui/src/ShotQueue.js:827-829` (button) → `escalateShot()`; guard at `backend/main.py:722-730`.

Why it matters for the 19th

low. It is the correct refusal, just shouted. But the recovery ("Re-run CharlesBot review" first) is one button to the left and the alert does not say to press it in those words.

Every nav button is 36px tall, not the ">44px" the CSS says

MINOR *found by agent A12*

The admin nav (finger-sized button row) on a real phone screen, not just a desktop browser.

What happens

`AdminCommon.module.css` sizes nav links `min-height: 3em` with the comment "Comfortably above the ~44px minimum touch target". It is not: `react-ui/src/index.css` line 14-17 sets `* { font-size: 12px }` for the whole app, so 1em = 12px and 3em = **36px**. Measured with `getBoundingClientRect()` on all eight links on all seven admin pages: height 36px, widths 107-187px. That is 18% under the iOS 44pt / Android 48dp minimum. The same 12px basis shrinks the rest of the house style: on `/admin/reference` the exemplar's `3.5em` primary button and `3em` button row come out at 42px and 36px; only the roster row (`3.2em` plus padding, measured 50px) actually clears 44. Elsewhere it is worse — on `/admin/shots` the Missed / Bystander / Refund / Hit adjudication buttons measure **24px** tall and the queue/contested radios **13px**; on `/admin/replay` the buttons are 24px and the two `<select>`s 18px.

How to reproduce

load any admin page at 390x844 and measure `document.querySelector("nav a").getBoundingClientRect().height` → 36.

Where

`react-ui/src/index.css:14-17` (`* { font-size: 12px }`), `react-ui/src/AdminCommon.module.css:13-14`.

Why it matters for the 19th

the whole point of the button row is one-handed use with a box of armbands in the other hand. Every `em`-based target in the admin UI is silently 25% smaller than the author intended, and the miss-rate cost lands on the buttons that adjudicate shots. Note this is a *policy* mismatch, not a rendering artefact of the container: the numbers are CSS pixels at DPR 3, exactly what a phone gets.

The footer identifies the backend only — nothing says which frontend you are looking at

MINOR *found by agent A12*

The running app version shown on admin pages, and that it matches what is actually deployed.

What happens

`get_version` is consumed in exactly one place (`react-ui/src/AdminCommon.js:193`) and reports the backend's git rev. The React bundle carries no version of its own. Backend and frontend are separate artefacts in every deployment target (`dockerFrontend` / `dockerBackend`, the Express server in `server/`), and a browser can be holding a cached bundle in any case, so "version 6cf5608" in the footer is not evidence that the JS drawing that footer is 6cf5608.

Why it matters for the 19th

the footer exists so a screenshot of a problem says which code produced it. For a frontend bug — which is most of what a screenshot shows — it currently does not.

The git fallback hides a dirty tree

MINOR *found by agent A12*

The running app version shown on admin pages, and that it matches what is actually deployed.

What happens

`_current_version`'s fallback runs `git rev-parse --short HEAD` with no `--dirty`, so a checkout with uncommitted edits reports the last commit's hash as though it were clean. The Nix path is careful about exactly this (`dirtyShortRev`), so the two disagree about the same situation.

How to reproduce

edit any tracked file in a dev checkout, restart the backend, `GET /api/get_version` — unchanged hash.

Where

`backend/main.py:144-150`.

Why it matters for the 19th

if anything is hot-fixed on the box on the night, the version string will lie about it.

`reset_game`'s item loop is dead code

MINOR *found by agent A13*

Cuts across both: confirm the game can be reset (resetdb) and replayed from a clean state without any of the above breaking, since that is exactly what happens between the dry run and the night itself.

What happens

`python # backend/admin_interface.py:1538-1540 # Loop through all the items in this game and delete them all for item in game.items: del item del` on a loop variable unbinds the name; it deletes nothing. The intended effect happens anyway a few lines later, because every collected item is reachable through `user.items` and is deleted there (`:1572`). So the behaviour is right by accident and the comment describes something that does not happen.

Why it matters for the 19th

it does not, today. It matters the first time somebody changes how items are associated with users and quietly loses the only line that was actually doing the work.

The page departs from the ReferencePhotos house style throughout

MINOR *found by agent A2*

The identity workbench / AdminIdentity.js: viewing a player's effective appearance and any overrides, and recording an override for a misdressed player so they stay distinguishable from their teammates.

What happens

every control is a bare unstyled browser control. `Save`, `Clear outfit`, `Close`, `Suggest` and `Apply` are default ~2.2em buttons (vs the house `min-height: 3.5em` for a primary action); overrides are set with four stacked native `<select>`s inside a three-column table; the roster is a dense 4-column table with ~2.2em rows rather than one column of large targets; the suggest panel uses native checkboxes at their default ~13px hit size. The one thing it does get right is that status is said in words with a shared colour tone (`toneGood/Warning/Danger/Critical` mirroring `statusGood/Warn/Bad`) and amber consistently means "not sure".

How to reproduce

compare `A2-identity-03b-alice-editor.png` with `react-ui/src/ReferencePhotos.module.css`.

Where

`react-ui/src/AdminIdentity.module.css` — no button sizing rules at all, `.playerTable { min-width: 32em }`.

Why it matters for the 19th

CLAUDE.md says this is driven one-handed on a phone with a box of armbands in the other hand. Four native dropdowns and a 2.2em Save is the wrong instrument for that, and it sits directly next to the much bigger `Clear outfit`/`Close` pair, so the destructive-ish button is the easiest one to hit.

`collect_item` 500s on QR payloads that are not items

MINOR *found by agent A3*

Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).

What happens

`QRParser` posts **every** QR it decodes to `collect_item`. Two shapes reach an unhandled exception rather than a 4xx: a URL with no `d` parameter (`KeyError` at `backend/items.py:79`) and base64 that decodes to valid JSON that isn't an object, e.g. `MTIz` → `123` (`TypeError` at `backend/items.py:97`). Both return `500 Internal Server Error` with a traceback in the log.

How to reproduce

`scratch/a3/run2.json`, steps `tamper_url_no_d` and `tamper_number`.

Where

`backend/main.py:382-390` catches only `ValueError`.

Why it matters for the 19th

any other QR code in the street — a poster, a menu, a bus stop — pointed at by an in-game camera produces a 500 and a traceback, which is noise you have to read past when something real breaks. No player-visible harm (the scan silently does nothing either way).

The received-`unreviewed` bubble state ("Shot at you") cannot occur

MINOR *found by agent A4*

The shot status bubble after taking a shot — every visual state it can be in, and that it stays on screen rather than disappearing.

What happens

`receivedStatus` has a non-hit branch — grey ⏳, "Shot at you - shot by X" — but nothing can produce it. A shot only enters `get_shots_received` once `target_user_id` is set, which only `hit_user` does, and it sets `result = "hit"` at the same time; any later re-ruling that is not a hit clears `target_user_id` and removes the row entirely. So the branch and its `stateUnreviewed` styling are dead.

Where

`react-ui/src/ShotHistory.js:80-91`; `backend/user_interface.py:589-607`; `backend/admin_interface.py:1184-1188`.

Why it matters for the 19th

harmless in itself, but it reads as if targets get told about shots taken at them before adjudication, and they do not. If that *was* the intent it is a missing feature, not a rendering bug.

At real phone size the bubble is colour-only — the glyph does not read

MINOR *found by agent A4*

The shot status bubble after taking a shot — every visual state it can be in, and that it stays on screen rather than disappearing.

What happens

the bubble is `4.5em` and its status disc `1.6em`. With `index.css:17` setting the root font to **12px**, that is a 54x54px tile with an **19px** disc. In the true-size 390x844 renders (`scratch/a4/view/a4_50_…png`, `a4_70_…png`) the border colour reads instantly but the emoji inside — 💥, ⚖️, 🤖, 👁, ⏳ — is a smudge; I could only identify them from the 3x crops. Two states also share a colour and differ only by that glyph: `escalated` and `appealOpen` are both `#f0a020`, i.e. 🤖 "CharlesBot has a guess" and ⚖️ "you have an appeal running" look identical at arm's length.

Where

`react-ui/src/ShotHistory.module.css:290-320`; `index.css:17`.

Why it matters for the 19th

`CLAUDE.md` states the house rule as "Status says the state in words … never a bare icon or colour alone", and the bubble is the only shot feedback a player sees without opening anything. The words are one tap away in the popup, so this is a judgement call rather than a defect — but amber-vs-amber is worth splitting.

Every tap target in the shot-history popup is under the 44px minimum

MINOR *found by agent A4*

The shot status bubble after taking a shot — every visual state it can be in, and that it stays on screen rather than disappearing.

What happens

measured in the running app — "Appeal" **327x36**, "<< All shots" **30px** tall, each appeal-reason row **36px**, the popup's exit "X" **36x36**. The CSS asks for `min-height: 3em`, which the house style describes as "comfortably past the 44px touch minimum", but the root font is 12px so 3em is 36px. (The bubble itself, at 54px, is fine, and the shot rows at 61px are fine.)

How to reproduce

`scratch/a4/t9_measure.js`.

Where

`react-ui/src/ShotHistory.module.css:205-260`, `react-ui/src/Popup.module.css:19-33`, `react-ui/src/index.css:17`.

Why it matters for the 19th

this is the control a player uses one-handed, mid-game, to spend one of three appeals, and the confirmation step's radio rows are the same size. A mis-tap either lodges the wrong reason or closes the popup.

Ticker text is locked to 12px regardless of the phone's text-size setting

MINOR *found by agent A5*

The ticker feed for game announcements.

What happens

`react-ui/src/index.css:14-17` sets `* { font-size: 12px }`, so every ticker line (and the whole UI) renders at 12 px CSS on every device and ignores the player's own accessibility text-size setting. Measured line boxes are 18 px tall.

Where

`react-ui/src/index.css:14-17`.

Why it matters for the 19th

the ticker is how "the circle has changed" reaches a player, read at arm's length, at night, while walking. 12 px is small for that, and a player who has turned their phone's text up gets nothing.

The public overturn line misattributes the call, and says nothing when the hit only moves

MINOR *found by agent A6*

Appealing a resolved shot as either shooter or target (R8), including seeing the appeal budget run out.

What happens

`APPEAL_UPHELD` reads "The referee overturned CharlesBot: {user}'s shot is now ruled {result}". Two problems seen in my ticker dumps: (1) CharlesBot was never involved in any of these shots — a human admin made both the original and the new call — yet it is named as the party overturned. With `ai_auto_actions_enabled` off (the default, and the plan for the 19th) this is true of *every* overturn. (2) When A1 moved from a hit on Bob to a hit on Carol, the line read "…A6-Alice's shot is now ruled **a hit**" — it was already ruled a hit; `_APPEAL_RESULT_WORDS` names the verdict, never the target, so the wrong-target correction announces nothing. ("miss → Refund" prints "now ruled **unreadable**", which is at least deliberate.)

How to reproduce

re-rule any contested hit onto a different player and read the public ticker.

Where

`backend/ticker_message_dispatcher.py:127`, `backend/admin_interface.py:136 _APPEAL_RESULT_WORDS`, `backend/admin_interface.py:1250`.

Why it matters for the 19th

the public line is, by R8's own words, "the correction's social event". Announcing a correction that names no change, and blaming a bot that had no part in it, spends that moment on nothing.

An admin error dialog says "AI" twice, and points at a button that no longer has that name

MINOR *found by agent A7*

*User-facing copy says "CharlesBot", never "AI", anywhere a verdict or explanation is shown (#1, #2 — "CharlesBot thinks: hit on *name*").*

What happens

Pressing **Run escalated review** on a shot with no completed review pops a `window.alert` reading verbatim: > `This shot has no completed AI review to escalate from - run the AI review first` Confirmed live, not just by grep — see `scratch/a7_dialogs.json` and the run log of `scratch/a7_alert.js`. It is the only user-visible string in the whole app that says "AI". It also misdirects: the button it tells the admin to press is labelled **"Re-run CharlesBot review"**.

How to reproduce

create a game with the CharlesBot-review toggle left off, have a player fire, open `/admin/shots`, page to that shot, press "Run escalated review". (`OPENROUTER_ESCALATION_MODEL` must be set, as it is in this container — otherwise you get the "No escalation model configured" message instead.)

Where

`backend/main.py:726-729` (the `HTTPException(400, ...)` detail), surfaced by `react-ui/src/ShotQueue.js:712-719`, which does `window.alert(body?.detail || "Could not escalate this shot")` — every `detail` from that endpoint is admin-facing copy by construction. The stale name also survives in two docstrings that FastAPI publishes at `/docs`: `backend/admin_interface.py:474` and `:657` both call the button "Re-run AI review".

Why it matters for the 19th

it is the one place the convention actually breaks, and it breaks in the worst spot — mid-queue, one-handed, when the admin is already stuck. Two-word fix.

`* { font-size: 12px }` shrinks the exemplar's own touch targets

MINOR *found by agent A8*

The reference-photo kit check at /admin/reference (R7): capturing a player's photo at the door and reading the resulting verdict, including the "unreadable photo identifies nobody" case.

What happens

`react-ui/src/index.css:14-17` sets `font-size: 12px` on `*`, so every `em` in `ReferencePhotos.module.css` resolves against 12 px (13.2 px inside `.bigButton`, which sets `font-size: 1.1em`). Measured with `boundingBox()` at 390x844: | element | CSS | resolves to | rendered height | | --- | --- | --- | --- | | `.bigButton` "Photograph X" | `min-height: 3.5em` | 46.2 px | 46.2 px | | `.buttonRow button` (Re-run / Delete) | `min-height: 3em` | 36 px | 52.8 px (label wraps) | | `.rosterRow` | `min-height: 3.2em` | 38.4 px | 50.1 px (content) | | `.backButton` "← Roster" | `min-height: 2.6em` | **31.2 px** | **31.2 px** | The roster rows and the capture button still clear 44 px on content and padding, so the page is usable. The **back button does not** — 31 px, and it is the control pressed once per player. The row also renders unevenly: "Retake" is 39.6 px next to two 52.8 px siblings, because `.buttonRow button` (0,1,1) beats `.bigButton` (0,1,0) on the min-height.

Where

`react-ui/src/index.css:14-17`; `react-ui/src/ReferencePhotos.module.css:38, 110, 129, 144`.

Why it matters for the 19th

CLAUDE.md names this file as the house style and quotes those `em` values as "comfortably past the 44px touch minimum". They are not, at the app's actual base font — worth knowing before another admin page is built by copying them.

Every runner-up prints `p=0.00`

MINOR *found by agent A9*

The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.

What happens

`candidateFacts` formats the posterior with `toFixed(2)`, so on a confident reading the ranking reads `1.00, 0.00, 0.00, 0.00, 0.00` — the order is visible but the magnitudes are gone, and a genuine second place at 0.004 looks the same as one at 2e-8. The headline's two-candidate branch uses `toFixed(1)`, so a 0.51/0.45 near-tie and a 0.49/0.49 dead heat both read "(0.5) or (0.5)".

Where

`react-ui/src/ShotQueue.js:565-572` (`candidateFacts`), `:160-166` (`charlesBotVerdict`).

Why it matters for the 19th

minor, because the ambiguity flags carry the real signal. But "0.00" reads as "impossible" rather than "unlikely", which is the wrong impression for the runner-up the admin might actually want.

The queue is global across games, and its two counters disagree

MINOR *found by agent A9*

The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.

What happens

`admin_get_shots_info` takes no game id, so "Shot queue (31)" and "Shot 18 of 31" count every game's shots — during this walkthrough my four shots sat among eleven other agents' at indices 17–20, reachable only by pressing Next seventeen times. There is no jump-to and no per-game filter. Separately, with "Show adjudicated shots" ticked the header said `Shot 79 of 79` while the nav badge still said `(35)`.

Where

`backend/main.py:526-528`; `react-ui/src/ShotQueue.js:750-756`.

Why it matters for the 19th

one live game means one queue, so this is mostly harmless — but Next/Previous being the only navigation means a 30-deep backlog is 30 taps from the end, and the two different totals on screen at once will be read as a bug by whoever is running the night.

"Missed", "Bystander" and "Refund" are three identical grey bars

MINOR *found by agent A9*

Recording a "hit a bystander" outcome on a shot.

What happens

the three rulings at the foot of the queue are visually indistinguishable — same width, same 24 px height, same default browser grey, no colour, no icon, no sublabel. Nothing on screen says that Bystander and Missed are two different rulings for appeal purposes, or that Refund (alone) hands the bullet back. The bot's *own* outcome tags are colour-coded (`outcomeHit` green, `outcomeBystander` amber, `outcomeMiss` grey) — the human's buttons are not.

Where

`react-ui/src/ShotQueue.js:861-884`; the tag palette that already exists is `react-ui/src/ShotQueue.module.css:41-55`.

Why it matters for the 19th

`CLAUDE.md` says miss and bystander "count as one ruling" at appeal *only* in the sense that both mean "hit no player" — but they are separately recorded and separately shown to the player, so picking the wrong one puts a wrong sentence on the shooter's phone. Three identical bars stacked in tap order is how that happens.

Cosmetic findings (26)

`body` is white behind a black page

COSMETIC *found by agent A1*

Join a team via QR/link and pick an outfit at /pick (#10): ranked outfit list, canonical-first ordering, colour swatches, pagination.

What happens

`.outerContainer` paints black but `document.body` is `rgb(255,255,255)`, so any rubber-band overscroll exposes a white band. Caught in `A1_25_empty_state.png` (white strip along the top).

Where

`react-ui/src/PickOutfit.module.css:6` — the black lives on the page container, not on `body`.

Why it matters for the 19th

a white flash on a night-time black UI, on every scroll bounce. Purely cosmetic.

A1_25_empty_state.png

Accuracy of exactly 0 draws no circle at all

COSMETIC *found by agent A11*

The per-shot map (R5) showing GPS accuracy and the shooter's compass heading.

What happens

`accuracyRadiusPx` is 0, which is falsy, so the circle is skipped entirely — the map looks identical to the "accuracy unknown" case while the caption says `±0 m`.

How to reproduce

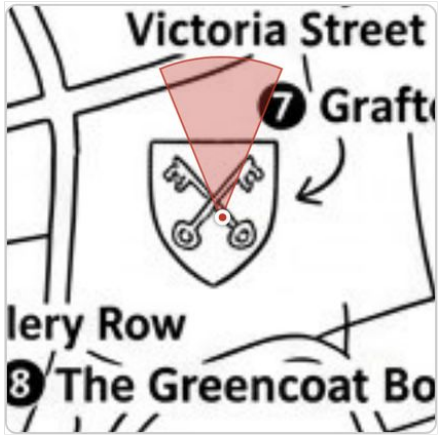
`set_location?...&accuracy=0`, then fire. `A11-map-h000-acc0.png`.

Where

`react-ui/src/ShotMap.js:133` (`{accuracyRadiusPx ? (`).

Why it matters for the 19th

essentially not at all — no browser reports 0. Noted only because it is the one falsy-vs-null slip in an otherwise careful null-handling job.



A11-map-h000-acc0.png

The nav's "Identity workbench" is the sandbox, not this page

COSMETIC *found by agent A2*

The identity workbench / AdminIdentity.js: viewing a player's effective appearance and any overrides, and recording an override for a misdressed player so they stay distinguishable from their teammates.

What happens

the R9 line says "the identity workbench / AdminIdentity.js", but `/admin/identity` ("Identity workbench" in the nav) is `IdentityDemo` — a scheme sandbox that explicitly "touches nothing in the database" and shows no real players. The page the line describes is `/admin/identity-overrides`, labelled "Identity overrides". The sandbox is also laid out for a desktop: `scrollWidth 610` at 390px, with the channels table's "labels" column running off the screen (`A2-workbench-01.png`).

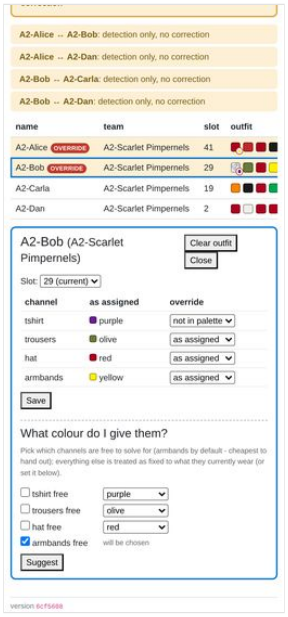
Where

`react-ui/src/AdminIdentity.module.css:189` — `.editorHeader` is a `display: flex` with `justify-content: space-between` and no `flex-wrap` handling for the button group.

Why it matters for the 19th

an admin looking for "the workbench" on the night lands on the page that cannot help them.

A2-workbench-01.png



A2-identity-09-outside-palette.png

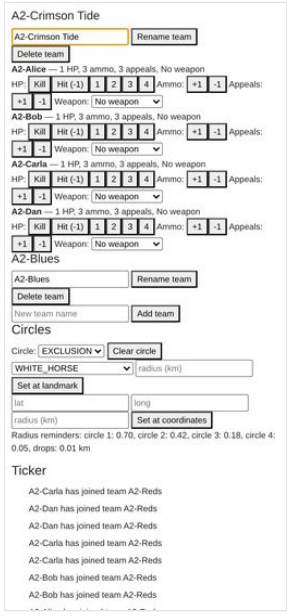
Historical ticker lines keep the old team name

COSMETIC *found by agent A2*

Renaming a team from the admin dashboard.

What happens

the ticker still reads "A2-Carla has joined team A2-Reds" after the rename (`A2-rename-03-after.png`). The messages are stored text, so this is expected — noting it only so it is not mistaken for a stale cache. The `touch_game_ticker_tag()` in `set_team_name` re-tags the ticker but does not rewrite past entries.



A2-rename-03-after.png

Ticker text is white-on-white when the camera is pointed at anything bright

COSMETIC *found by agent A3*

Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).

What happens

the ticker has no background panel — white text with a 2 px black shadow straight over the camera feed. The HUD panel next to it has `rgba(0,0,0,0.5)` behind it and is fine.

How to reproduce

`a3_40_camera_sees_qr.png` — the camera is looking at a printed white QR poster and the three ticker lines are barely readable. That is the normal posture for collecting loot.

Where

`react-ui/src/TickerView.module.css:1-5`.

a3_40_camera_sees_qr.png

On the knocked-out screen the headline is the smallest text

COSMETIC *found by agent A3*

Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).

What happens

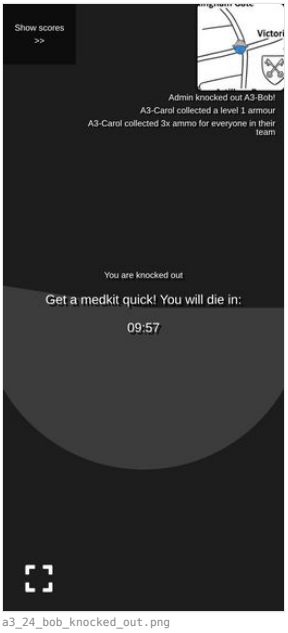
"You are knocked out" renders smaller than the two `.textSmaller` lines beneath it. `index.css` sets `* { font-size: 12px }`, which beats inheritance for the unclassed `<p>`, while the two lines below carry `.textSmaller` (`large`, 18 px) and win. The intended hierarchy is inverted; the heavy 0.1em drop shadow (from `.centeringDiv`, computed at the `xx-large` size) makes it look doubled.

How to reproduce

`a3_24_bob_knocked_out.png`.

Where

`react-ui/src/GuideImages.module.css:1-13` vs `react-ui/src/index.css:16-20`.



a3_24_bob_knocked_out.png

The popup is 70%-opaque over a live camera feed, and the ticker bleeds through

COSMETIC *found by agent A4*

The shot status bubble after taking a shot — every visual state it can be in, and that it stays on screen rather than disappearing.

What happens

`Popup.module.css` `outerContainer` is `rgba(0,0,0,0.7)`, so the crosshair's red arms and the white ticker lines show through the shot list and the detail view. In `a4_21_popup_list.png` the ticker strings sit visibly across the "Ammo refunded" and "You shot a bystander!" rows. Readable, but noisy — and my fake camera is a flat green field, so a real street scene will be worse, not better.

Where

`react-ui/src/Popup.module.css:12-18`.

a4_21_popup_list.png

The public "appeal upheld" ticker line always blames CharlesBot

COSMETIC *found by agent A4*

The shot status bubble after taking a shot — every visual state it can be in, and that it stays on screen rather than disappearing.

What happens

after an admin overruled a call CharlesBot had never seen (recognition was off for that shot), the public ticker read "The referee overturned CharlesBot: A4-Bob's shot is now ruled a miss" — visible in `a4_70_knocked_out.png`. The template is unconditional.

Where

`backend/ticker_message_dispatcher.py:127-130`.

Why it matters for the 19th

it is a public line, and on the night most calls will be the admin's own. Blaming the robot for a human's reversal is funny once and confusing after that.

a4_70_knocked_out.png

The popped-out map wastes more than half the panel on blank white

COSMETIC *found by agent A5*

The map view, including drop/circle locations on the live Westminster venue map (#12).

What happens

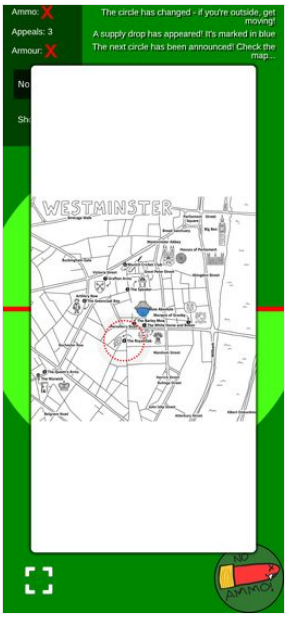
the popped-out map is 80vw x 80vh (312 x 675 px at 390x844), but the map image is fitted to the *width*, so it is drawn 312 x 312 with ~180 px of empty white above and below. Visible in `a5_clear_exclusion.png` and `a5_popped_all_circles.png`. When an exclusion circle is set, that blank margin is tinted red too, which makes the panel look like map you cannot reach.

Where

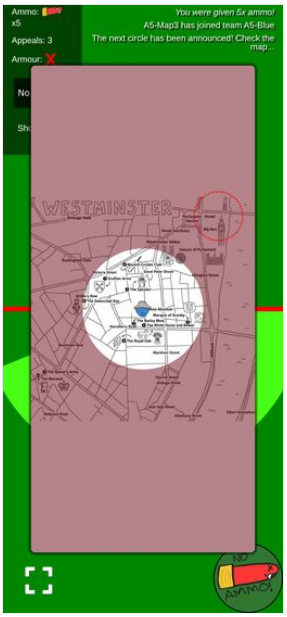
`react-ui/src/MapView.js:265-268` — `box_width_km = max(MAP_WIDTH_KM, MAP_HEIGHT_KM * aspect)`, i.e. fit-to-width, against a square 1.3 x 1.3 km Westminster crop in a portrait box.

Why it matters for the 19th

the map is the thing players squint at in the dark; it is currently drawn at 46% of the space the pop-out already takes. It is pinch-zoomable, so this is a default, not a wall.



a5_clear_exclusion.png



a5_popped_all_circles.png

The confirmation's scroll chevron sits on top of the appeal count

COSMETIC *found by agent A6*

Appealing a resolved shot as either shooter or target (R8), including seeing the appeal budget run out.

What happens

on the confirm step the popup overflows, so `Popup`'s floating chevron appears — centred, directly over "Are you sure? You have 3 of 3 appeals left.", hiding two words of it (`a6_04_alice_A3_confirm.png`).

Where

`react-ui/src/Popup.js:90 scrollHint`.

Why it matters for the 19th

it obscures precisely the sentence that makes the spend informed, on the screen where the spend happens.

a6_04_alice_A3_confirm.png

The queue calls the escalation "the stronger model", one line under a sentence that calls it CharlesBot

COSMETIC *found by agent A7*

*User-facing copy says "CharlesBot", never "AI", anywhere a verdict or explanation is shown (#1, #2 — "CharlesBot thinks: hit on *name*").*

What happens

in the same panel, the verdict sentence says "CharlesBot thinks: hit on A7-Target" and the escalation block below it is headed "**STRONGER MODEL**", with "Escalated to the stronger model - reviewing..." while it is in flight and "Escalation failed: ..." when it errors. The admin toggle for the same feature says "Hard shots escalate to a stronger **CharlesBot** model", so the product has two names for one thing on two adjacent screens. Nothing here says "AI", so this is a consistency finding, not a violation.

How to reproduce

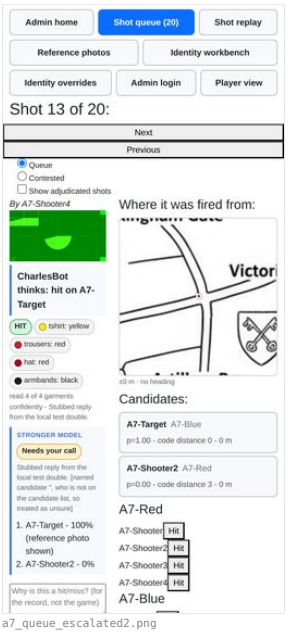
`a7_queue_escalated2.png` — "CharlesBot thinks" and "STRONGER MODEL" are ~2cm apart on a 390px viewport.

Where

`react-ui/src/ShotQueue.js:215` (`Stronger model` label), `:190-193` (pending text), `:200-202` (error text).

Why it matters for the 19th

small, but an admin under pressure reading "stronger model" beside "CharlesBot" has to work out whether those are the same robot. "CharlesBot, second opinion" would settle it.



a7_queue_escalated2.png

The reference-photo page shows a CharlesBot reading but never names it, and has no boundary comment

COSMETIC *found by agent A7*

`User-facing copy says "CharlesBot", never "AI", anywhere a verdict or explanation is shown (#1, #2 — "CharlesBot thinks: hit on *name*").`

What happens

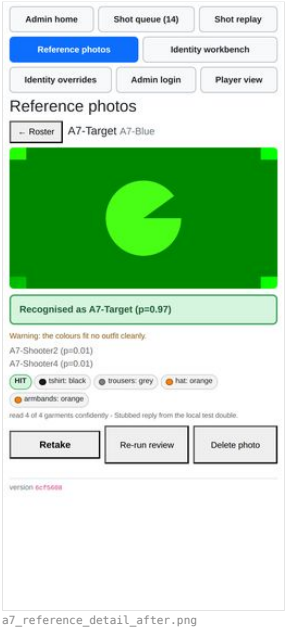
`/admin/reference` runs the same vision path and prints "Recognised as A7-Target (p=0.97)", "Warning: ...", the outcome/colour tags and "read 4 of 4 garments confidently"; the buttons are "Re-run review" and the empty state is "Not reviewed - **no vision model is configured**". CharlesBot is not named anywhere on the page, and the file carries none of the four `"CharlesBot" is the display name for what the API calls ai_review (#1)` boundary comments the convention asks for at each display site. Same on the identity workbench, which says "pretend this came from the image recognition" and "Pick what **the vision model** 'saw' per channel".

Where

`react-ui/src/ReferencePhotos.js:169-173` and `:390`-ish (button row); `react-ui/src/IdentityDemo.js:379`.

Why it matters for the 19th

low. But if a player at the door asks who decided their green was khaki, the page gives the admin no name to give them. Also worth noting that the queue's `outcomeTag` is reused verbatim here, so a kit-check photo is labelled a green **"HIT"** — see `a7_reference_detail_after.png`. That word means nothing at the door.



a7_reference_detail_after.png

"Review failed" is red, where the page's own rule says amber

COSMETIC *found by agent A8*

The reference-photo kit check at /admin/reference (R7): capturing a player's photo at the door and reading the resulting verdict, including the "unreadable photo identifies nobody" case.

What happens

a vision-call failure renders `styles.statusBad` in the roster and a red error box in the detail view, the same red used for "✗ Reads as somebody else". The page's own comment says green and red are for answers and amber is for everything the machine is not sure of; a 500 from the model is not an answer about the player at all.

How to reproduce

stub `{fail: true}` + "Re-run review" — `a8_17_verdict_error.png`, roster row in `a8_22_pending_roster.png`.

Where

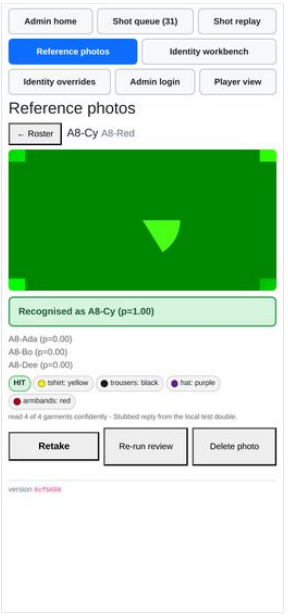
`react-ui/src/ReferencePhotos.js:47`, `:177-182`.

Why it matters for the 19th

at a glance a red pill on the roster means "this player's kit is wrong". "The API fell over" wearing the same colour will send an admin to re-dress a player who is fine. (The failure itself is reported well — no infinite spinner, and re-running with a working model recovers a full verdict: `a8_29_error_recovered.png`.)

a8_17_verdict_error.png

a8_22_pending_roster.png



a8_29_error_recovered.png

No way back to the curated view once "Show more outfits" is tapped

COSMETIC *found by agent A1*

The "recommended" vs "not ideal" outfit badges and the "show non-recommended outfits" reveal link.

What happens

`showingAll` is only reset by `reopenWardrobe()`, so after revealing the long tail the only route back to the seven good options is "Change what I own" (which discards nothing but re-opens the whole form).

Where

`react-ui/src/PickOutfit.js:385-388`.

The save button reads "Notes saved" on a shot that has never had a note

COSMETIC *found by agent A11*

Admin shot history and notes on a player.

What happens

with an empty textarea and nothing typed, the disabled button says "Notes saved" — asserting something that did not happen. It only means "not dirty".

Where

`react-ui/src/ShotQueue.js:394`.

"Player view" is a one-way trip

COSMETIC *found by agent A12*

The admin nav (finger-sized button row) on a real phone screen, not just a desktop browser.

What happens

the nav's last button goes to `/`, and the player view has no link back to `/admin` (only `AdminLogin.js:13` and `IdentityDemo.js:922` link to it). Browser Back works, so this is recoverable, but there is no on-page route back.

Why it matters for the 19th

minor annoyance for an admin who is also playing.

The replay error text is a double-escaped JSON blob

COSMETIC *found by agent A12*

The replay workbench at /admin/replay (R1) — trialling a prompt/zoom/schema change against real shots without it touching stored data.

What happens

the card shows `Replay failed: 502: {"detail":"Replay failed: vision call failed after 3 attempts: OpenRouter returned 500: {\"error\": {\"message\": \"stub configured to fail\"}}"}` — the status line, the FastAPI envelope and the provider's JSON, nested. Readable, but on a 390px screen it is four wrapped lines of punctuation.

Where

`react-ui/src/ShotReplay.js` (`status: "error"`, `${response.status}: ${await response.text()}`).

The rename form is fine but small, and there is no feedback that it worked

COSMETIC *found by agent A2*

Renaming a team from the admin dashboard.

What happens

at 390px the input and `Rename team` sit on one line and are tappable, but the button is ~2.2em, and submitting produces no toast or confirmation — you infer success from the `<h4>` above changing. `Delete team`, which deletes the players too, is the same size and directly below it.

Where

`react-ui/src/AdminMode.js:119-164` (`TeamSection`).

The cooldown and no-ammo refusals are pictures, not words

COSMETIC *found by agent A4*

Taking a shot photo of another player, including the on-screen crosshair overlay.

What happens

a second press during the timeout is refused by disabling the button and swapping the art to a sleeping bullet with a white progress ring; zero ammo swaps it to a dead bullet reading "NO AMMO!" in hand-drawn dark type. Nothing is written anywhere else on screen. See `scratch/a4/view/montage_firebutton.png` — at 390px the two refusal states are the same mid-green disc with a similar red bullet on it, and the "NO AMMO!" lettering is dark-on-green with fairly low contrast. It is legible in my fake green feed, and it is on-style for this game, but it is the only signal there is.

Where

`react-ui/src/FireButton.js:71-84`.

A re-acquired sentinel replaces the old one without releasing it

COSMETIC *found by agent A5*

The screen staying awake during play (R3 — Screen Wake Lock), and that the phone's own lock button still turns the screen off on request.

What happens

`acquire()` overwrites `sentinel` with the new lock. On a real device the old one has already been auto-released by the browser when the page was hidden, so nothing leaks; in this container, where the page never truly hides, the previously held native sentinel stayed unreleased after re-acquisition (observed: 2 unreleased sentinels).

Where

`react-ui/src/useWakeLock.js:15-21`.

Why it matters for the 19th

it doesn't, on a phone. Noted only so nobody re-discovers it as a "leak" when reading the hook.

The corner mini-map is flush against the top and right edges of the screen

COSMETIC *found by agent A5*

The map view, including drop/circle locations on the live Westminster venue map (#12).

What happens

the mini-map's box sits at left 270 / right 390 / top 0 in a 390-wide viewport — no margin at all, so its `box-shadow` (`0 0 0.2em 0.2em`) is cut on those two sides. Nothing overflows and no map content is lost (`scrollWidth === innerWidth`).

Where

`react-ui/src/UserMode.module.css:1-8` (`monitorsContainer`, `justify-content: space-between`, no padding) with `react-ui/src/MapView.module.css:19-22` (`10em` square, fixed 120 px because `index.css` sets `* { font-size: 12px }`).

Why it matters for the 19th

on a notched phone in fullscreen the top-right corner of the map will sit under the status bar / rounded display corner. This is what the earlier smoke test saw; it is a margin problem, not a clipping bug.

`admin_give_appeals` announces the raw number, including negatives

COSMETIC *found by agent A6*

The contested-shots list an appeal reopens (R8), separate from the main queue.

What happens

taking appeals away sends the player "You were given **-5x** appeals!" (seen in Dave's ticker), and the balance floors silently at 0 so the number in the message is a lie as well as ungrammatical.

Where

`backend/admin_interface.py:1127`, `backend/user_interface.py:315` (`award_appeals`, `max(0, ...)`).

The shooter's own history prints raw enum words where the admin gets a sentence

COSMETIC *found by agent A7*

*User-facing copy says "CharlesBot", never "AI", anywhere a verdict or explanation is shown (#1, #2 — "CharlesBot thinks: hit on *name*").*

What happens

`react-ui/src/ShotHistory.js:131` renders `CharlesBot thinks: ${shot.ai_suggestion}` unchanged for the non-hit cases, so the player reads "CharlesBot thinks: miss" and "CharlesBot thinks: bystander" where the admin queue says "CharlesBot thinks: miss" and "CharlesBot thinks: that's a bystander, not a hit". "CharlesBot thinks: bystander" is not a sentence.

Where

`react-ui/src/ShotHistory.js:125-140` vs `react-ui/src/ShotQueue.js:176-179` (`charlesBotVerdict`, which is exported and could be shared).

Why it matters for the 19th

trivial, but it is player-facing, and the shared renderer to fix it already exists.

A raw OpenRouter error is shown to the admin verbatim

COSMETIC *found by agent A7*

*User-facing copy says "CharlesBot", never "AI", anywhere a verdict or explanation is shown (#1, #2 — "CharlesBot thinks: hit on *name*").*

What happens

when the vision call fails the queue shows `CharlesBot review failed: vision call failed after 3 attempts: OpenRouter returned 500: {"error": {"message": "stub configured to fail"}}` — the prefix obeys the convention, the payload leaks the provider's name and a JSON blob. Observed live in `scratch/a7_text_queue_named2.txt` (another agent had the stub in `fail` mode at the time).

Where

`react-ui/src/ShotQueue.js:95-99`, printing `review.error` as-is.

Why it matters for the 19th

nothing breaks; it just means the one place the admin sees a failure is also the one place the abstraction leaks.

A green "HIT" tag on a kit check

COSMETIC *found by agent A8*

The reference-photo kit check at /admin/reference (R7): capturing a player's photo at the door and reading the resulting verdict, including the "unreadable photo identifies nobody" case.

What happens

`outcomeTag(review)` is reused from the shot queue, so every reference photo carries a green **HIT** pill — including under the amber "No garment could be read in this photo" verdict, where it is the only green thing on the page. "HIT" is a shot-adjudication word ("the photograph shows a person") and means nothing at the door.

Where

`react-ui/src/ReferencePhotos.js:197` (`outcomeTag`), rendered from `react-ui/src/ShotQueue.js:271`.

Why it matters for the 19th

minor, but it is one green pill on a page whose whole colour language is "green means yes".

The human verdict is plain black text, not a status pill

COSMETIC *found by agent A9*

Recording a "hit a bystander" outcome on a shot.

What happens

`Adjudicated: Bystander` renders as unstyled bold text above the CharlesBot block, while the machine's opinion below it gets a coloured pill. The page gives more visual weight to the guess than to the ruling.

Where

`react-ui/src/ShotQueue.js:813-815` (`styles.verdict`); the `statusGood` / `statusWarn` / `statusBad` tone the house style asks for is in `react-ui/src/ReferencePhotos.module.css` / `AdminIdentity.module.css`.

Why it matters for the 19th

small, but it is the one line that says what actually happened to a shot, and `CLAUDE.md` names this exact pattern as the house style.

Open questions (13)

Not bugs — decisions for Charles.

Nothing on the page says what "not ideal" means

QUESTION *found by agent A1*

The "recommended" vs "not ideal" outfit badges and the "show non-recommended outfits" reveal link.

What happens

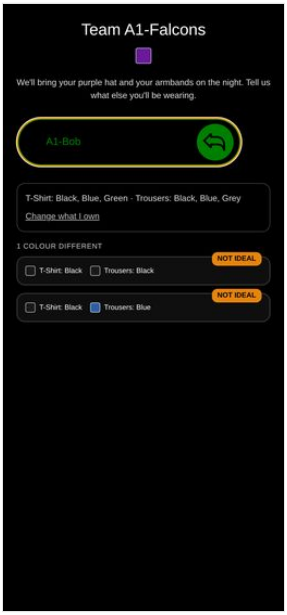
the badge is the only signal. There is no legend, no tooltip, no sentence under the "1 COLOUR DIFFERENT" heading explaining that a non-canonical outfit costs identification accuracy. A player whose wardrobe supports no canonical outfit (A1-Bob: 2 options, both NOT IDEAL, no explanation, no recommended anchor to compare against — `A1_22_bob_options.png`) is shown two orange warning badges and left to guess whether he has done something wrong.

Where

`react-ui/src/PickOutfit.js:188-212` (`OptionRow`); the group heading at `:236-242` says "1 colour different" but never says different *from what*.

Why it matters for the 19th

it is the second and third person from each team who will see mostly NOT IDEAL rows, and they will ask the door staff.



A1_22_bob_options.png

`admin_reset_game` and `resetdb` leave very different states, and nothing says so

QUESTION *found by agent A13*

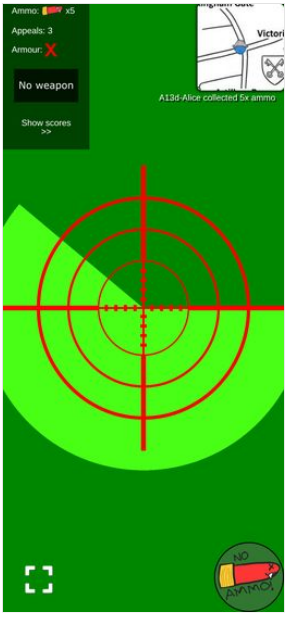
Cuts across both: confirm the game can be reset (resetdb) and replayed from a clean state without any of the above breaking, since that is exactly what happens between the dry run and the night itself.

What happens

the two "resets" are not the same operation and the admin has no way to learn the difference except by reading the code. Measured side by side: | | `admin_reset_game` (the button) | `npm run resetdb` | | --- | --- | --- | | games / teams | kept | destroyed and re-created with **new team UUIDs** | | usernames | kept | gone | | identity slot / outfit picked | **kept** | gone | | reference photos | wiped | gone | | shots, ticker, HP, ammo, appeals | reset | gone | | collected items | deleted, so the QR works again | gone, so the QR works again | | **printed join QRs** | **still valid** | **dead** | | player's open phone | keeps working, correctly | shows a stale game forever | `a13_20_after_admin_reset_game.png` is the player's screen straight after `admin_reset_game`: clean ticker, correct ammo, fully playable, no reload needed. The button's own confirm text ("Wipes scores, shots, collected items and the ticker. Keeps teams and usernames.", `react-ui/src/AdminMode.js:259`) is accurate — but it is the *only* place either operation is described, and it does not warn that the other reset invalidates the QRs.

Why it matters for the 19th

given the join-QR finding above, `admin_reset_game` is almost certainly the right reset to use between the dry run and the night: it clears the state that matters and keeps the printed codes alive. That is a decision worth making deliberately rather than by reflex, and right now nothing in the app tells the admin the choice exists.



a13_20_after_admin_reset_game.png

Outside the exclusion circle, the corner map is a featureless red square

QUESTION *found by agent A5*

The map view, including drop/circle locations on the live Westminster venue map (#12).

What happens

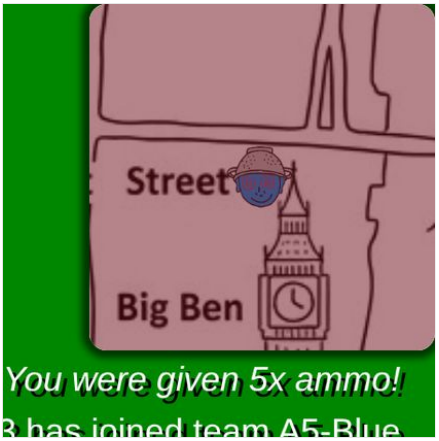
the exclusion overlay is a 10000 px box-shadow, so once the player is more than ~100 m outside the circle the entire 0.2 km-wide corner map is tinted with no boundary visible and no direction cue (`a5_corner_bigben.png`, `a5_corner_warwick.png` — the map is legible through the tint, but the circle edge is nowhere on it). The player has to pop the map out to find out which way to run.

Where

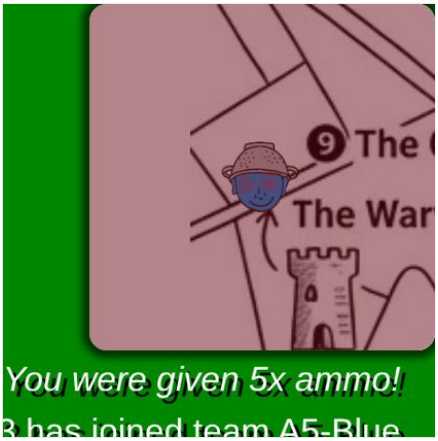
`react-ui/src/MapView.module.css:58-64` plus the 0.2 km corner window (`corner_width_km` in `backend/venues.py`).

Why it matters for the 19th

"get moving" is exactly the moment the corner map is supposed to answer a question, and it is the moment it stops being able to. Raised as a question because the pop-out does answer it and this may be an accepted trade.



a5_corner_bigben.png



a5_corner_warwick.png

The roster gives no sense of how much of the door queue is done

QUESTION *found by agent A8*

The reference-photo kit check at /admin/reference (R7): capturing a player's photo at the door and reading the resulting verdict, including the "unreadable photo identifies nobody" case.

What happens

rows are ordered by team then name, with no count and no grouping of the unchecked. With five players that is fine (`a8_15_roster_mixed.png`); with twenty-five arriving in a rush, "who have I not done yet" means scanning every row for the grey "No photo yet" pill.

Where

`react-ui/src/ReferencePhotos.js:69-95` (`Roster`); the ordering comes from `AdminInterface.get_reference_photo_status` (`backend/admin_interface.py:798`, `order_by(Team.name, User.name)`).

Why it matters for the 19th

the stated job is one-handed with a box of armbands. A "6 of 24 checked" line, or unchecked-first ordering, is the difference between a glance and a scan. Raising it as a question rather than a bug — it may simply not matter at the real headcount.

a8_15_roster_mixed.png

The "You're set" screen is a dead end, and never names the armband colour

QUESTION *found by agent A1*

The name-then-confirm flow — entering a name, seeing the committed outfit, and the outfit becoming locked in.

What happens

`ResultScreen` shows only the wardrobe channels (T-shirt, trousers) plus "This is final. Screenshot it." There is no link onward to the game, and no mention of which armband colour the player has been assigned (A1-Alice's slot 45 carries `armbands: black`) — the header covers the hat, but not the armband. Colour names are also lower-cased here ("blue", "grey") where every other screen title-cases them.

Where

`react-ui/src/PickOutfit.js:327-355`.

Why it matters for the 19th

probably intentional — the armband is handed out at the door and the pick happens before the game — but if the intent is "screenshot this and show it at the door", the screenshot is missing the one thing the door staff have to hand over.

The zero-shots case is not reachable on the shared container

QUESTION *found by agent A11*

Downloading all shot images as a zip.

What happens

the dump is system-wide (`get_all_shot_ids()` has no game filter), and the shared database has 40+ shots from other agents, so I could not exercise the empty case without a reset — which the brief forbids. By inspection `zip_shot_images` would return a valid but empty archive with HTTP 200, and `adminDownload` would save a 22-byte `shot_images.zip` with no message distinguishing it from a failure.

Where

`backend/postprocess_shot_images.py:52-68`.

Why it matters for the 19th

minor, but "I pressed it and got an empty file" is indistinguishable from "it silently broke".

The version is invisible to players, and to a logged-out admin

QUESTION *found by agent A12*

The running app version shown on admin pages, and that it matches what is actually deployed.

What happens

the footer is inside `AdminPage`'s authenticated branch, so it does not render on the login form, and no player-facing page shows a version at all. A player reporting "it did X on my phone" cannot say which build.

Where

`react-ui/src/AdminCommon.js:189-206, 210-245`.

Why it matters for the 19th

it is the players' screenshots you will be triaging at 11pm, not the admin's. Worth deciding deliberately rather than by omission.

In "screened" mode the schema box does not apply to turn 1, and the follow-up wording is fixed

QUESTION *found by agent A12*

The replay workbench at /admin/replay (R1) — trialling a prompt/zoom/schema change against real shots without it touching stored data.

What happens

with a custom prompt and the default (screened) shape, the transcript shows the model asked my prompt but forced into the pipeline's own screening schema (`{"person_fills_less_than_half": false}`), then a fixed second user turn the workbench cannot edit: *"The person fills at least half of the screen… Now answer in full with the JSON described above."* Only the final turn uses the schema in the box. This is arguably correct — "screened" is defined as the live shape, and R1's answer is to pick `single` or `upfront` when you are trialling a different question — but the page says the reply is "forced into" the schema you typed, and for two of the up-to-three turns it is not. Nothing in the UI hints at it; you find out by opening the transcript.

How to reproduce

`/admin/replay`, leave the shape on "Screened zoom (live pipeline)", replace the prompt, replay, open "Full model transcript".

Where

`backend/shot_vision.py:945` and `:980` (`build_screening_schema()` on the first call) and the `ZOOM_FOLLOW_UP` / `FULL_READING_REQUEST` constants around `backend/shot_vision.py:430-457`.

Why it matters for the 19th

the roadmap's own warning is that a prompt edited alone is a prompt overruled. The remaining hard-coded turns are a smaller version of the same trap, and worth one sentence of UI copy rather than a code change.

Nothing in the clear flow frees the *wardrobe* back in a useful way

QUESTION *found by agent A2*

An admin clearing a player's outfit so they can pick again.

What happens

`clear_identity` nulls slot, overrides *and* wardrobe, so the player has to re-tick every garment they own on `/pick` before they can even see options. On the night that is ~14 taps re-entered by a player who already did it once.

Where

`backend/identity_admin.py:301` (`clear_identity`) → `UserInterface.clear_identity`.

Why it matters for the 19th

minor friction, but it is friction at the queue-forming moment (the door). Worth deciding deliberately rather than by accident — keeping the wardrobe and clearing only slot+overrides would make the re-pick two taps.

Only a distance-0 override is gated; distance-1 is written silently

QUESTION *found by agent A2*

The identity workbench / AdminIdentity.js: viewing a player's effective appearance and any overrides, and recording an override for a misdressed player so they stay distinguishable from their teammates.

What happens

`_validate_slot_assignment` refuses only when `overlap_distance == 0`. An override that leaves two players **one** misread apart ("danger" in the report's own vocabulary) saves with no confirmation at all; you find out afterwards from the warnings list, which you have to scroll up to see.

Where

`backend/identity_admin.py:268-270`.

Why it matters for the 19th

the report already grades d=1 as "a single misread confuses them", which is the failure the code exists to prevent. Worth deciding whether that deserves the same Force gate as d=0, or at least an inline "this will put you at d=1 with A2-Dan" note next to Save before you commit.

Geolocation watch errors spam the console when the fix changes

QUESTION *found by agent A5*

The ticker feed for game announcements.

What happens

`Error watching position: GeolocationPositionError` is logged repeatedly whenever the Playwright fake GPS is moved, from `MapView.js`'s `watchPosition` error handler. The map still updates correctly on the next good fix, and nothing user-visible breaks.

Why it matters for the 19th

I believe this is the headless fake-geolocation provider, not the app — but the same handler only `console.error`s, so on a real phone a GPS dropout is silent to the player and their dot simply stops moving. Worth knowing when someone says "the map froze".

A player newly convicted by someone else's upheld appeal has no recourse

QUESTION *found by agent A6*

Appealing a resolved shot as either shooter or target (R8), including seeing the appeal budget run out.

What happens

A1 was ruled a hit on Bob; Bob appealed; the admin re-ruled it a hit on **Carol**. Carol got the private line "You were hit by A6-Alice!" with the shot id and the shot now sits in her received list — but `appeal_state` is `upheld`, so `appeal_refusal` returns "The referee has already ruled on an appeal against this shot" and her Appeal button never appears. The same happens on A3, where Alice's appeal turned a miss into a hit on Bob. Verified in `user_shots_received`: `{"appeal_state":"upheld","can_appeal":false}`.

How to reproduce

rule a shot a hit on X, have X appeal, re-rule it a hit on Y, then look at Y's phone.

Where

`backend/user_interface.py:175`, `backend/admin_interface.py:1225`.

Why it matters for the 19th

R8's own error table says the wrong-target case is one of the core things appeals exist to fix, and the fix creates a fresh victim who is told they were hit and has no way to say otherwise. "Resolutions are terminal" was a deliberate decision, so this may be intended — but the person it binds never got a turn, which is different from the shooter/target pair the scheme reasons about. Worth a deliberate yes/no before the night.

The zoom tag says a zoom was spent but the admin cannot see what the model was shown

QUESTION *found by agent A9*

The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.

What happens

"Zoomed in ×2" tells the admin the model looked at two magnified crops. The crops exist and are served — `admin_get_shot_vision_images` — but the queue never fetches them; only the replay workbench (`react-ui/src/ShotReplay.js:143`) does.

Why it matters for the 19th

when CharlesBot names a player off a zoom, the admin's check is "did it zoom onto the right person?", and that is exactly the frame they cannot see. Possibly deliberate (the queue is meant to be fast); worth a decision rather than a fix.

Recorded, not a defect (1)

Facts worth having on the record.

Loot QRs *do* survive a reset — they can safely be printed and hidden in advance

NOTE *found by agent A13*

Cuts across both: confirm the game can be reset (resetdb) and replayed from a clean state without any of the above breaking, since that is exactly what happens between the dry run and the night itself.

What happens

an item QR carries its whole payload — id, type, data, flags, signature — in the URL (`backend/items.py`, `ItemModel`). The `items` row is only created *at collection*, purely to stop a second pickup (`backend/user_interface.py:747`). So a reset removes the "already collected" record and the printed code simply becomes collectable again. Verified: an ammo QR minted before `resetdb` returned `200` from `collect_item` after it, and gave 5 rounds.

Why it matters for the 19th

the loot can be printed, laminated and hidden days before the night, and re-used across the dry run and the game itself. Only the *join* codes are perishable. Worth saying out loud because the two artefacts look identical to whoever is doing the printing.

Evidence by checklist line

The screenshots each agent cited as evidence for a whole checklist line, rather than for one finding.

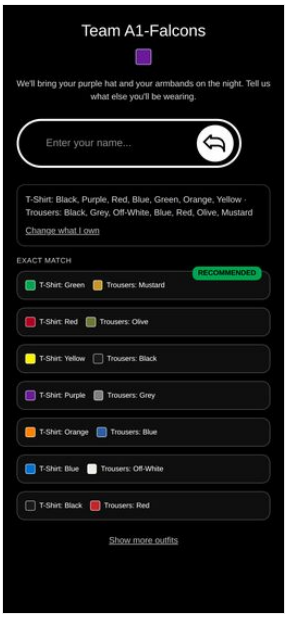
Join a team via QR/link and pick an outfit at /pick (#10): ranked outfit list, canonical-first ordering, colour swatches, pagination.

Agent A1 — WORKS WITH CAVEATS

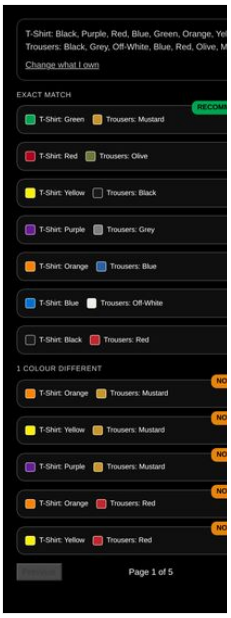
A1_01_after_join.png

A1_02_pick_wardrobe_top.png

A1_04_pick_wardrobe_bottom.png



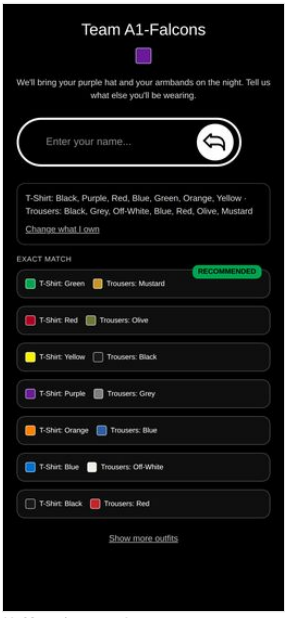
A1_06_options_page1_top.png



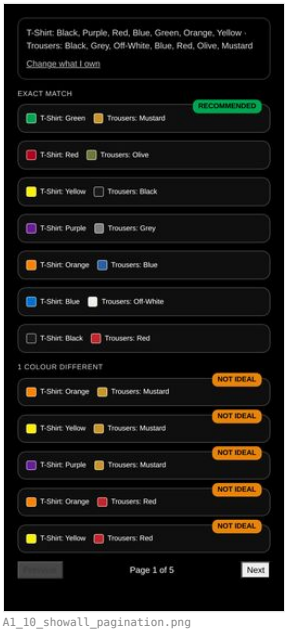
A1_10_showall_pagination.png

The "recommended" vs "not ideal" outfit badges and the "show non-recommended outfits" reveal link.

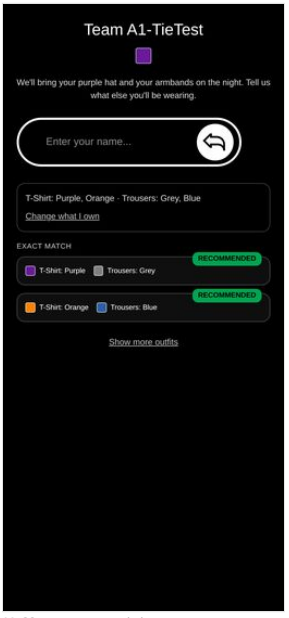
Agent A1 — WORKS



A1_06_options_page1_top.png



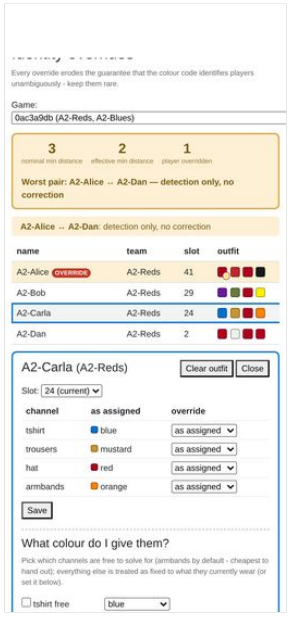
A1_10_showall_pagination.png



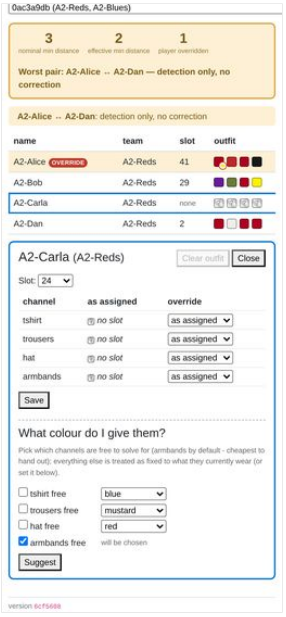
A1_28_two_recommended.png

An admin clearing a player's outfit so they can pick again.

Agent A2 — WORKS WITH CAVEATS

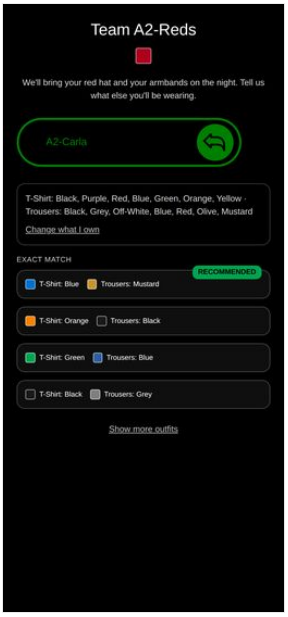


A2-clear-01-carla-before.png

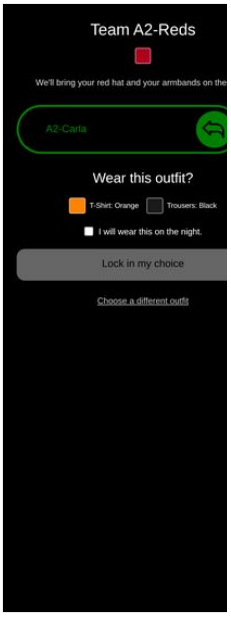


A2-clear-02-carla-after.png

A2-pick-01-landing.png



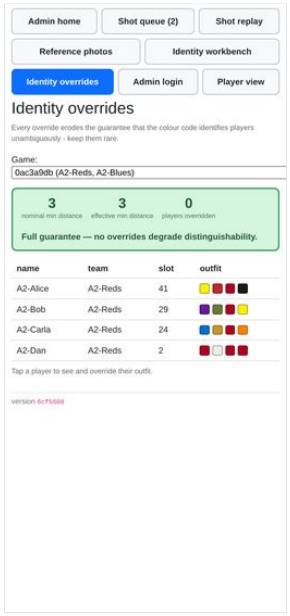
A2-pick-02-options.png



A2-pick-03-confirm.png

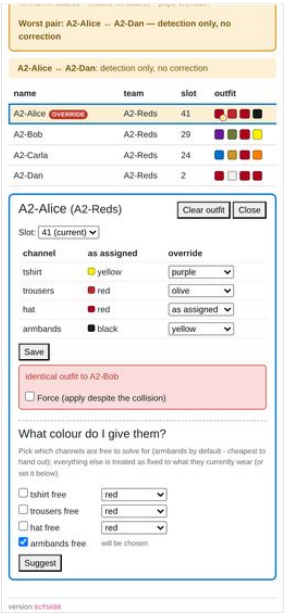
The identity workbench / AdminIdentity.js: viewing a player's effective appearance and any overrides, and recording an override for a misdressed player so they stay distinguishable from their teammates.

Agent A2 — WORKS WITH CAVEATS — the substance is right, the mobile layout is not



A2-identity-02-mygame.png

A2-identity-05b-override-saved-top.png



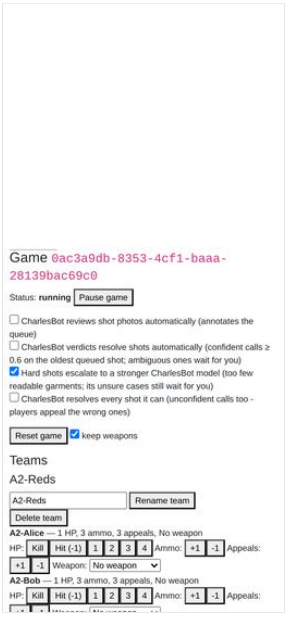
A2-identity-06-collision-refused.png

A2-identity-07-collision-forced.png

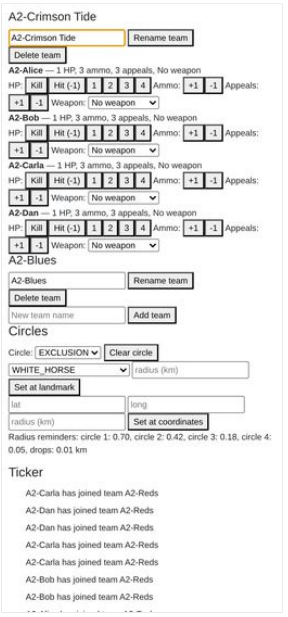
A2-identity-08-suggest.png

Renaming a team from the admin dashboard.

Agent A2 — WORKS

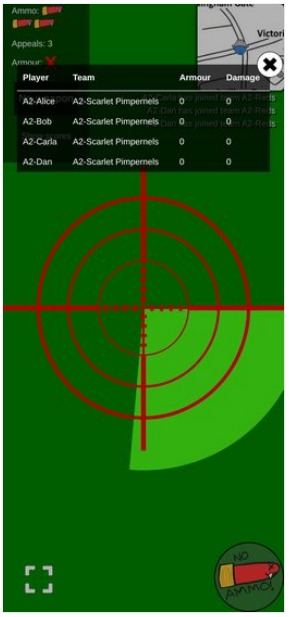


A2-rename-01-admin-game.png



A2-rename-03-after.png

A2-rename-05-carla-scoreboard.png

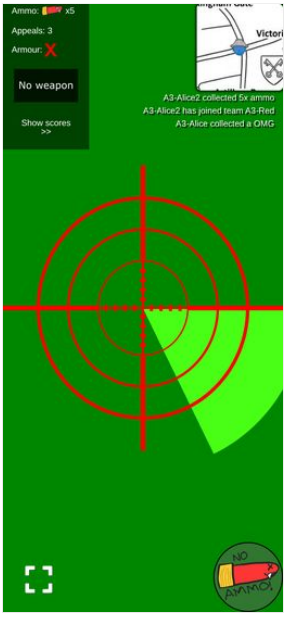


A2-rename-06-carla-live-update.png

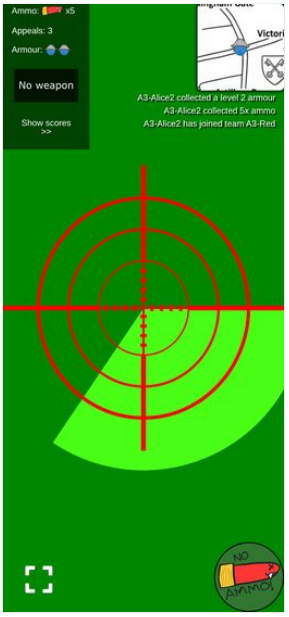
Scanning a loot QR code to pick up an item, and using it (weapons, armour, ammo, medpacks).

Agent A3 — WORKS WITH CAVEATS

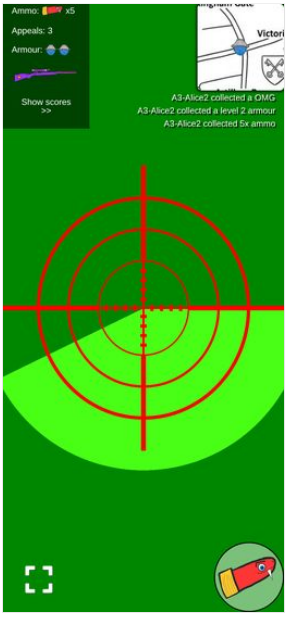
a3_40_camera_sees_qr.png



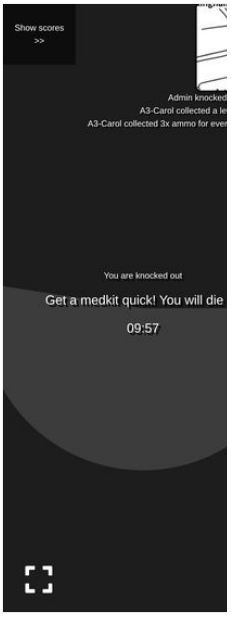
a3_11_ammo_after.png



a3_12_armour_after.png



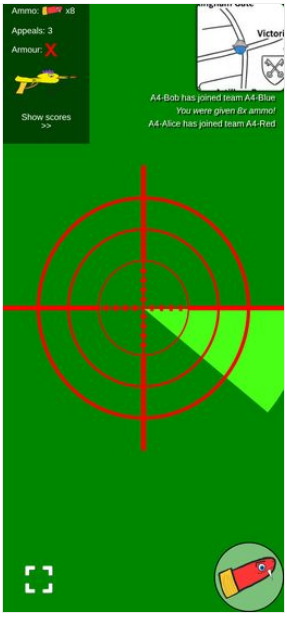
a3_13_omg_after.png



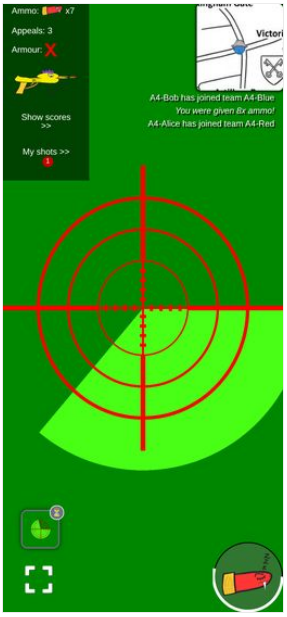
a3_24_bob_knocked_out.png

Taking a shot photo of another player, including the on-screen crosshair overlay.

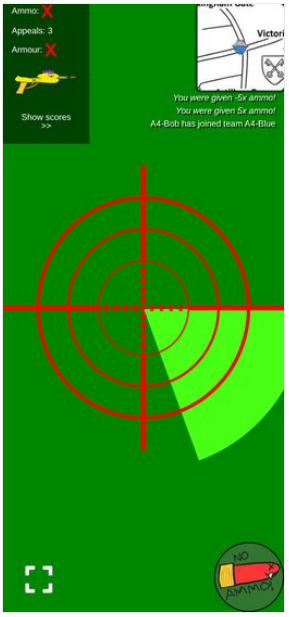
Agent A4 — WORKS WITH CAVEATS



a4_01_usermode.png



a4_02_cooldown.png

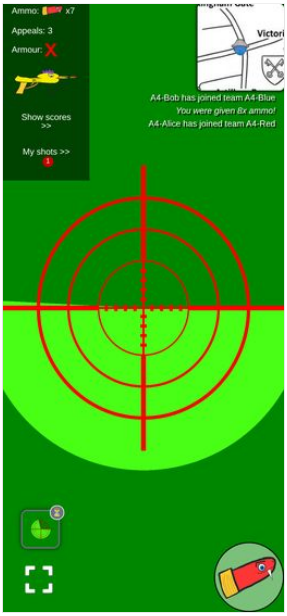


a4_05_no_ammo.png

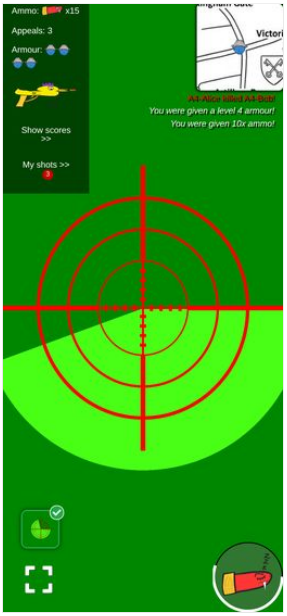
a4_70_knocked_out.png

The shot status bubble after taking a shot — every visual state it can be in, and that it stays on screen rather than disappearing.

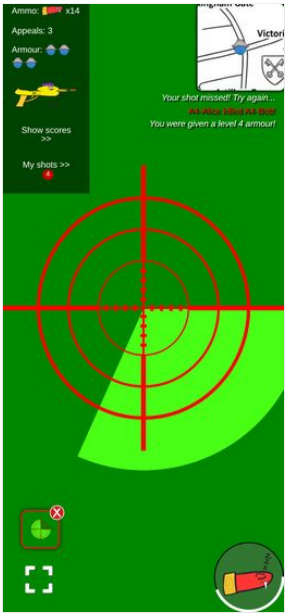
Agent A4 — WORKS WITH CAVEATS



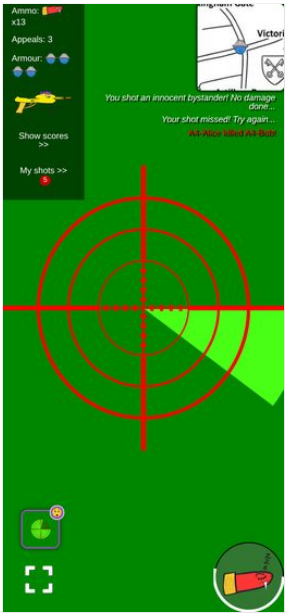
a4_03_bubble_pending.png



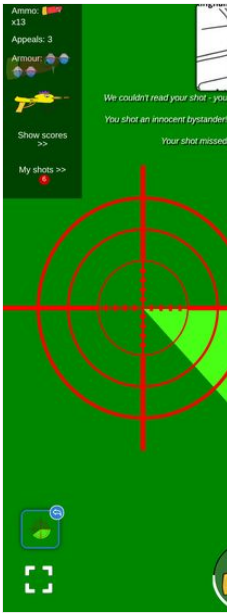
a4_10_bubble_hit.png



a4_11_bubble_miss.png



a4_12_bubble_bystander.png

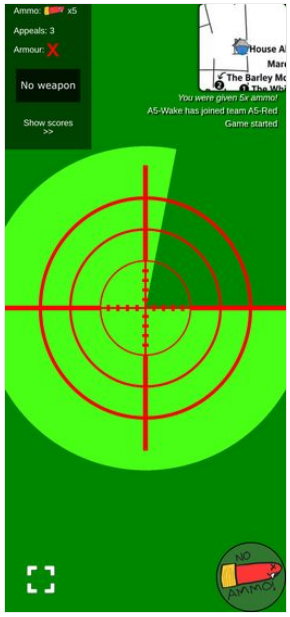


a4_13_bubble_refunded.png

The screen staying awake during play (R3 — Screen Wake Lock), and that the phone's own lock button still turns the screen

off on request.

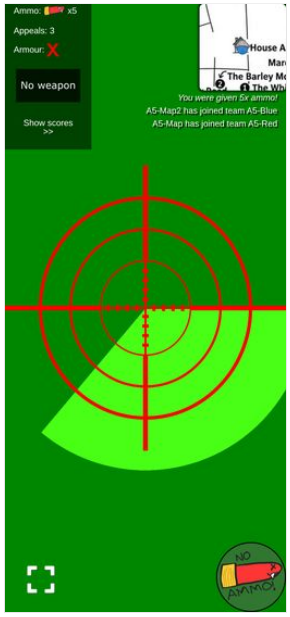
Agent A5 — WORKS WITH CAVEATS (the "lock button" half is NOT TESTABLE HERE)



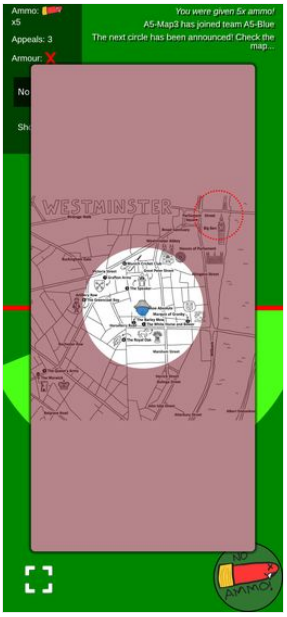
a5_wake_playing.png

The map view, including drop/circle locations on the live Westminster venue map (#12).

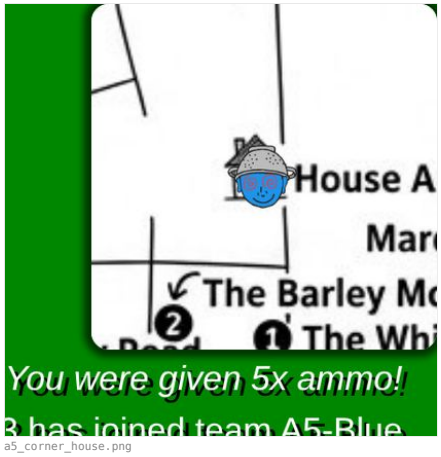
Agent A5 — WORKS WITH CAVEATS



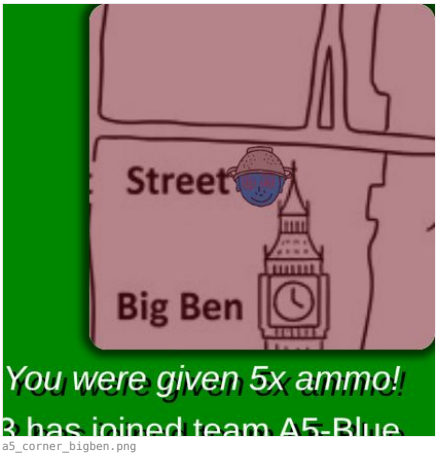
a5_map_main_view.png



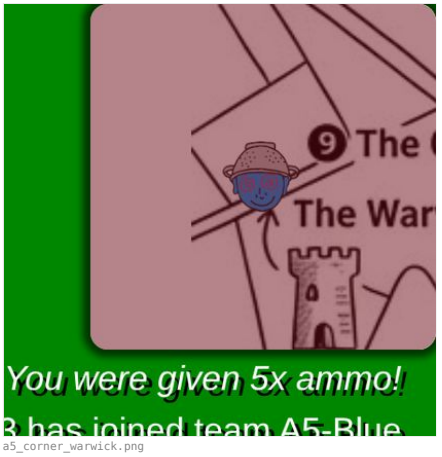
a5_popped_all_circles.png



a5_corner_house.png

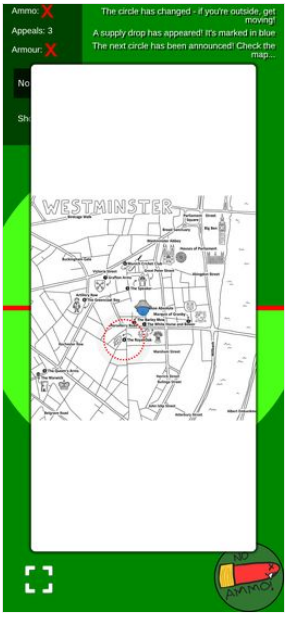


a5_corner_bigben.png



a5_corner_warwick.png

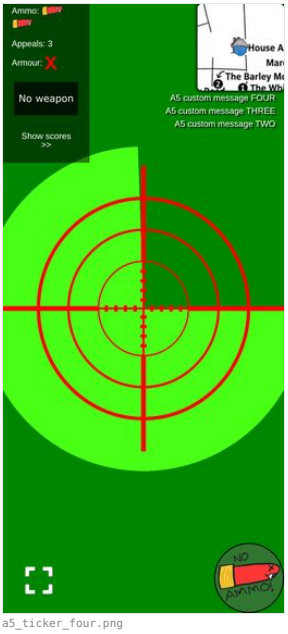
a5_corner_



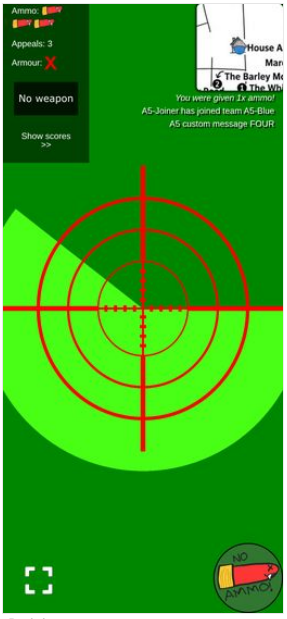
a5_clear_exclusion.png

The ticker feed for game announcements.

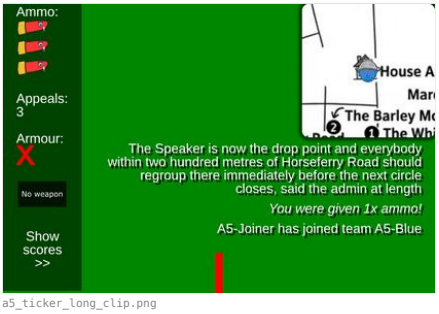
Agent A5 — WORKS



a5_ticker_four.png



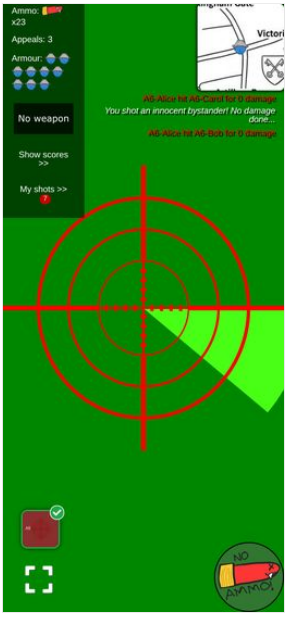
a5_ticker_events.png



a5_ticker_long_clip.png

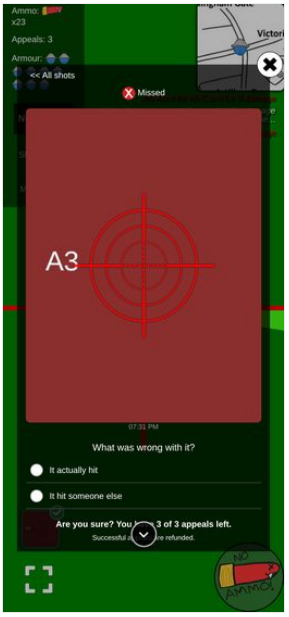
Appealing a resolved shot as either shooter or target (R8), including seeing the appeal budget run out.

Agent A6 — WORKS WITH CAVEATS



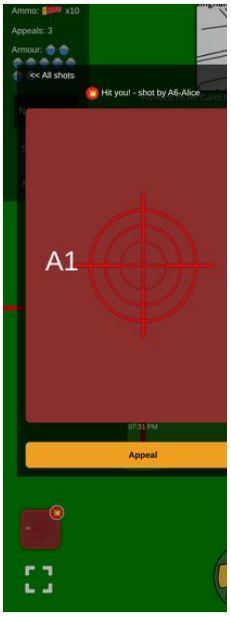
a6_01_alice_hud.png

a6_03_alice_A3_detail_top.png



a6_04_alice_A3_confirm.png

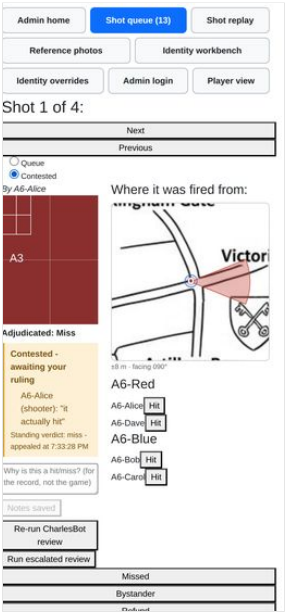
a6_06_alice_A3_after_appeal.png



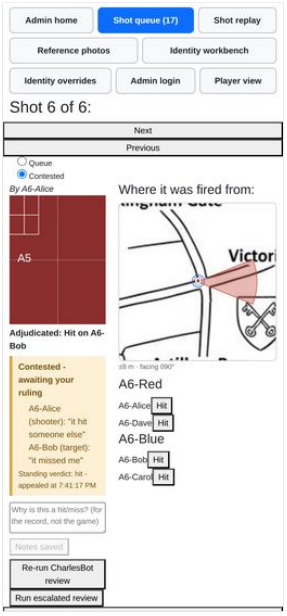
a6_08_bob_A1_detail.png

The contested-shots list an appeal reopens (R8), separate from the main queue.

Agent A6 — WORKS



a6_13_admin_queue_contested.png

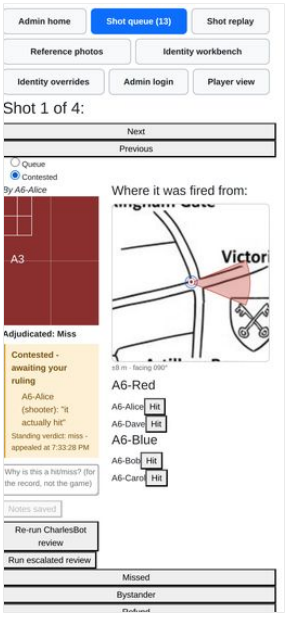


a6_28_admin_A5_both_appeals.png

Notes that are not mine to file

Agent A6 — see findings

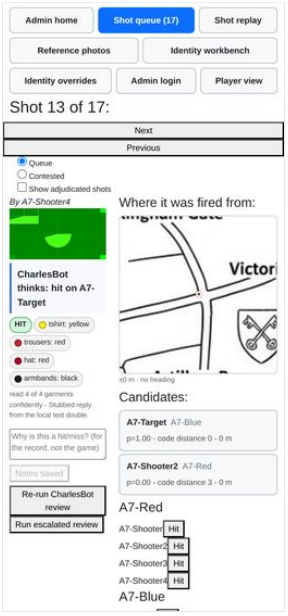
a6_38_devserver_overlay.png



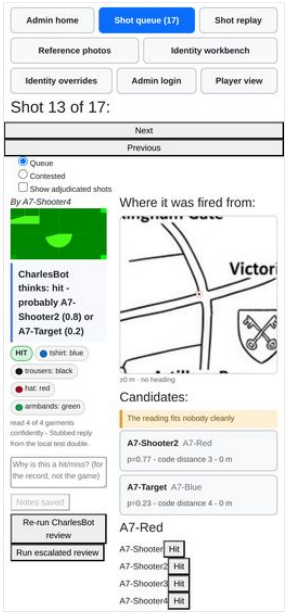
a6_13_admin_queue_contested.png

`User-facing copy says "CharlesBot", never "AI", anywhere a verdict or explanation is shown (#1, #2 — "CharlesBot thinks: hit on *name*").`

Agent A7 — WORKS WITH CAVEATS

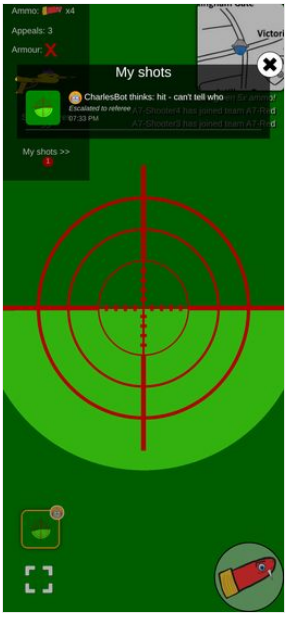


a7_queue_named3.png

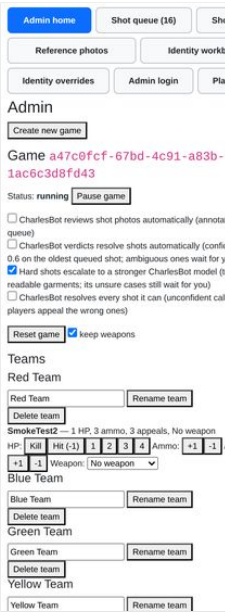


a7_queue_named.png

a7_queue_myshot.png



a7_player_list.png



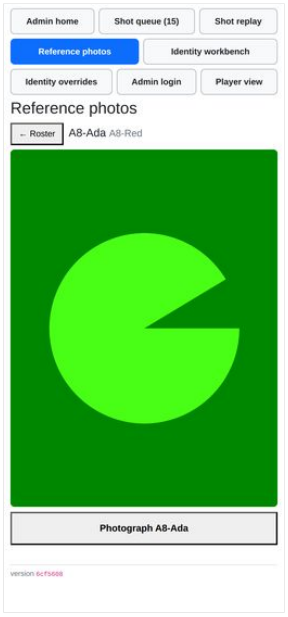
a7_admin_home.png

The reference-photo kit check at /admin/reference (R7): capturing a player's photo at the door and reading the resulting

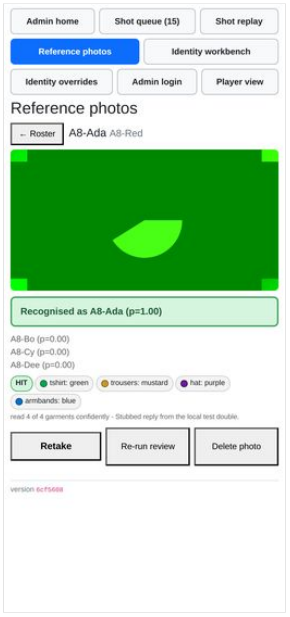
verdict, including the "unreadable photo identifies nobody" case.

Agent A8 — WORKS WITH CAVEATS

a8_02_roster_mygame.png



a8_03_capture_view.png



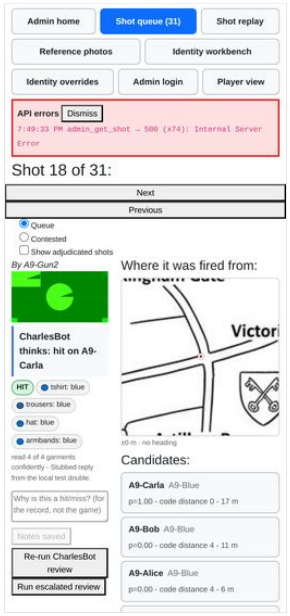
a8_04_verdict_match.png

a8_06_verdict_wrong_person.png

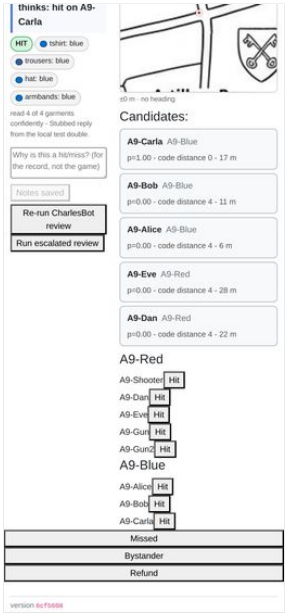
a8_07_verdict_ambiguous.png

The shot review queue: ranked candidate list per shot (#3), CharlesBot's verdict and confidence, the AI vision zoom usage indicator, and manually approving/rejecting a shot.

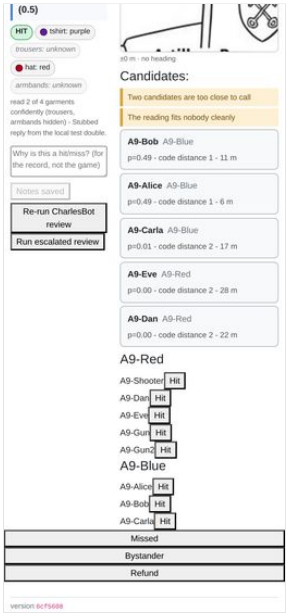
Agent A9 — WORKS WITH CAVEATS



a9_queue_exactmatch_top.png

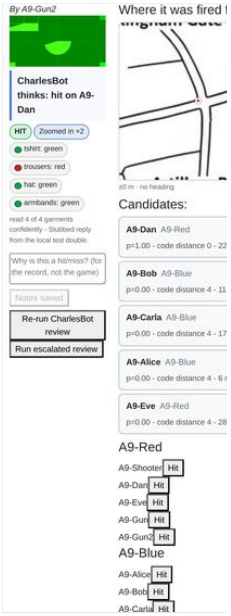


a9_queue_exactmatch_candidates.png



a9_queue_ambiguous_candidates.png

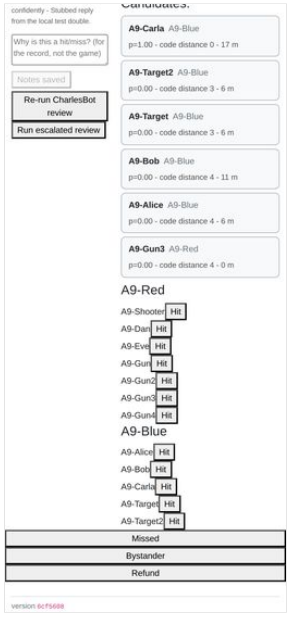
a9_queue_unreadable.png



a9_queue_zoom_on.png

Recording a "hit a bystander" outcome on a shot.

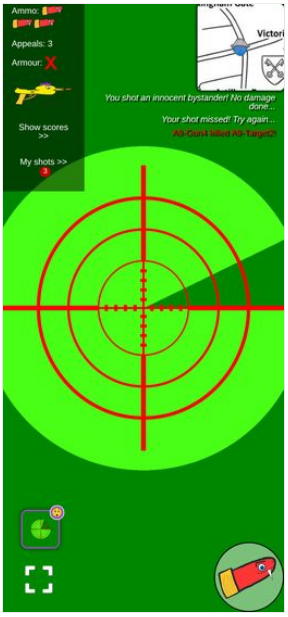
Agent A9 — WORKS



a9_adj_before_bystander.png

a9_queue_adjudicated_bystander.png

a9_player_shot_history.png

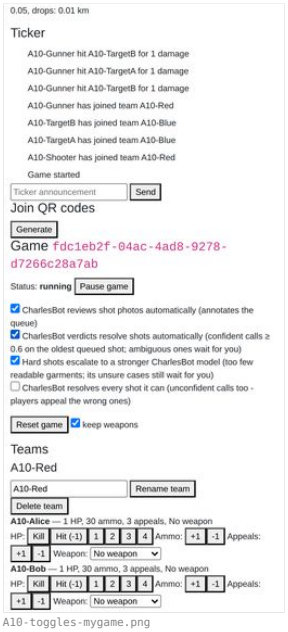


a9_player_shooter_after.png

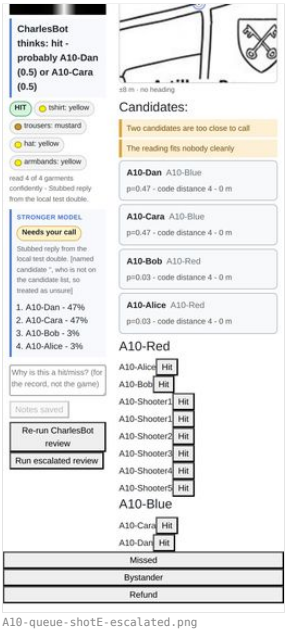
Toggling ai_shot_review_enabled, ai_auto_actions_enabled, ai_escalation_enabled and ai_resolve_everything_enabled, and confirming each one actually changes queue behaviour (not just the toggle state)

Agent A10 — WORKS (all four gates observably change behaviour, not just state)

A10-admin-top.png



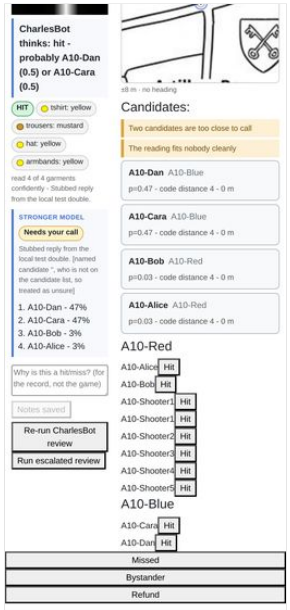
A10-toggles-mygame.png



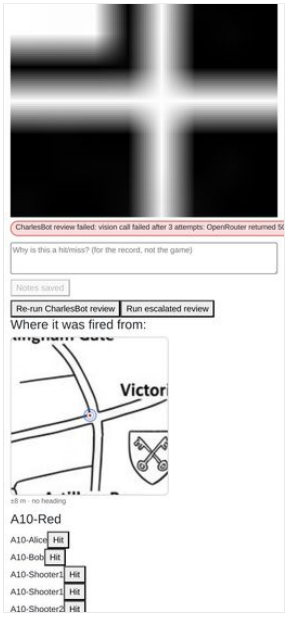
A10-queue-shotE-escalated.png

Running an escalated review by hand on a shot (#11 — "Run escalated review")

Agent A10 — WORKS WITH CAVEATS



A10-queue-shotE-escalated.png

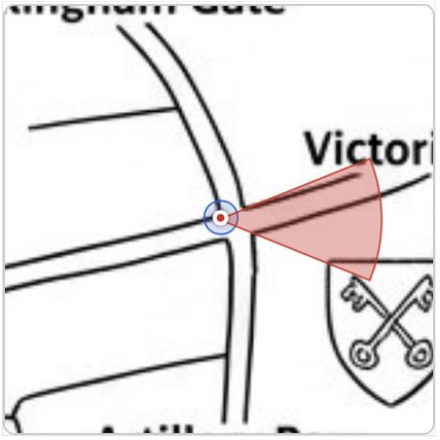


A10-queue-shotF-escalate-alert.png

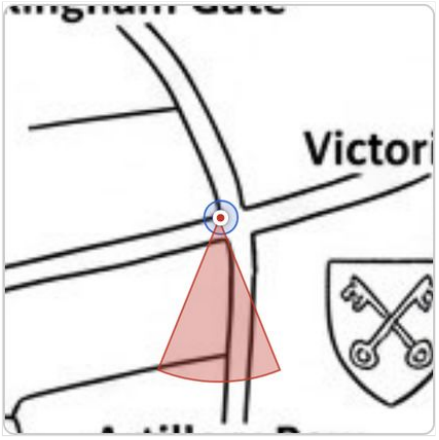
The per-shot map (R5) showing GPS accuracy and the shooter's compass heading.

Agent A11 — WORKS

A11-map-h000-acc8.png

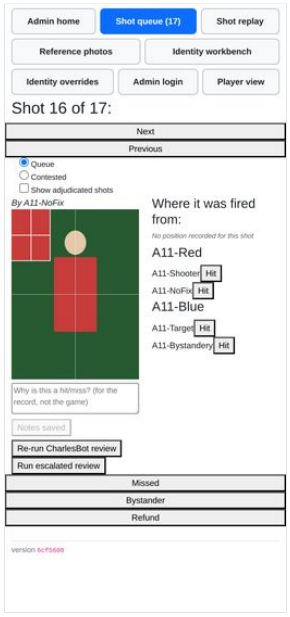


A11-map-h090-acc8.png

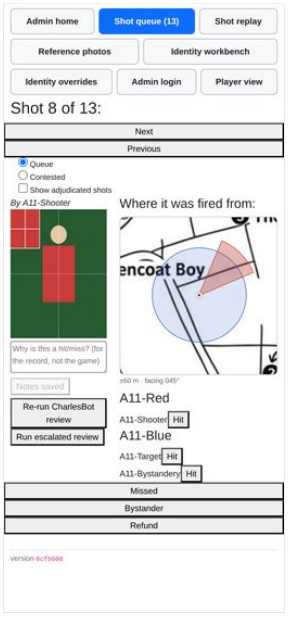


A11-map-h180-acc8.png

A11-map-h…



A11-queue-nofix.png

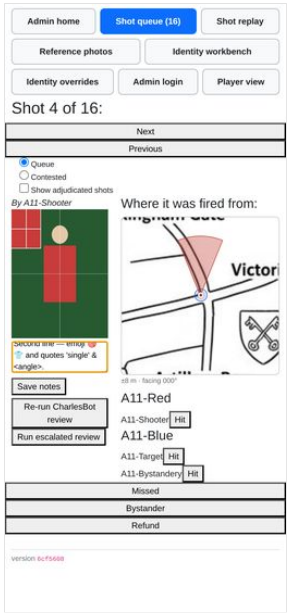


A11-queue-h045-acc60.png

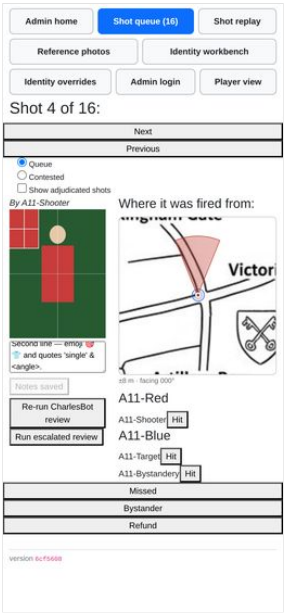
Admin shot history and notes on a player.

*Agent A11 — WORKS WITH CAVEATS — the **notes** half works well; the **history "on a player"** half does not exist. What ships is a global, one-at-a-time pager, and the word "player" in this*

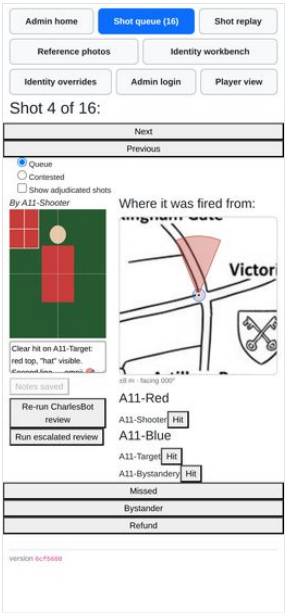
checklist line has no implementation behind it.



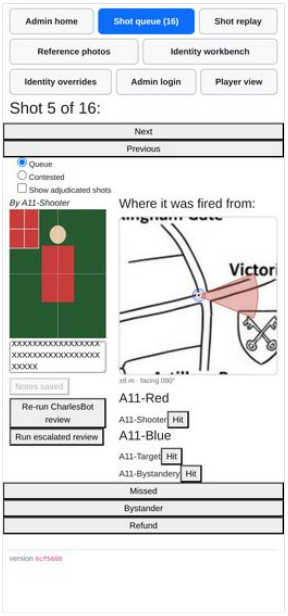
A11-notes-typed.png



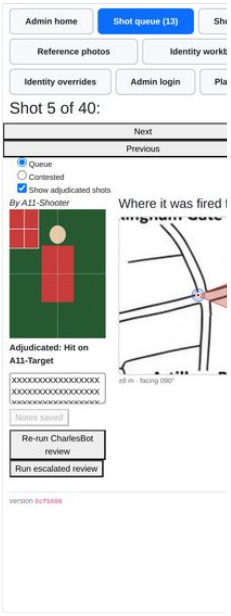
A11-notes-saved.png



A11-notes-after-reload.png



A11-notes-long.png



A11-history-view.png

Downloading all shot images as a zip.

Agent A11 — WORKS WITH CAVEATS

A11-zip-button.png

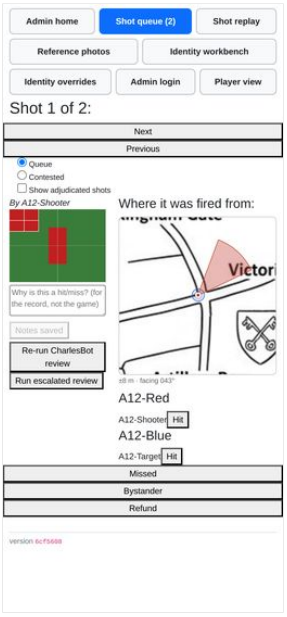
A11-zip-during.png

A11-zip-after.png

The admin nav (finger-sized button row) on a real phone screen, not just a desktop browser.

Agent A12 — WORKS WITH CAVEATS

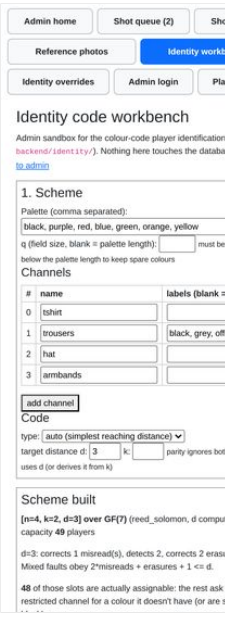
A12-nav_admin.png



A12-nav_admin_shots.png

A12-nav_admin_replay.png

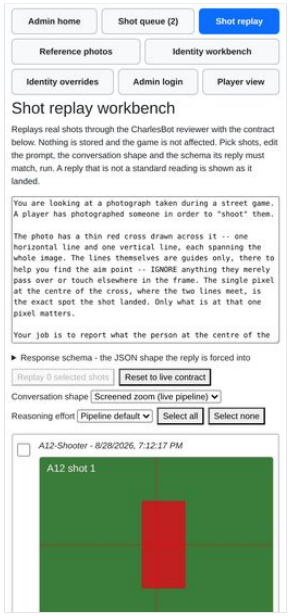
A12-nav_admin_reference.png



A12-nav_admin_identity.png

The replay workbench at /admin/replay (R1) — trialling a prompt/zoom/schema change against real shots without it touching stored data.

Agent A12 — WORKS

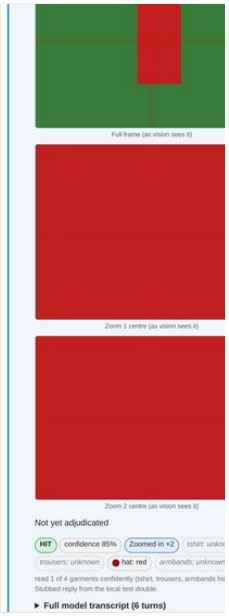


A12-replay-load.png

A12-replay-screened.png

A12-replay-upfront.png

A12-replay-single.png

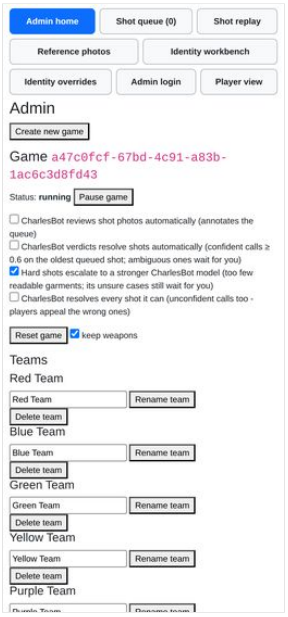


A12-replay-screened-zoomed.png

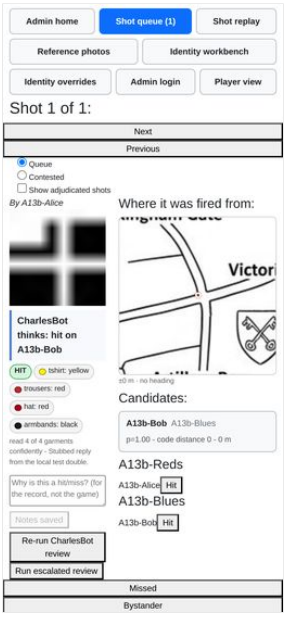
Cuts across both: confirm the game can be reset (resetdb) and replayed from a clean state without any of the above

breaking, since that is exactly what happens between the dry run and the night itself.

Agent A13 — WORKS WITH CAVEATS



a13_07_post_admin_home.png



a13_12_replay_admin_queue.png